PENGEMBANGAN APLIKASI EDUKATIF UNTUK KLASIFIKASI MOTIF BATIK PEKALONGAN MENGGUNAKAN *CONVOLUTIONAL NEURAL NETWORK* (CNN) ARSITEKTUR MOBILENETV2

SKRIPSI

Diajukan sebagai Pedoman Pelaksanaan Penelitian Tugas Akhir
Pada Jurusan Informatika Fakultas Teknik
Universitas Jenderal Soedirman

Disusun Oleh:

MUHAMMAD KHADZIQ

H1D022059

KEMENTERIAN PENDIDIKAN TINGGI, SAINS, DAN TEKNOLOGI

UNIVERSITAS JENDERAL SOEDIRMAN

FAKULTAS TEKNIK

PROGRAM STUDI INFORMATIKA

PURBALINGGA

2026

LEMBAR PENGESAHAN

LAPORAN TUGAS AKHIR

PENGEMBANGAN APLIKASI EDUKATIF UNTUK KLASIFIKASI MOTIF BATIK PEKALONGAN MENGGUNAKAN *CONVOLUTIONAL NEURAL NETWORK* (CNN) ARSITEKTUR *MOBILENETV2*

Oleh:

Muhammad Khadziq

H1D022059

Diajukan untuk memenuhi salah satu persyaratan memperoleh gelar
Sarjana Komputer pada Jurusan Informatika
Fakultas Teknik
Universitas Jenderal Soedirman

Diterima dan disetujui

Pada tanggal: 22 Mei 2026

Dosen Pembimbing I	**Dosen Pembimbing II**
Dr. M. Agus Syamsul Arifin, S.St., M.Kom.	**Devi Astri Nawangnugraeni, S.Pd., M.Kom.**
NIP. 198312022015041001	**NIP. 199312042024062004**

Mengetahui,
Dekan Fakultas Teknik

Prof. Dr. Eng. Ir. Agus Maryoto S.T., M.T., IPU., ASEAN Eng.
NIP. 197109202006041001

MOTTO

"Man Jadda Wajada"

"Barangsiapa bersungguh-sungguh akan berhasil"

PERSEMBAHAN

Puji syukur ke hadirat Allah SWT atas segala rahmat, kekuatan, dan kemudahan yang diberikan. Tanpa ridho-Nya, penyusunan tugas akhir ini hanyalah sebuah angan yang tak akan pernah terwujud menjadi nyata.

Karya sederhana ini saya persembahkan sebagai wujud bakti dan rasa terima kasih yang tak terhingga kepada Bapak dan Ibu. Terima kasih atas setiap doa yang dipanjatkan, keringat yang menetes demi pendidikan saya, serta dukungan tanpa henti yang selalu menjadi alasan saya untuk bangkit setiap kali merasa lelah.

KATA PENGANTAR

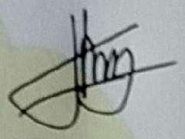
Puji syukur kehadirat Allah SWT atas segala rahmat dan karunia-Nya, sehingga penulis dapat menyelesaikan penelitian yang berjudul Pengembangan Aplikasi Edukatif untuk Klasifikasi Motif Batik Pekalongan menggunakan *Convolutional Neural Network* (CNN) Arsitektur *MobilenetV2*. Laporan ini disusun sebagai salah satu syarat untuk menempuh Tugas Akhir pada Jurusan Informatika, Fakultas Teknik, Universitas Jenderal Soedirman.

Dalam penyusunan laporan ini, penulis menyadari bahwa tanpa bantuan, dukungan, dan bimbingan dari berbagai pihak, penyelesaian laporan ini tidak akan dapat terlaksana dengan baik. Oleh karena itu, penulis ingin menyampaikan terima kasih kepada:

1. Allah SWT, atas segala rahmat, hidayah, dan kemudahan yang diberikan selama proses penyusunan.
2. Bapak Prof. Dr. Eng. Ir. Agus Maryoto, ST., M.T., IPU., ASEAN Eng., selaku Dekan Fakultas Teknik, Universitas Jenderal Soedirman.
3. Bapak Dr. Ir. Lasmedi Afuan, S.T., M.Cs., IPM., selaku Ketua Jurusan Informatika, Fakultas Teknik, Universitas Jenderal Soedirman.
4. Ibu Ir. Nur Chasanah, S.Kom., M.Kom. dan Ibu Ir. Nofiyati, S.Kom., M.Kom., selaku dosen pembimbing akademik yang telah memberikan bimbingan dan arahan selama perkuliahan.
5. Bapak Ir. Dadang Iskandar, S.T., M.Eng selaku Dosen Pembimbing I, Bapak Dr. M. Agus Syamsul Arifin, S.St., M.Kom. selaku pengganti Dosen Pembimbing I, dan Ibu Devi Astri Nawangnugraeni, S.Pd., M.Kom selaku Dosen Pembimbing II yang dengan penuh perhatian telah memberikan bimbingan dan arahan selama penyusunan praproposal ini.
6. Orang tua dan keluarga penulis, yang selalu memberikan motivasi, dukungan, doa dan ridhonya.
7. Rekan-rekan seperjuangan di Jurusan Informatika angkatan 2022, serta seluruh pihak yang telah memberikan dukungan dan masukan.

Penulis menyadari bahwa penyusunan laporan ini masih jauh dari kata sempurna karena keterbatasan pengetahuan dan pengalaman yang dimiliki. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun dari semua pihak demi perbaikan di masa mendatang. Semoga karya ini dapat berkontribusi pada pengembangan ilmu, khususnya dalam bidang pengolahan citra digital dan *Artificial Intelligence.*

Purbalingga, 25 April 2026



Muhammad Khadziq

DAFTAR ISI

LEMBAR PENGESAHAN ii
MOTTO iii
PERSEMBAHAN iv
KATA PENGANTAR v
DAFTAR ISI vii
DAFTAR GAMBAR xi
DAFTAR TABEL xiii
DAFTAR LISTING PROGRAM xiv
LEMBAR PENGESAHAN KEASLIAN SKRIPSI xvi
ABSTRAK xvi
ABSTRACT xvii
BAB I PENDAHULUAN 1
1.1. Latar Belakang 1
1.2. Rumusan Masalah 3
1.3. Pertanyaan Penelitian 4
1.4. Batasan Masalah 4
1.5. Tujuan Penelitian 5
1.6. Manfaat Penelitian 5
BAB II TINJAUAN PUSTAKA 7
2.1. Batik 7
2.1.1. Batik Pagi-Sore 7
2.1.2. Batik Jlamprang 8
2.1.3. Batik Terang Bulan 10
2.1.4. Batik Jawa Hokokai 11
2.1.5. Batik Tirta Teja 11
2.1.6. Batik Buketan 12
2.1.7. Batik Lunglungan 13
2.2. Klasifikasi 14

2.3. *Machine Learning* 14
2.4. *Deep Learning* 15
2.5. *Convolutional Neural Network* 16
2.5.1. *Convolutional Layer* 16
2.5.2. *Activation Function* 17
2.5.3. *Pooling Layer* 20
2.5.4. *Fully Connected Layer* 21
2.6. MobileNetV2 22
2.6.1. *Depthwise Separable Convolution* 22
2.6.2. *Inverted Residual Block* 23
2.6.3. *Linear Bottleneck* 23
2.7. *Confusion Matrix* 24
2.7.1. Akurasi 25
2.7.2. Presisi 25
2.7.3. *Recall* 25
2.7.4. *F1-score* 26
2.8. Python 26
2.9. Flutter 26
2.10. *Unified Modeling Language* 27
2.10.1. *Use Case Diagram* 28
2.10.2. *Sequence Diagram* 29
2.10.3. *Activity Diagram* 30
2.10.4. *Class Diagram* 31
2.11. *Software Development Life Cycle (SDLC)* 32
2.12. Metode *Waterfall* 32
2.12.1. Analisis Kebutuhan 33
2.12.2. Desain Sistem 33
2.12.3. Implementasi 34
2.12.4. Pengujian 34
2.12.5. Pemeliharaan 34
2.13. *Black Box Testing* 35

2.14. Penelitian Sejenis 35
BAB III 39
METODE PENELITIAN 39
3.1. Waktu dan Tempat Penelitian 39
3.2. Data dan Alat Penelitian 39
3.2.1. Data Penelitian 39
3.2.2. Alat Penelitian 40
3.3. Tahap Penelitian 40
3.3.1. Metode Pengumpulan Data 41
3.3.2. Metode Pengembangan Sistem 41
BAB IV HASIL DAN PEMBAHASAN 46
4.1. Analisis Kebutuhan 46
4.2. Desain Sistem 48
4.3. Implementasi 60
4.3.1. Pengumpulan Dataset 61
4.3.2. *Preprocessing* 61
4.3.3. Pembagian Dataset 63
4.3.4. Penanganan *Imbalance Data* 64
4.3.5. Perancangan Arsitektur Model 68
4.3.6. *Training* Model Awal 72
4.3.7. *Fine-Tuning* 76
4.3.8. Evaluasi Model 78
4.3.9. Implementasi Aplikasi *Mobile* 87
4.4. *Testing* 112
BAB V KESIMPULAN DAN SARAN 115
5.1. Kesimpulan 115
5.2. Saran 116
DAFTAR PUSTAKA 117

DAFTAR GAMBAR

Gambar 1. Batik Pagi-Sore 8
Gambar 2. Batik Jlamprang 9
Gambar 3. Batik Terang Bulan 10
Gambar 4. Batik Jawa Hokokai 11
Gambar 5. Batik Tirta Teja 12
Gambar 6. Batik Buketan 12
Gambar 7. Batik Lunglungan 13
Gambar 8. Arsitektur *Convolutional Neural Network* 16
Gambar 9. Proses *Convolution Layer* 17
Gambar 10. Transformasi Nilai Matriks pada Lapisan ReLU 18
Gambar 11. Grafik ReLU dan ReLU6 19
Gambar 12. Proses *Pooling Layer* 21
Gambar 13. Arsitektur MobileNetV2 22
Gambar 14. *Confusion Matrix* 24
Gambar 15. Alur Kerja Metode *Waterfall* 33
Gambar 16. *Flowchart* Penelitian 42
Gambar 17. *Use Case Diagram* Aplikasi Klasifikasi Batik Pekalongan 48
Gambar 18. *Activity Diagram* Klasifikasi Batik Pekalongan 49
Gambar 19. *Sequence Diagram* Klasifikasi Motif Batik 50
Gambar 20. *Sequence Diagram* Melihat Informasi Edukasi Batik 51
Gambar 21. *Sequence Diagram* Melihat Galeri Motif Batik 51
Gambar 22. *Sequence Diagram* Melihat Panduan Penggunaan 52
Gambar 23. *Class* Diagram Aplikasi Klasifikasi Batik 53
Gambar 24. Rancangan Antarmuka Halaman Menu Utama 57
Gambar 25. Rancangan Antarmuka Halaman Klasifikasi 57
Gambar 26. Rancangan Antarmuka Halaman Hasil Klasifikasi 58
Gambar 27. Rancangan Antarmuka Halaman Galeri 59
Gambar 28. Rancangan Antarmuka Halaman Detail Motif 59
Gambar 29. Rancangan Antarmuka Halaman Panduan Pengguna 60
Gambar 30. Hasil *Rescaling Image* 63

Gambar 31. Hasil Augmentasi Citra 67
Gambar 32. Hasil SMOTE 68
Gambar 33. *Confusion Matrix* CNN + SMOTE 79
Gambar 34. Kesalahan Klasifikasi Model CNN + SMOTE 80
Gambar 35. *Confusion Matrix CNN* + Augmentasi dan *Class Weight* 81
Gambar 36. Perbandingan *Confusion Matrix* 82
Gambar 37. Grafik Akurasi dan *Loss* pada Model CNN + SMOTE 83
Gambar 38. Grafik Akurasi dan *Loss* pada Model CNN Augmentasi 83
Gambar 39. Tampilan Halaman Utama 100
Gambar 40. Tampilan Halaman Galeri 103
Gambar 41. Tampilan Halaman Detail Galeri 105
Gambar 42. Tampilan Halaman Klasifikasi 108
Gambar 43. Tampilan Halaman Hasil Klasifikasi 110
Gambar 44. Tampilan Halaman Panduan 112

DAFTAR TABEL

Tabel 1. Simbol *Use Case Diagram* 28
Tabel 2. Simbol *Sequence Diagram* 29
Tabel 3. Simbol *Activity Diagram* 30
Tabel 4. Simbol *Class Diagram* 31
Tabel 5. Penelitian Sejenis 36
Tabel 6. Informasi Data Penelitian 39
Tabel 7. Spesifikasi Kebutuhan Pengguna 46
Tabel 8. Spesifikasi Kebutuhan Sistem 47
Tabel 9. Resolusi Citra Sebelum *Resizing* 61
Tabel 10. Hasil Akhir Setelah *Resizing* 62
Tabel 11. Hasil *Splitting Data* 64
Tabel 12. Hasil Pembobotan Kelas 66
Tabel 13. Hasil *Training* Model CNN 73
Tabel 14. *Extended Training* 73
Tabel 15. Hasil *Training* Model CNN + SMOTE 75
Tabel 16. Hasil *Fine-tuning* 77
Tabel 17. Perbandingan Performa Model 78
Tabel 18. *Classification Report* CNN + SMOTE 80
Tabel 19. *Classification Report* CNN + Augmentasi dan *Class Weight* 82
Tabel 20. Perbandingan Kedua Model 84
Tabel 21. Perbandingan Kedua Model Pada *Data Testing* 86
Tabel 22. *Classification Report* Model SMOTE Pada Data Test 86
Tabel 23. *Classification Report* Model Augmentasi + *Class Weight* Data Test ... 87
Tabel 24. Hasil Pengujian *Blackbox* 113

DAFTAR LISTING PROGRAM

Listing Program 1. *Pseudocode* Keseluruhan Model 55
Listing Program 2. *Pseudocode* Preprocessing Citra 56
Listing Program 3. *Pseudocode* Klasifikasi 56
Listing Program 4. *Image Resizing* dan *One-Hot Encoding* 61
Listing Program 5. *Rescaling Image* 62
Listing Program 6. *Splitting Dataset* 64
Listing Program 7. *Class Weight* 65
Listing Program 8. Augmentasi Data 67
Listing Program 9. *Oversampling* SMOTE 67
Listing Program 10. Perancangan Arsitektur CNN MobileNetV2 69
Listing Program 11. Perancangan Arsitektur CNN + SMOTE 70
Listing Program 12. Pelatihan Awal Model CNN 72
Listing Program 13. Pelatihan Awal Model CNN + SMOTE 74
Listing Program 14. *Fine-tuning* 77
Listing Program 15. Prediksi Pada Data *Testing* 78
Listing Program 16. *BatikModel* 88
Listing Program 17. *FaktaBatik* 89
Listing Program 18. *HasilPrediksiModel* 90
Listing Program 19. *ImageModel* 91
Listing Program 20. *PanduanModel* 92
Listing Program 21. Deklarasi *Class* 92
Listing Program 22. Inisialisasi *Database* 93
Listing Program 23. *GetAllBatik* 93
Listing Program 24. Deklarasi *Class TfLiteServices* 94
Listing Program 25. Inisiasi Model *TfLite* 95
Listing Program 26. Fungsi Prediksi 95
Listing Program 27. Proses *Output* 96
Listing Program 28. *Routing* dan Inisiasi Aplikasi pada *main.dart* 97
Listing Program 29. Implementasi Navigasi Menu pada *home_view.dart* 98
Listing Program 30. Menampilkan Daftar Batik dan Navigasi 99

Listing Program 31. Deklarasi *Class* Galeri 100
Listing Program 32. Inisiasi dan *Load* Data 101
Listing Program 33. Menampilkan Data Galeri..... 102
Listing Program 34. Deklarasi *Class* Detail Galeri 103
Listing Program 35. Menampilkan Data Detail Batik 104
Listing Program 36. Menampilkan Galeri Gambar 105
Listing Program 37. Fungsi Ambil Gambar..... 106
Listing Program 38. Proses Klasifikasi dan Navigasi..... 107
Listing Program 39. Implementasi Hasil Klasifikasi..... 109
Listing Program 40. Implementasi Halaman Panduan 111
Listing Program 41. Tampilan Detail Panduan.....111

LEMBAR PENGESAHAN KEASLIAN SKRIPSI

Saya yang bertanda tangan di bawah ini:

Nama	: Muhammad Khadziq
NIM	: H1D022059
Program Studi	: Informatika

Dengan ini saya menyatakan bahwa dalam skripsi ini tidak terdapat karya yang pernah diajukan untuk memperoleh gelar kesarjanaan di suatu Perguruan Tinggi, dan sepanjang pengetahuan saya juga tidak terdapat karya atau pendapat yang pernah ditulis atau diterbitkan oleh orang lain, kecuali yang secara tertulis diacu dalam naskah ini dan disebutkan dalam daftar pustaka.

Demikian pernyataan ini yang saya buat dengan sebenar-benarnya tanpa ada unsur paksaan dari pihak manapun.

Purbalingga, 23 Mei 2026



Muhammad Khadziq

H1D022059

ABSTRAK

PENGEMBANGAN APLIKASI EDUKATIF UNTUK KLASIFIKASI MOTIF BATIK PEKALONGAN MENGGUNAKAN *CONVOLUTIONAL NEURAL NETWORK* (CNN) ARSITEKTUR *MOBILENETV2*

Muhammad Khadziq
H1D022059

Kompleksitas visual motif Batik Pekalongan akibat akulturasi budaya menjadikan beberapa motif sulit dibedakan secara manual, sehingga diperlukan media pembelajaran digital yang interaktif. Penelitian ini bertujuan mengembangkan aplikasi edukatif berbasis *mobile* untuk identifikasi motif Batik Pekalongan. Metode yang digunakan adalah *Waterfall*, dimulai dari analisis kebutuhan berupa kebutuhan fungsional dan non fungsional sistem. Dilanjutkan tahap desain sistem menggunakan pemodelan UML mulai dari *use case, activity, sequence*, dan *class diagram*, hingga penyusunan *pseudocode sistem*. Tahap implementasi menggunakan CNN dengan *transfer learning* berbasis MobileNetV2 yang membandingkan model CNN + *augmentation* dan *class weight* dengan CNN + SMOTE. Selanjutnya tahap pengujian dilakukan dengan *blackbox testing* serta evaluasi performa model menggunakan *accuracy*, *precision, recall,* dan *F1-score*. Serta tahap terakhir yaitu dokumentasi sistem. Hasil penelitian dikembangkan aplikasi *mobile* yang mampu mengidentifikasi motif batik dan menyajikan informasi kepada pengguna, dengan model CNN + SMOTE memiliki performa lebih baik yang menghasilkan *accuracy* sebesar 0,8475, *precision* 0,8642, *recall* 0,8311, dan *F1-score* 0,8336, sedangkan model CNN dengan *augmentation* dan *class weight* memperoleh *accuracy* 0,8305, *precision* 0,8615, *recall* 0,8339, dan *F1-score* 0,8309. Berdasarkan hasil tersebut, model CNN + SMOTE dipilih sebagai model terbaik karena memberikan peningkatan performa klasifikasi secara keseluruhan. Diharapkan penelitian ini berkontribusi pada ilmu komputer dalam penerapan *deep learning* untuk klasifikasi citra budaya serta pengembangan aplikasi *mobile* sebagai media pelestarian warisan budaya di era digital.

Kata kunci: Aplikasi *Mobile*, Batik Pekalongan, *Convolutional Neural Network*, Klasifikasi Citra, MobileNetV2, *Synthetic Minority Oversampling Technique*.

ABSTRACT

DEVELOPMENT OF AN EDUCATIONAL APPLICATION FOR PEKALONGAN BATIK MOTIF CLASSIFICATION USING A CONVOLUTIONAL NEURAL NETWORK (CNN) WITH THE MOBILENETV2 ARCHITECTURE

Muhammad Khadziq
H1D022059

The visual complexity of Pekalongan batik motifs, resulting from cultural acculturation, makes some motifs difficult to distinguish manually, necessitating the use of interactive digital learning tools. This study aims to develop a mobile-based educational application for identifying Pekalongan batik motifs. The Waterfall methodology was employed, beginning with a needs analysis focusing on the system's functional and non-functional requirements. This was followed by the system design phase using UML modeling, starting with use case, activity, sequence, and class diagrams, up to the development of system pseudocode. The implementation phase utilized a CNN with transfer learning based on MobileNetV2, comparing the CNN + augmentation and class weight models with the CNN + SMOTE model. Subsequently, the testing phase was conducted using black-box testing, and model performance was evaluated using accuracy, precision, recall, and F1-score. The final stage is system documentation. The research resulted in the development of a mobile application capable of identifying batik patterns and presenting information to users. The CNN + SMOTE model demonstrated superior performance, achieving an accuracy of 0.8475, precision of 0.8642, recall of 0.8311, and an F1-score of 0.8336, while the CNN model with augmentation and class weights achieved an accuracy of 0.8305, precision of 0.8615, recall of 0.8339, and an F1-score of 0.8309. Based on these results, the CNN + SMOTE model was selected as the best model because it provided an overall improvement in classification performance. It is hoped that this research will contribute to computer science in the application of deep learning for cultural image classification as well as the development of mobile applications as a medium for preserving cultural heritage in the digital age.

Keywords: *Convolutional Neural Network, Image Classification, Mobile Application, MobileNetV2, Pekalongan Batik, Synthetic Minority Oversampling Technique.*

BAB I
PENDAHULUAN

1.1. Latar Belakang

Batik merupakan warisan budaya tak benda Indonesia yang telah diakui secara global oleh UNESCO pada tahun 2009 dan menjadi bagian integral dari identitas nasional (Fitri Widiastuti dkk., 2024). Berdasarkan data Kementerian Perindustrian Republik Indonesia, tercatat telah ada 5.849 motif batik yang tersebar di seluruh Indonesia pada tahun 2015 (Deni Akbar dkk., 2022). Setiap daerah produsen batik memiliki ratusan variasi motif dengan karakteristik unik yang terus berkembang, salah satunya adalah Pekalongan. Hal ini menunjukkan kekayaan kreativitas bangsa. Namun, kekayaan ini membawa tantangan di era digital, khususnya dalam hal pelestarian dan edukasi kepada masyarakat, terutama generasi muda. Metode pembelajaran konvensional seringkali kurang interaktif, sehingga diperlukan sebuah media pembelajaran digital yang menarik dan mudah diakses untuk mengenalkan nilai-nilai budaya ini.

Secara spesifik, pemilihan Batik Pekalongan memiliki kompleksitas visual yang tinggi sebagai akibat dari akulturasi budaya yang panjang. Motif 'Jlamprang', misalnya, merupakan hasil adaptasi dari tenun Patola India yang memiliki struktur geometris rumit, sehingga sering kali sulit dibedakan secara manual dengan motif Nitik dari Yogyakarta (Maziyah & Alamsyah, 2023). Selain itu, pengaruh budaya Eropa dan peranakan Tionghoa menciptakan variasi ornamen yang padat pada motif 'Buketan' dan 'Liong', yang menuntut ketelitian tinggi dalam proses identifikasi (Dewinta Sari Rahmaputri, 2023).

Sebagai respon terhadap tantangan-tantangan pelestarian dan edukasi tersebut, penelitian ini diarahkan untuk mengembangkan aplikasi edukatif. Aplikasi tersebut diimplementasikan dalam bentuk aplikasi *mobile*. Sistem ini diposisikan sebagai solusi integratif yang menggabungkan kecerdasan buatan untuk identifikasi motif dengan modul informasi budaya, sehingga dapat membantu masyarakat mengenali dan memahami nilai filosofis Batik Pekalongan secara lebih mudah dan akurat.

Fitur utama yang dikembangkan dalam aplikasi ini adalah kemampuan identifikasi motif secara otomatis. Hal ini menjadi penting mengingat proses identifikasi batik secara manual menghadapi berbagai tantangan, terutama karena kompleksitas visual motif Batik yang sangat tinggi (Andrianto dkk., 2025). Banyak motif menunjukkan variasi intra-kelas yang tinggi, satu jenis motif dapat memiliki beragam variasi visual, sementara beberapa motif yang berbeda justru bisa terlihat sangat mirip (Dewinta Sari Rahmaputri, 2023). Karakteristik ini membuat proses identifikasi manual menjadi sangat subjektif, lambat, dan sulit untuk diskalakan. Selain itu, proses klasifikasinya tidak hanya bergantung pada objek utama, tetapi juga pada detail *isen-isen* (motif pengisi) yang rumit, sehingga memerlukan sebuah pendekatan komputasional yang objektif dan efisien (Galih Pramuja Inngam Fanani dkk., 2025).

Untuk mengatasi tantangan tersebut, maka dapat memanfaatkan teknologi *deep learning*, khususnya *Convolutional Neural Network* (CNN). CNN merupakan algoritma yang menggabungkan proses ekstraksi ciri melalui konvolusi dengan proses klasifikasi menggunakan jaringan saraf (Misela dkk., 2023). Berbeda dengan metode *machine learning* klasik yang memerlukan rekayasa fitur secara manual untuk mengenali tekstur atau bentuk, CNN memiliki kemampuan unggul untuk mempelajari hierarki fitur visual secara otomatis langsung dari data piksel (Bahmani dkk., 2025). Kemampuan ini membuat CNN sangat efektif dalam tugas pengenalan pola visual yang kompleks dan non-linear seperti motif batik.

Strategi *transfer learning* dipilih untuk mengoptimalkan efisiensi pelatihan model. *Transfer learning* memungkinkan untuk memanfaatkan model CNN yang telah dilatih pada dataset besar (seperti *ImageNet*) untuk kemudian diadaptasi pada tugas spesifik dengan dataset yang lebih kecil (Faturrahman dkk., 2023). Pengetahuan visual dasar seperti deteksi tepi, bentuk, dan tekstur yang telah dipelajari oleh model *pre-trained* tersebut dapat ditransfer dan disesuaikan untuk mengenali pola-pola unik pada motif batik.

Model *pre-trained* yang dipilih adalah MobileNetV2, model yang dirancang untuk memberikan latensi rendah dan efisiensi daya yang tinggi, serta dioptimalkan untuk bekerja dengan baik dalam berbagai situasi dengan keterbatasan sumber daya

(Nesta Suandana dkk., 2024). Dengan ukuran model yang lebih ringan dibandingkan arsitektur CNN tradisional, MobileNetV2 memungkinkan proses pelatihan dan inferensi dilakukan lebih cepat, sekaligus tetap mempertahankan akurasi tinggi, sehingga sangat sesuai untuk diterapkan pada dataset batik yang terbatas.

Yang membedakan penelitian ini dengan studi-studi terdahulu terletak pada implementasi *Convolutional Neural Network* (CNN) yang diintegrasikan secara spesifik ke dalam aplikasi berbasis *mobile*. Penelitian terkait klasifikasi motif batik sebelumnya banyak dilakukan menggunakan metode klasik dengan ekstraksi fitur manual dan jarang yang khusus membahas Batik Pekalongan. Sebagai contoh, penelitian Tegar dkk. (2022) pada motif Batik Banyuwangi memanfaatkan kombinasi *K-Nearest Neighbor* (K-NN) dan ekstraksi fitur *Local Binary Pattern* (LBP) menghasilkan akurasi sebesar 65%, sementara studi lain yang menggunakan *Support Vector Machine* (SVM) berbasis ekstraksi GLCM pada citra bunga mencatat akurasi sebesar 78.3%. Di sisi lain, sejumlah penelitian telah menerapkan metode CNN, seperti Suhardi, dkk. (2023) yang berhasil menggunakan EfficientNet-B2 dengan teknik *fine-tuning* untuk klasifikasi batik Papua dengan akurasi 86,3%, serta Ardyani & Sari (2024) yang berfokus pada Batik Demak menggunakan VGG16 memiliki akurasi 91,23% tetapi membutuhkan komputasi besar.

Berdasarkan masalah tersebut, maka dikembangkan aplikasi edukatif untuk klasifikasi motif batik Pekalongan berbasis *mobile* dengan menggunakan arsitektur MobileNetV2 yang lebih ringan dengan mengimplementasikan algoritma *Convolutional Neural Network* (CNN), sehingga dapat berfungsi sebagai media pembelajaran interaktif yang efektif dan mudah diakses oleh masyarakat luas dalam mendukung upaya pelestarian warisan budaya. Dengan demikian, aplikasi ini diharapkan mampu menjembatani antara teknologi modern dengan seni tradisional, sekaligus menjaga relevansi Batik Pekalongan di tengah arus modernisasi.

1.2. Rumusan Masalah

Dari latar belakang yang sudah disampaikan sebelumnya, fokus utama rumusan masalah, yaitu:

a. Kurangnya media edukasi untuk mengenalkan motif Batik Pekalongan kepada masyarakat, khususnya generasi muda.
b. Proses identifikasi motif Batik Pekalongan secara manual masih bersifat subjektif, tidak efisien, dan sulit dilakukan karena kompleksitas motif dan kemiripan visual antar motif.
c. Belum banyak penelitian yang memfokuskan klasifikasi motif Batik Pekalongan secara khusus menggunakan arsitektur CNN MobileNetV2 melalui pendekatan *transfer learning*.
d. Keterbatasan dataset motif Batik Pekalongan untuk proses pelatihan model *deep learning* sehingga diperlukan metode yang mampu mengoptimalkan performa dengan data terbatas.

1.3. Pertanyaan Penelitian

Berdasarkan identifikasi masalah yang telah dijelaskan, maka pertanyaan dalam penelitian ini dapat dirumuskan sebagai berikut:

a. Bagaimana merancang dan membangun aplikasi edukatif berbasis mobile untuk klasifikasi motif Batik Pekalongan menggunakan algoritma CNN dengan arsitektur MobileNetV2 dan pendekatan *transfer learning*?
b. Berapa akurasi, presisi, *recall*, dan *F1-score* yang dapat dicapai oleh model CNN MobileNetV2 dalam mengklasifikasikan motif Batik Pekalongan?

1.4. Batasan Masalah

Agar penelitian ini dapat dilakukan secara lebih terarah, maka ditetapkan batasan masalah sebagai berikut:

a. Dataset yang digunakan adalah kumpulan citra batik yang terdiri dari 7 kelas yaitu Pagi-sore, Jlamprang, Jawa Hokokai, Tirtateja, Terang Bulan, Buketan, dan Lunglungan.
b. Penelitian berfokus pada penggunaan arsitektur *Convolutional Neural Network* MobileNetV2 tanpa membandingkan dengan arsitektur lain.
c. Metode utama yang digunakan adalah *transfer learning* dengan memanfaatkan bobot *(weights)* dari model MobileNetV2 yang telah pra-terlatih *(pre-trained)* pada dataset *ImageNet*.

d. Pada aspek pengembangan perangkat lunak, penelitian berfokus pada pembangunan fitur klasifikasi yang menyajikan informasi sejarah, filosofi motif, ciri visual, dan galeri contoh motif.
e. Aplikasi *mobile* yang dikembangkan berfokus pada platform Android dan dibangun menggunakan *framework* Flutter.

1.5. Tujuan Penelitian

Adapun tujuan dari penelitian ini adalah sebagai berikut:

a. Merancang dan membangun sebuah aplikasi edukatif berbasis *mobile* yang fungsional dan informatif untuk mengidentifikasi serta mempelajari motif Batik Pekalongan.
b. Membangun sebuah model klasifikasi motif Batik Pekalongan menggunakan pendekatan *transfer learning* dengan arsitektur dasar CNN MobileNetV2.
c. Mengevaluasi performa model secara kuantitatif menggunakan metrik akurasi, presisi, *recall*, dan *F1-score* untuk mengukur kemampuannya dalam mengklasifikasikan motif batik pada data uji.
d. Mengimplementasikan model *Convolutional Neural Network* (CNN) dengan arsitektur MobileNetV2 untuk klasifikasi motif batik Pekalongan pada aplikasi edukatif berbasis *mobile*.

1.6. Manfaat Penelitian

Diharapkan penelitian ini dapat memberikan manfaat bagi dunia akademis serta bagi peneliti, adapun manfaat yang didapat antara lain:

a. Bagi Peneliti
    1) Menambah wawasan dan keterampilan teknis dalam penerapan *deep learning*, khususnya pada penggunaan arsitektur MobileNetV2 untuk pengenalan citra pada aplikasi *mobile*.
    2) Melatih kemampuan analisis dan eksperimen ilmiah, mulai dari pengumpulan dataset, pelatihan model, hingga implementasi hasil klasifikasi ke dalam aplikasi edukatif berbasis Android.
    3) Menjadi dasar pengembangan penelitian lanjutan, seperti optimasi model CNN, perluasan dataset, atau pengembangan aplikasi identifikasi batik berbasis *mobile*/web.

b. Bagi Pembaca dan Dunia akademik

1) Menambah kontribusi literatur ilmiah di bidang *deep learning* dan pengembangan aplikasi terutama pada topik klasifikasi citra motif batik.
2) Memberikan referensi metodologis bagi peneliti lain yang ingin meneliti pengenalan pola citra menggunakan arsitektur MobileNetV2 atau CNN lainnya.
3) Mendukung upaya digitalisasi dan dokumentasi budaya lokal, sehingga hasil penelitian dapat berkontribusi terhadap pelestarian serta pengenalan Batik Pekalongan melalui pendekatan teknologi modern.

c. Bagi Perguruan Tinggi

1) Menjadi bahan ajar atau contoh studi kasus dalam mata kuliah yang berkaitan dengan kecerdasan buatan, pemrosesan citra digital, serta pengembangan aplikasi mobile.
2) Mendorong peningkatan kualitas penelitian dan inovasi di lingkungan kampus, khususnya dalam pengembangan teknologi yang mendukung pelestarian budaya lokal.
3) Memperkuat peran perguruan tinggi sebagai lembaga yang berkontribusi pada pengembangan IPTEK sekaligus pelestarian warisan budaya melalui penelitian terapan.

BAB II
TINJAUAN PUSTAKA

2.1. Batik

Menurut Asti M. dan Ambar B. Arini, berdasarkan etimologi dan terminologinya, batik merupakan rangkaian kata *mbat* dan *tik*. *Mbat* dalam bahasa Jawa dapat diartikan sebagai *ngembat* atau melempar berkali-kali, sedangkan *tik* berasal dari kata titik (Musman & Arini, 2011). Secara khusus, batik merupakan seni menulis atau melukis yang dilakukan diatas kain. dalam pengerjaannya, pembatik menggunakan lilin atau malam untuk mendapatkan ragam hias atau pola di atas kain yang dibatik dengan menggunakan alat bernama canting (Supriono, 2024).

Berdasarkan pemaparan Herry Lisbijanto, batik dapat dikategorikan menjadi tiga jenis menurut teknik pembuatannya, yaitu batik tulis, batik cap, dan batik lukis. Batik tulis diproses secara manual menggunakan canting dengan tingkat ketelatenan tinggi, menghasilkan motif unik yang tidak identik satu sama lain serta memiliki nilai ekonomi tinggi sebagai simbol kemewahan yang dahulu dikenakan oleh kalangan bangsawan. Berbeda halnya dengan batik cap yang prosesnya dipercepat menggunakan stempel tembaga sebagai pengganti canting, sehingga mampu menghasilkan motif yang seragam secara massal dengan harga yang lebih terjangkau namun dinilai memiliki nilai seni yang lebih rendah dibandingkan batik tulis. Jenis terakhir adalah batik lukis, yang pengerjaannya dilakukan dengan melukiskan malam secara langsung pada kain tanpa terpaku pada pakem motif tradisional, melainkan sepenuhnya mengikuti kebebasan ekspresi dan kreativitas pelukisnya (Lisbijanto, 2013).

2.1.1. Batik Pagi-Sore

Batik Pagi-Sore merupakan salah satu varian batik khas Pekalongan yang muncul pada masa pendudukan Jepang, sekitar tahun 1942–1945 pasca Perang Dunia II (Ni'mah & Dartono, 2021). Kehadiran gaya ini erat kaitannya dengan fenomena Batik Djawa Hokokai yang lahir sebagai respons terhadap kelangkaan bahan baku tekstil pada masa itu. Keterbatasan ketersediaan kain mendorong para

pengrajin menciptakan inovasi desain Pagi-Sore, yaitu sehelai kain memiliki dua sisi dengan nuansa warna yang berbeda atau bertentangan. Konsep ini memungkinkan satu lembar kain digunakan untuk dua kesempatan waktu sekaligus, yakni waktu pagi dan sore, sebagai bentuk efisiensi. Secara visual, batik periode ini identik dengan ragam hias yang padat, penuh dengan detail *isen-isen* (titik-titik), serta penggunaan warna-warna tegas yang dimaknai sebagai simbol semangat masyarakat dalam menghadapi situasi sulit selama masa pendudukan (Paramanandana dkk., 2021). Tampilan dari motif batik Pagi-Sore dapat dilihat pada Gambar 1.



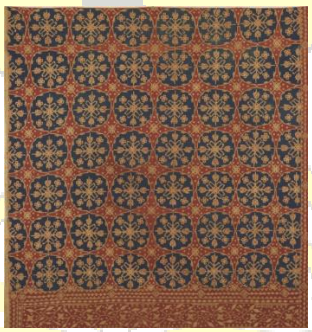
Gambar 1. Batik Pagi-Sore

Secara morfologi, ciri khas utama Batik Pagi-Sore ditandai dengan adanya garis pembatas diagonal ataupun vertikal di tengah kain yang memisahkan dua komposisi motif dan warna yang berbeda. Dalam penerapannya, terdapat pola tata letak yang disebut *susumoyo*, yakni motif yang bermula dari salah satu sudut kain dan menyebar ke arah tepi tanpa menyambung dengan motif dari sudut yang berlawanan. Unsur ornamen pada batik ini banyak dipengaruhi oleh akulturasi budaya Tiongkok yang masuk melalui estetika Jepang, dengan dominasi ragam hias flora dan fauna. Salah satu ornamen fauna yang paling menonjol adalah kupu-kupu. Penggunaan ornamen kupu-kupu ini tidak didasarkan pada makna filosofis lokal tertentu, melainkan merupakan bentuk adaptasi pasar terhadap preferensi estetika orang Jepang yang sangat menggemari keindahan serangga tersebut pada masa itu (Chandra & Ari Dartono, 2021).

2.1.2. Batik Jlamprang

Batik Jlamprang merupakan varian motif khas Pekalongan yang memiliki akar sejarah akulturasi kuat antara budaya lokal dengan pengaruh India dan Arab.

Motif ini berkembang pesat di kawasan Krapyak, Pekalongan, yang secara historis merupakan permukiman komunitas Arab. Secara visual, Jlamprang diadaptasi dari struktur pola tenun ikat ganda asal Gujarat, India, yang dikenal sebagai kain Patola atau *Cinde*. Pada masa lalu, kain Patola dianggap sebagai simbol status yang sakral, sehingga para pengrajin batik Pekalongan mereplikasi pola tenun tersebut ke dalam teknik batik untuk memenuhi kebutuhan pasar lokal yang tidak terjangkau oleh kain impor (Maziyah dkk., 2021). Berbeda dengan motif Pekalongan lainnya yang lugas dalam menggambarkan makhluk hidup, Jlamprang merepresentasikan kepatuhan terhadap nilai-nilai Islam yang dianut masyarakat Krapyak dengan menghindari penggambaran figur bernyawa secara realistis, menggantinya dengan simbol-simbol geometris (Kusrianto, 2024). Tampilan dari motif batik Jlamprang dapat dilihat pada Gambar 2.



Gambar 2. Batik Jlamprang

Batik Jlamprang dikategorikan sebagai motif geometris yang tersusun atas ornamen *ceplokan* (pola berulang) berupa lingkaran, bujur sangkar, dan bintang delapan yang disusun secara simetris. Karakteristik ini membedakannya secara signifikan dari motif Buketan yang organik dan luwes. Meskipun memiliki kemiripan struktur struktural dengan motif Nitik dari Yogyakarta, Jlamprang memiliki ciri khas pembeda pada aspek pewarnaan. Jlamprang menggunakan warna-warna cerah khas pesisir seperti merah, hijau, dan biru, sedangkan Nitik cenderung menggunakan warna sogan (Nuraryo dkk., 2023).

2.1.3. Batik Terang Bulan

Batik Terang Bulan merupakan salah satu motif batik pesisir Pekalongan yang memiliki karakteristik visual khas berupa pemanfaatan ruang kosong *(lowong)* yang relatif luas pada bidang tengah kain, sementara motif utama terkonsentrasi pada bagian pinggir (*papan* atau *rembang*) atau pada satu sisi kain. Komposisi ini membedakan Batik Terang Bulan dari motif batik klasik pedalaman yang cenderung padat dan memenuhi seluruh bidang kain. Penggunaan ruang kosong tersebut menciptakan kesan visual yang bersih, ringan, dan elegan, sekaligus menonjolkan motif hias di bagian tepi sebagai fokus estetika. Fenomena pemanfaatan ruang dalam batik pesisir ini sejalan dengan karakter batik Pekalongan yang lebih bebas dalam komposisi dan tidak terikat pakem ketat sebagaimana batik keraton (Ishwara, 2023). Tampilan dari motif batik Terang Bulan dapat dilihat pada Gambar 3.



Gambar 3. Batik Terang Bulan

Secara filosofis, penamaan "Terang Bulan" merepresentasikan keindahan bulan purnama yang bersinar di langit malam yang bersih dan luas. Bidang kosong pada kain dimaknai sebagai visualisasi langit malam yang terang, sedangkan ornamen bunga, kupu-kupu, atau burung di bagian pinggir kain dianalogikan sebagai elemen alam yang menghiasi semesta. Simbolisme bulan purnama dalam budaya Jawa dan pesisir sering dikaitkan dengan makna kesempurnaan, ketenangan, dan harapan akan kehidupan yang seimbang. Dari sisi elemen visual, Batik Terang Bulan didominasi oleh ragam hias flora dan fauna yang disusun secara naturalistis, seperti buketan bunga, daun, kupu-kupu, serta burung. Pemilihan motif tersebut mencerminkan ciri khas batik pesisir yang kuat dipengaruhi oleh selera estetika peranakan Tionghoa dan Indo-Eropa, yang lebih menyukai penggambaran alam secara realistis dan dekoratif.

2.1.4. Batik Jawa Hokokai

Batik Jawa Hokokai merupakan salah satu varian batik Pekalongan yang berkembang pada masa pendudukan Jepang di Indonesia (1942–1945). Istilah Hokokai merujuk pada organisasi bentukan pemerintah militer Jepang *(Djawa Hōkōkai)*, yang turut memengaruhi estetika dan produksi batik pada periode tersebut. Batik ini dikenal melalui karakter visualnya yang sangat detail, padat ornamen, serta penggunaan warna-warna cerah dan kontras seperti merah, kuning, hijau, biru, dan ungu yang disusun secara harmonis (Kusrianto, 2024). Tampilan dari motif batik Jawa Hokokai dapat dilihat pada Gambar 4.



Gambar 4. Batik Jawa Hokokai

Secara visual, Batik Jawa Hokokai menampilkan pengaruh kuat budaya Jepang yang berpadu dengan tradisi batik pesisir Pekalongan. Ragam hias flora seperti bunga sakura, krisan, dan anggrek digambarkan dengan gaya naturalis dan teknik isen-isen yang rumit, menciptakan kesan mewah dan penuh dinamika. Selain itu, motif fauna seperti kupu-kupu, burung merak, dan burung bangau sering hadir sebagai simbol keindahan, umur panjang, dan kemakmuran dalam budaya Jepang (Tjandrawibawa & Tanzil, 2022). Komposisi desainnya cenderung memenuhi seluruh bidang kain *(horror vacui)*, mencerminkan nilai estetika ketelitian dan kesempurnaan visual yang menjadi ciri khas Batik Jawa Hokokai.

2.1.5. Batik Tirta Teja

Batik Tirta Teja, yang kadang juga dieja *Tirto Tejo*, adalah salah satu motif batik tradisional yang dikenal dalam ragam motif batik Jawa dan sering dianggap memiliki nilai filosofis kuat meskipun literatur akademis khusus tentang motif ini masih jarang dalam publikasi jurnal ilmiah. Secara etimologis, istilah *Tirta* berasal dari bahasa Jawa yang berarti air, sedangkan *Teja* berarti cahaya atau bersinar.

Kombinasi istilah ini secara simbolis merepresentasikan air yang bercahaya melambangkan aliran kehidupan, kemakmuran yang terus mengalir, serta semangat yang dinamis dan optimis dalam perjalanan hidup orang yang mengenakannya. Tampilan dari motif batik Tirta Teja dapat dilihat pada Gambar 5.



Gambar 5. Batik Tirta Teja

2.1.6. Batik Buketan

Batik Buketan merupakan varian batik pesisir yang khas dengan komposisi ragam hias bunga-bungaan yang tersusun menyerupai karangan bunga. Motif ini berkembang pesat di batik pesisir seperti Pekalongan karena lingkungan budaya yang terbuka serta adanya pengaruh selera asing, khususnya preferensi noni Belanda yang kemudian diolah kembali oleh para pembatik lokal menjadi gaya visual yang unik (Nafisah dkk., 2021). Karakter estetis batik Buketan ditandai oleh elemen vegetal yang padat dan detail naturalistik, sering kali menampilkan flora yang umum ditemukan di Eropa, seperti bunga krisan, anggur, dan rangkaian bunga lainnya, menjadikannya berbeda dari motif geometris kraton yang lebih formal dan terstruktur. Tampilan dari motif batik Buketan dapat dilihat pada Gambar 6.



Gambar 6. Batik Buketan

Secara historis, istilah buketan yang berasal dari kata *bouquet* merefleksikan hubungan erat antara motif bunga dengan tradisi mode serta selera busana pada

masa kolonial hingga era kontemporer. Seiring waktu, motif ini mengalami reinterpretasi oleh berbagai komunitas pembatik, termasuk pembatik peranakan Tionghoa dan pengrajin berpengaruh lainnya, sehingga melahirkan beragam gaya seperti Buketan gaya Cina maupun Buketan gaya Belanda. Keberagaman tersebut memperkaya identitas visual batik pesisir dan menjadikan buketan sebagai salah satu motif yang tetap dikenali, diminati, dan terus berkembang hingga sekarang (Angelika, 2024).

2.1.7. Batik Lunglungan

Secara etimologi, istilah lunglungan berasal dari kata *lung* dalam bahasa Jawa yang berarti pucuk pohon atau tunas tumbuhan yang melilit dan merambat. Secara filosofis, motif ini melambangkan kelangsungan hidup yang dinamis, kesuburan, serta hubungan harmonis yang tidak terputus, yang direpresentasikan melalui penggayaan (stilasi) batang, daun, dan bunga yang saling mengait (Nursanti et al., 2022). Karakteristik utama dari ornamen ini adalah bentuknya yang luwes dan elastis, mencerminkan kemampuan adaptasi manusia terhadap lingkungan tanpa kehilangan kekuatan prinsip dasarnya (Fajaryani & Santosa, 2021). Tampilan dari motif batik Lunglungan dapat dilihat pada Gambar 7.



Gambar 7. Batik Lunglungan

Dalam aspek estetika visual, motif lunglungan sering kali digunakan sebagai pengisi ruang atau ornamen pendukung yang memberikan efek ritme atau gerak pada kain melalui pengulangan pola sulur-suluran yang berkelanjutan. Berbeda dengan motif buketan yang cenderung statis dan terfokus pada satu titik, lunglungan memiliki struktur yang lebih kompleks dan dapat memenuhi seluruh bidang kain atau menjadi penghubung antar-motif utama lainnya (Hariyani & Sularso, 2020). Penggunaan motif ini pada batik pesisir biasanya dipadukan dengan warna-warna

cerah dan elemen fauna seperti burung atau kupu-kupu untuk menambah kesan naturalis dan estetis (Ramadhani & Musman, 2021).

2.2. Klasifikasi

Klasifikasi adalah sebuah metode dalam pembelajaran mesin yang berfungsi untuk mengelompokkan objek atau *instance* ke dalam kelas-kelas yang telah ditetapkan berdasarkan pola atau fitur dari data latih. Dalam konteks Bahasa Indonesia, misalnya didefinisikan sebagai "teknik dalam pembelajaran mesin yang bertujuan memprediksi kelas atau label suatu data dengan memanfaatkan fitur-fiturnya" (Wibowo dkk., 2024). Secara teknis, klasifikasi merupakan bagian dari *supervised learning* yaitu pembelajaran yang menggunakan data berlabel dengan fokus utamanya adalah menghasilkan *output* berupa kategori atau kelas diskret, berbeda dengan *regresi* yang menghasilkan nilai kontinu (Alnuaimi & Albaldawi, 2024). Klasifikasi dapat dilihat sebagai proses prediktif, yaitu algoritma pembelajaran mesin menggunakan data latih untuk mempelajari karakteristik masing-masing kelas, kemudian menerapkan model tersebut untuk memprediksi label pada data baru yang belum berlabel (Marques dkk., 2025).

Dalam konteks citra digital, klasifikasi citra merupakan suatu proses dimana setiap citra atau elemen dalam citra (seperti piksel, objek, atau potongan gambar) dikategorikan ke dalam kelas-tertentu berdasarkan karakteristik visual seperti warna, tekstur, bentuk atau intensitas piksel (Ambarwari dkk., 2023).

2.3. *Machine Learning*

Pembelajaran mesin (*machine learning*) merupakan cabang dari kecerdasan buatan yang berfokus pada pengembangan algoritma dan model statistik yang memungkinkan sistem komputer belajar dari data dan meningkatkan kinerjanya tanpa pemrograman eksplisit untuk setiap tugas (Riska Rismaya dkk., 2025). Pembelajaran mesin pada umumnya dibagi menjadi tiga kategori utama, yaitu pembelajaran terawasi (*supervised*), pembelajaran tak terawasi (*unsupervised*), dan pembelajaran penguatan (*reinforcement learning*), yang masing-masing kategori memiliki pendekatan dan aplikasi yang berbeda sesuai tujuan penelitian atau sistem (Fahmi, 2023).

Secara umum, *machine learning* bekerja dengan menemukan pola tersembunyi dalam data melalui proses pelatihan yang memungkinkan model melakukan generalisasi untuk memprediksi data baru yang sebelumnya tidak pernah dilihat (Bagaskara, 2023). Kemampuan ini menjadikan *machine learning* banyak diterapkan dalam berbagai bidang seperti pengolahan citra digital, pengenalan suara, analisis data kesehatan, sistem rekomendasi, hingga otomasi industri.

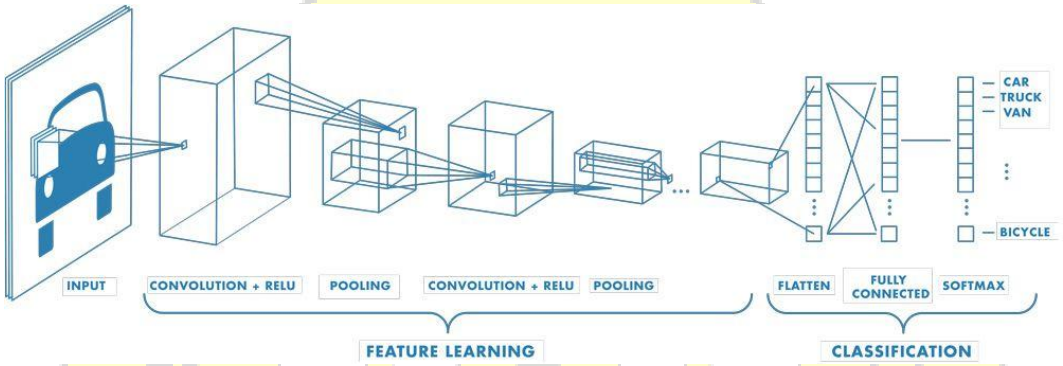
2.4. *Deep Learning*

Deep learning merupakan subbidang dari *machine learning* dan kecerdasan buatan yang berbasis pada *Artificial Neural Network* (ANN) dengan banyak lapisan *(multiple layers)*. Teknologi ini dikembangkan untuk menyelesaikan tugas-tugas kompleks dengan akurasi tinggi, seperti deteksi objek, pengenalan suara, dan penerjemahan bahasa (Raup dkk., 2022). Pendekatan *deep learning* adalah kelas algoritma *machine learning* yang menggunakan banyak lapisan unit pemrosesan nonlinear untuk representasi dan transformasi. Setiap lapisan menggunakan output dari lapisan sebelumnya sebagai input dan representasi hierarkis dapat diperoleh dengan tingkat abstraksi yang berbeda.

Pendekatan deep learning memungkinkan model mengekstraksi representasi fitur secara hierarkis *(hierarchical feature representation)* dari data mentah melalui jaringan saraf tiruan berlapis. Melalui proses *hierarchical learning*, setiap lapisan dalam jaringan secara bertahap membentuk representasi data mulai dari fitur dasar hingga fitur tingkat tinggi, sehingga model dapat melakukan *representation learning* secara otomatis tanpa memerlukan rekayasa fitur manual (Alzubaidi dkk., 2021). Dalam arsitektur jaringan saraf tiruan tersebut, lapisan input diikuti oleh satu atau lebih *hidden layer* yang berisi neuron-neuron internal untuk menangkap pola non-linear serta memproses fitur secara mendalam sebelum informasi diteruskan ke lapisan *output*. Kemampuan pembelajaran berlapis inilah yang membuat *deep learning* mampu mengenali pola kompleks dan menghasilkan generalisasi yang lebih baik saat memprediksi data baru.

2.5. *Convolutional Neural Network*

Convolutional Neural Network (CNN) adalah jenis *neural network* yang digunakan untuk analisis citra. CNN bekerja dengan mendeteksi dan mengenali objek dalam gambar menggunakan proses konvolusi. Proses konvolusi dilakukan dengan menerapkan filter berukuran tertentu yang memindai seluruh area citra melalui operasi perkalian matematis pada tahap ini bertujuan untuk mengekstraksi representasi fitur dari citra asli (Ramadhani & Tarigan, 2025). Arsitektur CNN ditunjukkan pada Gambar 8.



Gambar 8. Arsitektur *Convolutional Neural Network*

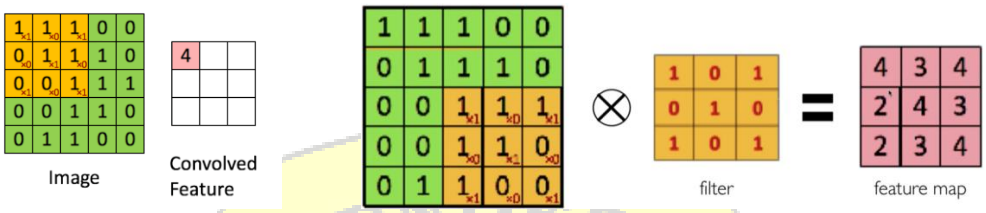
Sumber: https://uk.mathworks.com/discovery/convolutional-neural-network.html

Sebagaimana diilustrasikan pada Gambar 1, arsitektur CNN secara umum terbagi menjadi dua tahapan utama, yaitu ekstraksi fitur dan klasifikasi. Pada tahap ekstraksi fitur, citra input diproses melalui lapisan tersembunyi *(hidden layer)* yang terdiri dari operasi konvolusi dan *pooling* untuk menghasilkan peta fitur *(feature map)* menggunakan filter. Selanjutnya, data tersebut memasuki tahap klasifikasi, yaitu dengan dimensi peta fitur menjadi vektor satu dimensi melalui proses *flattening*. Vektor ini kemudian diproses oleh lapisan *fully connected* hingga mencapai lapisan output, yang jumlah neuronnya merepresentasikan total kelas yang akan diklasifikasikan (Ghandi dkk., 2024).

2.5.1. *Convolutional Layer*

Lapisan konvolusi (*convolutional layer*) merupakan elemen inti dari arsitektur CNN yang berfungsi mengekstrak fitur-spasial dari citra input. Dalam tahap ini, sejumlah filter (kernel) dengan ukuran tertentu akan digeser (*stride*)

melintasi seluruh input citra atau peta aktivasi dari lapisan sebelumnya untuk menghasilkan peta fitur (*feature map*) baru, dimana setiap filter belajar mengenali pola-tertentu seperti tepi, sudut, tekstur, atau motif visual lainnya (Azmi dkk., 2023). Proses pengolahan pada *convolutinal layer* dapat dilihat pada Gambar 9.



Gambar 9. Proses *Convolution Layer*

Sumber: https://skillplus.web.id/pengenalan-convolutional-neural-network-cnn/

Dalam pengolahan citra, konvolusi didefinisikan sebagai operasi matematis yang mengaplikasikan kernel atau filter pada citra input. Sebagaimana diilustrasikan pada Gambar 2, proses ini dilakukan dengan menggeser kernel (ditandai dengan kotak kuning) melintasi seluruh area citra (kotak hijau) secara sistematis, dimulai dari sudut kiri atas menuju kanan bawah (Mbaba dkk., 2022).

2.5.2. *Activation Function*

Fungsi aktivasi *(activation function)* didefinisikan sebagai fungsi matematis yang menentukan apakah sebuah neuron harus diaktifkan atau tidak berdasarkan input yang diterimanya. Fungsi ini bekerja dengan mentransformasikan total input berbobot menjadi nilai keluaran tertentu yang akan diteruskan ke lapisan berikutnya. Dalam beberapa literatur, istilah ini juga sering dirujuk sebagai fungsi transfer *(transfer function)* (Kurniawan dkk., 2024). Pada arsitektur *Convolutional Neural Network* (CNN) MobileNetV2, fungsi aktivasi yang umum digunakan adalah ReLU6 (*Rectified Linear Unit clipped at* 6) pada sebagian besar lapisan internal. Sementara itu, pada lapisan output untuk tugas klasifikasi multi-kelas, fungsi aktivasi yang digunakan adalah *softmax*.

1. Fungsi ReLU

Rectified Linear Unit (ReLU) merupakan salah satu fungsi aktivasi yang paling dominan digunakan dalam arsitektur jaringan saraf tiruan modern. Secara

matematis, fungsi ini bekerja dengan mekanisme *thresholding* (ambang batas) pada nilai nol sebagaimana ditunjukkan pada Persamaan 1.

$$f(x) = max(0, x) \quad (1)$$

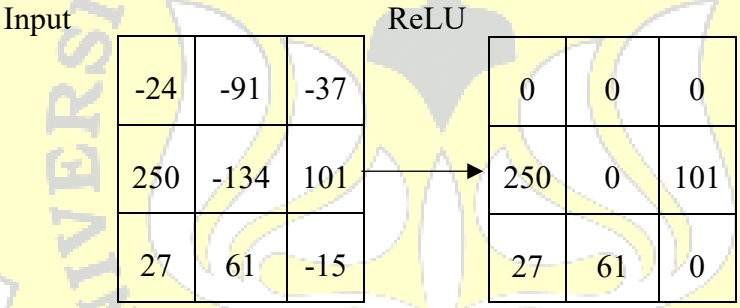
Keterangan:

$f(x)$: nilai output dari fungsi aktivasi ReLU untuk input x.

x : input yang masuk ke neuron.

$\max(0, x)$: operasi yang memilih nilai maksimum antara 0 dan x.

Jika diberikan masukan x bernilai positif, maka fungsi $f(x)$ akan meneruskan nilai x tersebut apa adanya. Sebaliknya, jika nilai x negatif, maka fungsi akan menghasilkan keluaran 0 (Kurniawan dkk., 2024). Contoh ilustrasi perhitungan untuk fungsi aktivasi ReLU dapat dilihat pada Gambar 10.



Gambar 10. Transformasi Nilai Matriks pada Lapisan ReLU

Sumber: https://www.researchgate.net/figure/Working-of-ReLu-Activation-Function-with-an-example_fig5_370852091

2. Fungsi ReLU6

ReLU6 merupakan modifikasi dari fungsi *Rectified Linear Unit* (ReLU) yang membatasi nilai aktivasi maksimum di angka 6. Pembatasan ini dirancang agar model lebih tahan (*robust*) saat dijalankan pada perangkat *mobile* dengan presisi rendah (Salam dkk., 2024). Secara matematis, formulasi fungsi ReLU6 dinyatakan dalam Persamaan 2.

$$f(x) = min(max(0, x), 6) \quad (2)$$

Keterangan:

$f(x)$: nilai output dari fungsi aktivasi ReLU6 untuk input x.

x : input yang masuk ke neuron.

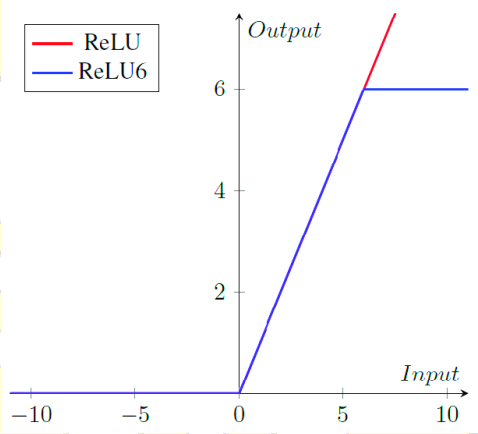
$\max(0, x)$: bagian ini bekerja sama seperti ReLU standar;

Jika $x > 0$, output sama dengan x, jika $x \leq 0$, output menjadi 0.

$\min(\ldots, 6)$: membatasi output maksimum hingga 6.

Jika nilai dari $\max(0, x)$ > dari 6, output dibatasi menjadi 6.

Perbedaan karakteristik antara fungsi aktivasi ReLU dan ReLU6 diilustrasikan pada Gambar 11.



Gambar 11. Grafik ReLU dan ReLU6

Sumber: https://www.researchgate.net/figure/Graph-demonstrating-the-difference-between-ReLU-and-ReLU6-activation-functions_fig1_350148089

3. Fungsi *Softmax*

Fungsi aktivasi *softmax* digunakan pada lapisan output untuk masalah klasifikasi multi-kelas, karena mampu mengubah vektor *logits* (input) menjadi distribusi probabilitas yang dijumlahkan menjadi satu. Secara matematis, fungsi *Softmax* didefinisikan pada Persamaan 3.

$$\sigma(z)_i = \frac{e^{z_i}}{\Sigma_{j=1}^{K} e^{z_j}} \quad (3)$$

Keterangan:

$\sigma(z)_i$: Probabilitas prediksi untuk kelas ke-i (nilai antara 0-1).

z : Vektor input (*logits*) yang dihasilkan oleh lapisan sebelumnya.

e : Bilangan Euler / konstanta eksponensial $(\approx 2,718)$.

z_i : Nilai input (skor) untuk kelas target ke-i.

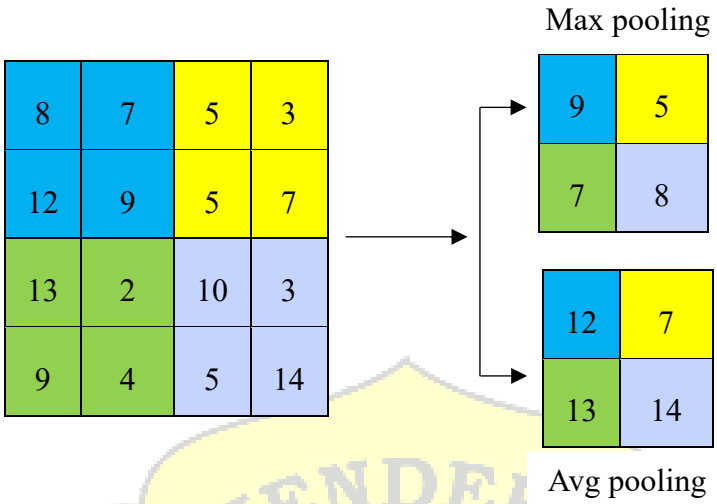
z_i : Nilai input untuk setiap kelas dalam jangkauan jumlahan.

K : Jumlah total kelas atau kategori klasifikasi.

Dimana z adalah vektor input dan K jumlah kelas. *Softmax* memungkinkan interpretasi probabilistik dari hasil prediksi dan biasanya dipasangkan dengan *cross-entropy loss* untuk optimasi pelatihan jaringan saraf (Shatravin dkk., 2023). Fungsi ini juga digunakan secara luas dalam implementasi model *deep learning* modern, termasuk pada arsitektur konvolusi dan *transformer*, karena stabilitasnya dalam proses *training* dan kemampuannya menghasilkan prediksi yang dapat diandalkan.

2.5.3. *Pooling Layer*

Lapisan *pooling (pooling layer)* merupakan komponen penting dalam arsitektur CNN yang berfungsi untuk mengurangi dimensi spasial dari *feature map* hasil proses konvolusi dengan cara melakukan operasi *down-sampling*. Tahapan ini bertujuan menurunkan jumlah parameter dan kompleksitas komputasi, sekaligus menjaga informasi inti yang paling penting dari fitur visual citra. *Pooling* bekerja dengan menggeser sebuah kernel pada feature map dan menghasilkan nilai ringkas berdasarkan fungsi tertentu seperti nilai maksimum *(max pooling)* atau nilai rata-rata *(average pooling)* pada area yang dilaluinya (Anhar & Putra, 2023). Ilustrasi pada proses *pooling layer* dapat dilihat pada Gambar 12.



Gambar 12. Proses *Pooling Layer*

Sumber: https://www.researchgate.net/figure/Pooling-layer-operation-oproaches-1-Pooling-layers-For-the-function-of-decreasing-the_fig4_340812216

Gambar 5 memperlihatkan mekanisme lapisan *pooling* yang bekerja dengan cara memindai *feature map* menggunakan kernel berukuran 2x2. Langkah ini menghasilkan representasi fitur yang lebih ringkas. Pada metode *max pooling*, sistem akan mengambil nilai piksel tertinggi dalam jendela kernel, sedangkan pada *average pooling*, nilai yang diambil adalah hasil rata-rata dari elemen-elemen yang tertangkap oleh kernel tersebut.

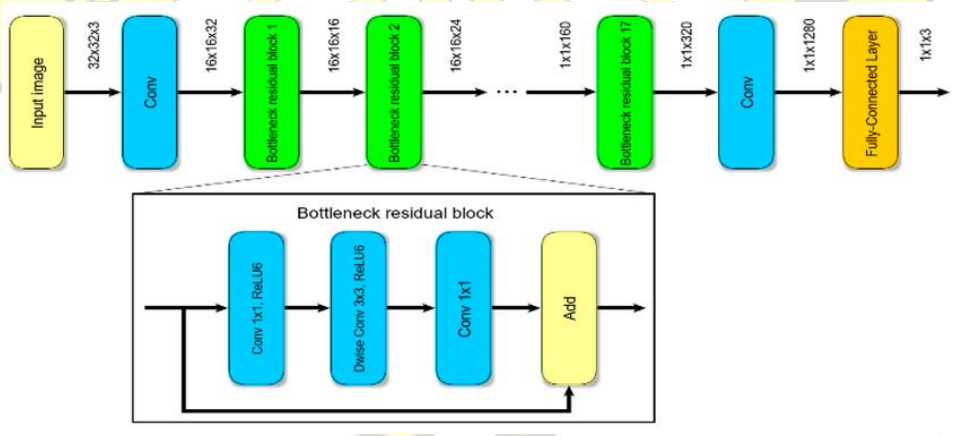
2.5.4. *Fully Connected Layer*

Lapisan *Fully-connected* atau *dense layer* adalah lapisan yang setiap neuronnya memiliki koneksi penuh terhadap seluruh neuron atau aktivasi dari lapisan sebelumnya. Mekanisme ini serupa dengan struktur *Multi-Layer Perceptron* (MLP) pada jaringan saraf tiruan tradisional (Azmi dkk., 2023). Dalam arsitektur CNN, lapisan ini ditempatkan di bagian akhir jaringan untuk berfungsi sebagai pengklasifikasi *(classifier)*. Lapisan ini akan memetakan fitur-fitur mendalam yang telah diekstraksi oleh lapisan konvolusi menjadi skor probabilitas untuk setiap kelas target, biasanya dengan bantuan fungsi aktivasi *Softmax* pada neuron keluaran. Selain itu, *dense layer* berperan penting dalam mengintegrasikan informasi global dari seluruh *feature maps*, sehingga jaringan mampu memahami representasi abstrak yang tidak lagi bergantung pada posisi spasial. Lapisan ini juga memiliki

parameter yang jauh lebih banyak dibandingkan lapisan konvolusi, sehingga sering menjadi penyumbang terbesar dalam kompleksitas model (Amylia dkk., 2025).

2.6. MobileNetV2

MobileNetV2 merupakan arsitektur *Convolutional Neural Network* (CNN) ringan yang dirancang khusus untuk beroperasi secara efisien pada perangkat dengan sumber daya komputasi terbatas, seperti perangkat *mobile* atau *embedded system*. Arsitektur ini diperkenalkan oleh Sandler, dkk dalam penelitian mereka yang berjudul *'MobileNetV2: Inverted Residuals and Linear Bottlenecks'* (Barat dkk., 2025). Inovasi utama model ini terletak pada penggunaan blok *inverted residual* dengan koneksi *shortcut* ditempatkan secara langsung di antara lapisan-lapisan *bottleneck* yang sempit. Selain itu, MobileNetV2 menerapkan konsep *linear bottleneck* dengan meniadakan fungsi aktivasi nonlinear (seperti ReLU) pada lapisan proyeksi akhir. Pendekatan ini bertujuan untuk meminimalkan hilangnya informasi *(information loss)* dan menjaga representasi fitur yang lebih kaya. Arsitektur MobileNetV2 ditunjukan pada gambar 13.



Gambar 13. Arsitektur MobileNetV2

Sumber: https://www.researchgate.net/figure/The-architecture-of-the-MobileNetv2-network_fig3_342856036

2.6.1. *Depthwise Separable Convolution*

Depthwise Separable Convolution (DSC) merupakan teknik optimasi pada jaringan saraf tiruan yang memecah operasi konvolusi standar menjadi dua tahapan

independen untuk meningkatkan efisiensi. Tahap pertama adalah *depthwise convolution,* yaitu setiap saluran *(channel)* input dikonvolusi secara terpisah menggunakan filter spasialnya masing-masing, tanpa pencampuran antar-saluran. Hasil dari tahap ini kemudian diproses pada tahap kedua, yaitu *pointwise convolution*. Pada tahap ini, filter berukuran 1x1 diterapkan untuk menggabungkan *(combine)* kembali fitur-fitur dari seluruh saluran tersebut secara linear. Meskipun menghasilkan dimensi output yang setara dengan konvolusi standar, metode ini menggunakan jumlah parameter dan operasi komputasi yang jauh lebih sedikit (Indraswari dkk., 2022).

2.6.2. *Inverted Residual Block*

Inverted Residual Block merupakan inovasi arsitektural fundamental dalam MobileNetV2 yang membedakannya dari jaringan residual standar seperti ResNet, dengan koneksi pintas *(shortcut connection)* secara unik menghubungkan lapisan-lapisan *bottleneck* yang berdimensi rendah alih-alih lapisan berdimensi tinggi. Mekanisme ini disebut *inverted* (terbalik) karena alur pemrosesannya dimulai dengan memperluas input berkanal sempit menjadi dimensi yang jauh lebih besar menggunakan konvolusi 1 x 1 (fase ekspansi) dengan faktor perluasan tertentu untuk memperkaya kapasitas informasi, dilanjutkan dengan ekstraksi fitur spasial menggunakan *depthwise convolution* pada dimensi luas tersebut, dan diakhiri dengan memampatkan kembali fitur ke dimensi rendah melalui lapisan proyeksi linear. Pendekatan desain ini memungkinkan jaringan untuk melakukan komputasi kompleks di ruang dimensi tinggi demi akurasi ekstraksi fitur motif yang optimal, sekaligus secara signifikan mengurangi beban memori *(memory footprint)* saat inferensi karena data yang ditransfer antar-blok tetap dipertahankan dalam dimensi yang ringkas dan efisien (Sandler dkk., 2018).

2.6.3. *Linear Bottleneck*

Linear Bottleneck merupakan komponen krusial yang ditempatkan pada akhir setiap *inverted residual block* sebagai lapisan proyeksi untuk mengembalikan dimensi fitur dari ruang ekspansi tinggi ke rendah *(low-dimensional)*. Berbeda dengan lapisan konvolusi standar yang umumnya selalu diikuti oleh fungsi aktivasi

nonlinear seperti ReLU, arsitektur MobileNetV2 secara spesifik menghilangkan fungsi aktivasi tersebut pada lapisan output blok ini dan mempertahankan sifat linearnya. Keputusan desain ini didasarkan pada temuan eksperimental bahwa penerapan operasi nonlinear pada tensor berdimensi rendah akan menyebabkan hilangnya informasi *(information loss)* yang signifikan karena fungsi seperti ReLU akan secara agresif menjolkan nilai-nilai negatif, sehingga dengan menerapkan *linear bottleneck*, model dapat meminimalisir degradasi informasi dan memastikan representasi fitur-fitur penting dari citra batik yang telah diekstraksi tetap utuh saat diteruskan ke lapisan berikutnya (Sandler dkk., 2018).

2.7. *Confusion Matrix*

Confusion Matrix merupakan metode evaluasi kuantitatif yang efektif digunakan untuk mengukur kinerja model klasifikasi. Tujuan utama penggunaan matriks ini adalah untuk memvalidasi keandalan sistem dengan cara membandingkan hasil prediksi model terhadap label sebenarnya *(ground truth)* pada data uji, sehingga akurasi klasifikasi dapat diukur secara presisi (Nurhidayat & Dewi, 2023). Analisis elemen dalam *confusion matrix* memungkinkan untuk mengevaluasi *trade-off* antara presisi dan *recall*, khususnya pada kondisi dataset yang tidak seimbang atau ketika risiko kesalahan prediksi memiliki bobot dampak yang berbeda. Untuk memperjelas pemetaan antara kelas prediksi dan kelas sebenarnya, ilustrasi *confusion matrix* dapat dilihat pada Gambar 14.

		Prediksi	
		Positif	**Negatif**
Aktual	**Positif**	TP (True Positive)	FP (False Positive)
	Negatif	FN (False Negative)	TN (True Negative)

Gambar 14. Confusion Matrix

Sumber: https://medium.com/@RobuRishabh/classification-measures-in-machine-learning-fe293feaa358

Pada Gambar 7, memberikan informasi yang komprehensif mengenai empat parameter utama: *True Positive* (TP) yaitu data positif yang diprediksi benar, *True*

Negative (TN) yaitu data negatif yang diprediksi benar, *False Positive* (FP) yaitu data negatif yang salah diprediksi sebagai positif (kesalahan Tipe I), dan *False Negative* (FN) yaitu data positif yang salah diprediksi sebagai negatif (kesalahan Tipe II).

2.7.1. Akurasi

Akurasi *(Accuracy)* merupakan metrik evaluasi yang paling mendasar dan intuitif dalam mengukur performa model klasifikasi. Akurasi didefinisikan sebagai rasio antara jumlah prediksi yang benar (baik positif maupun negatif) terhadap keseluruhan jumlah data yang diuji. Nilai akurasi merepresentasikan seberapa sering model MobileNetV2 memprediksi motif batik dengan tepat secara keseluruhan. Perhitungan akurasi model dirumuskan pada Persamaan 4.

$$Accuracy = \frac{TP+TN+FP+FN}{TP+TN} \quad (4)$$

2.7.2. Presisi

Presisi *(Precision)* adalah metrik yang mengukur tingkat ketepatan antara informasi yang diminta oleh pengguna dengan jawaban yang diberikan oleh sistem. Dalam konteks klasifikasi menggunakan *Convolutional Neural Network* (CNN), presisi didefinisikan sebagai rasio jumlah data yang terklasifikasi positif secara benar *(True Positive)* dibagi dengan total data yang diprediksi sebagai positif oleh model (gabungan *True Positive* dan *False Positive*). Nilai presisi dihitung menggunakan formula sebagaimana ditunjukkan pada Persamaan 5.

$$Precision = \frac{TP}{TP+FP} \quad (5)$$

2.7.3. *Recall*

Recall, yang dalam beberapa literatur juga disebut sebagai sensitivitas *(sensitivity)* atau *True Positive Rate* (TPR), adalah metrik yang mengukur kemampuan model dalam menemukan kembali seluruh data yang positif dari dataset. *Recall* didefinisikan sebagai rasio antara jumlah prediksi positif yang benar *(True Positive)* terhadap jumlah total data yang sebenarnya positif (gabungan *True*

Positive dan *False Negative*). Nilai *recall* dihitung menggunakan formula sebagaimana ditunjukkan pada Persamaan 6.

$$Recall = \frac{TP}{TP+FN} \tag{6}$$

2.7.4. *F1-score*

F1-score (atau *F-Measure*) adalah rata-rata harmonis *(harmonic mean)* dari presisi dan *recall*. Berbeda dengan rata-rata aritmatika biasa, rata-rata harmonis memberikan bobot lebih pada nilai yang lebih kecil, sehingga *F1-score* akan bernilai rendah jika salah satu dari presisi atau *recall* bernilai rendah. Metrik ini menjadi indikator tunggal yang paling objektif untuk mengevaluasi model ketika terdapat ketidakseimbangan distribusi kelas *(imbalanced dataset)* atau ketika pencarian keseimbangan antara *False Positive* dan *False Negative* dianggap sama pentingnya. Perhitungan *f1-score* dirumuskan pada Persamaan 1.

$$F1 - Score = 2 \times \frac{Precision \times Recall}{Precision + Recall} \tag{7}$$

2.8. Python

Python adalah bahasa pemrograman tingkat tinggi berbasis interpreter yang telah menjadi standar *de facto* dalam pengembangan aplikasi kecerdasan buatan dan sains data karena sintaksisnya yang ringkas serta keterbacaannya yang tinggi (Kalyani dkk., 2023). Keunggulan utama Python terletak pada ekosistem pustaka *(libraries)* sumber terbuka yang sangat luas, seperti TensorFlow, Keras, NumPy, dan Matplotlib, yang menyediakan fungsi-fungsi siap pakai untuk operasi matematika kompleks, manipulasi data, hingga pembangunan model *deep learning*. Dukungan komunitas yang besar dan dokumentasi yang lengkap menjadikan Python alat yang sangat efisien untuk mengimplementasikan algoritma klasifikasi citra secara cepat dan akurat.

2.9. Flutter

Flutter merupakan *Software Development Kit* (SDK) *open-source* yang dikembangkan oleh Google untuk memfasilitasi pembangunan aplikasi lintas platform meliputi Android, iOS, web, dan desktop hanya dengan menggunakan satu basis kode *(single codebase)*(Muslim dkk., 2022). Penggunaan bahasa Dart

memberikan keunggulan signifikan pada kerangka kerja ini, terutama dalam aspek kecepatan yang mendekati performa native. Salah satu keunggulan utama Flutter adalah fitur *Hot Reload* yang memungkinkan pengembang melihat perubahan kode secara *real-time* tanpa memuat ulang aplikasi, serta ketersediaan pustaka *widget* yang kaya untuk menyusun antarmuka pengguna *(User Interface)* yang ekspresif (Rozi dkk., 2025). Lebih lanjut, dalam konteks pengembangan aplikasi cerdas, Flutter mendukung integrasi model *machine learning* (seperti TensorFlow Lite) secara efisien, memungkinkan inferensi model berjalan stabil pada perangkat seluler dengan latensi rendah.

2.10. *Unified Modeling Language*

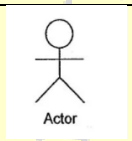
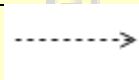
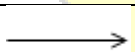
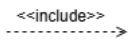
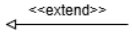
Unified Modeling Language (UML) adalah standar pemodelan visual yang digunakan secara luas dalam rekayasa perangkat lunak untuk merepresentasikan struktur, perilaku, dan interaksi suatu sistem. UML membantu pengembang, analis, serta pemangku kepentingan dalam memahami kebutuhan sistem dan mendokumentasikan desain sebelum proses implementasi dimulai. Sebagai bahasa pemodelan yang bersifat umum *(general-purpose modeling language)*, UML dapat diterapkan pada berbagai jenis sistem, baik berbasis web, *mobile*, maupun aplikasi desktop (Wayahdi & Ruziq, 2023). Melalui representasi visual yang terstandarisasi, UML mampu menyederhanakan kompleksitas sistem dan memberikan cetak biru *(blueprint)* yang mencakup proses bisnis, rancangan kelas, desain basis data, serta komponen-komponen pendukung dalam pengembangan perangkat lunak (Siska Narulita dkk., 2024).

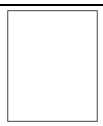
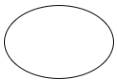
Secara praktis, UML menyediakan beberapa jenis diagram yang masing-masing memiliki fungsi spesifik, seperti *Use Case Diagram* untuk menggambarkan kebutuhan fungsional pengguna, *Activity Diagram* untuk memodelkan alur kerja, *Sequence Diagram* untuk menjelaskan interaksi antar objek berdasarkan waktu, serta *Class Diagram* untuk memodelkan struktur data dan relasi antar kelas. Penggunaan kombinasi berbagai diagram ini memungkinkan perancangan sistem yang lebih komprehensif, terstruktur, dan mudah diverifikasi (Wayahdi & Ruziq, 2023).

2.10.1. *Use Case Diagram*

Use Case Diagram merupakan salah satu diagram dalam UML yang digunakan untuk menggambarkan hubungan antara pengguna *(actor)* dengan fungsionalitas yang disediakan oleh sistem. Tujuannya adalah untuk menunjukkan apa saja yang dapat dilakukan oleh pengguna terhadap sistem. Dengan demikian, *use case diagra*m berperan penting dalam tahap analisis dan perancangan sistem karena mampu menjelaskan ruang lingkup sistem serta kebutuhan pengguna secara jelas dan terstruktur. Secara umum, sebuah use case diagram terdiri atas tiga elemen utama, yaitu aktor, *use case*, dan hubungan *(relationship)*(Noe dkk., 2025). Adapun simbol yang digunakan dalam penyusunan *use case diagram* dapat dilihat secara rinci dalam Tabel 1.

Tabel 1. Simbol *Use Case Diagram*

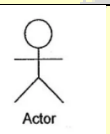
No	Gambar	Nama	Keterangan
1		*Actor*	Menspesifikasikan himpunan peran yang pengguna mainkan ketika berinteraksi dengan *use case.*
2		*Dependency*	Hubungan dimana perubahan yang terjadi pada suatu elemen mandiri *(dependent)* akan mempengaruhi elemen yang bergantung padanya elemen yang tidak mandiri *(independent).*
3		*Generalization*	Hubungan dimana objek anak *(descendent)* berbagi perilaku dan struktur data dari objek yang ada di atasnya objek induk *(ancestor).*
4		*Include*	Menspesifikasikan bahwa *use case* sumber secara eksplisit.
5		*Extend*	Menspesifikasikan bahwa *use case* target memperluas perilaku dari *use case* sumber pada suatu titik yang diberikan.

No	Gambar	Nama	Keterangan
6		*Association*	Apa yang menghubungkan antara objek yang satu dengan objek lainnya.
7		*System*	Menspesifikasikan paket yang menampilkan sistem secara terbatas.
8		*Use Case*	Deskripsi dari urutan aksi-aksi yang ditampilkan sistem yang menghasilkan suatu hasil yang terukur bagi suatu aktor.

2.10.2. *Sequence Diagram*

Sequence Diagram adalah diagram perilaku yang digunakan untuk memodelkan urutan interaksi antar objek dalam suatu sistem berdasarkan alur waktu. Diagram ini menekankan pada pesan yang dipertukarkan antar objek serta urutan kronologis dari proses yang terjadi (Wayahdi & Ruziq, 2023). *Sequence Diagram* menjadi elemen penting dalam perancangan perangkat lunak karena memberikan representasi visual yang jelas tentang bagaimana suatu *use case* diimplementasikan pada level objek. Penjelasan mengenai simbol-simbol yang digunakan dalam *Sequence Diagram* dapat dilihat pada Tabel 2.

Tabel 2. Simbol *Sequence Diagram*

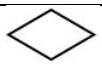
No	Gambar	Nama	Keterangan
1		*Actor*	Mengspesifikasikan peran yang pengguna mainkan ketika berinteraksi dengan sesama *actor* atau *lifeline*
2		*Entity Class*	Menggambarkan sistem sebagai landasan dalam menyusun basis data
3		*Boundary Class*	Menangani komunikasi antar linkungan sistem
4		*Control Class*	Bertanggung jawab terhadap kelas-kelas terhadap objek yang berisi logika

No	Gambar	Nama	Keterangan
5		*Lifeline*	Objek *entity*, antarmuka yang saling berinteraksi
6		*Message*	Spesifikasi dari komunikasi antar objek yang memuat informasi tentang aktivitas yang terjadi

2.10.3. *Activity Diagram*

Activity Diagram adalah diagram yang menggambarkan alur proses atau rangkaian aktivitas dalam suatu sistem yang dikembangkan, mulai dari tahapan awal, keputusan-keputusan yang terjadi selama proses berlangsung, hingga penyelesaian dari proses tersebut (Siska Narulita dkk., 2024). Diagram ini menitikberatkan pada alur kontrol dari satu aktivitas ke aktivitas lainnya, yang menggambarkan proses yang berjalan berdasarkan pemrosesan internal. *Activity diagram* bersifat hierarkis, artinya setiap aktivitas dapat tersusun atas tindakan-tindakan lebih kecil atau sub-aktivitas beserta aliran objek yang berhubungan di dalamnya (Wayahdi & Ruziq, 2023). Representasinya yang menyerupai *flowchart* membuat *activity diagram* mudah dipahami oleh analis maupun pemangku kepentingan non-teknis. Adapun simbol yang digunakan dalam penyusunan *activity diagram* dapat dilihat secara rinci dalam Tabel 3.

Tabel 3. Simbol *Activity Diagram*

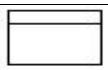
No	Gambar	Nama	Keterangan
1		*Initial State*	Tindakan awal atau titik awal aktiitas pada *aktivity diagram*
2		*Activiy*	Menunjukan aktivitas yang dilakukan atau yang sedang terjadi pada *activity diagram*
3		*Decision*	Titik yang mengindikasian suatu kondisi dimana ada kemungkinan perbedaan transisi atau percabangan

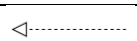
No	Gambar	Nama	Keterangan
4		*Join*	Berfungsi menggabungkan aktivitas yang paralel
5		*Final State*	Menunjukan bagian akhir dari aktivitas
6		*Swimlane*	Digunakan untuk memecah *activity diagram* menjadi baris dan kolom untuk membagi tanggung jawab objek yang melakukan aktivitas

2.10.4. *Class Diagram*

Class Diagram merupakan salah satu diagram struktural paling fundamental dalam *Unified Modeling Language* (UML) yang berfungsi menggambarkan struktur sistem melalui kumpulan kelas, atribut, operasi, serta hubungan atau interaksi antar kelas dalam suatu perangkat lunak (Aurellia, 2025). Diagram ini digunakan untuk memodelkan representasi struktur sistem yang mencakup kemampuan dalam menyimpan data dan menjalankan operasi, serta menjadi landasan penting dalam tahap pengembangan perangkat lunak selanjutnya, seperti pada fase perancangan (Wayahdi & Ruziq, 2023). Penjelasan mengenai simbol-simbol yang digunakan dalam *class diagram* dapat dilihat pada Tabel 4.

Tabel 4. Simbol *Class Diagram*

No	Gambar	Nama	Keterangan
1		*Generalization*	Hubungan dimana objek anak *(descendent)* berbagi perilaku dan struktur data dari objek yang ada di atasnya objek induk *(ancestor).*
2		*Nary Association*	Upaya untuk menghindari asosiasi dengan lebih dari dua objek.
3		*Class*	Himpunan dari objek-objek yang berbagi atribut serta operasi yang sama.

No	Gambar	Nama	Keterangan
4		*Realization*	Merincikan bahwa *use case* sumber secara eksplisit.
5		*Dependency*	Hubungan dimana perubahan yang terjadi pada suatu elemen mandiri *(dependent)* akan mempengaruhi elemen yang bergantung padanya elemen yang tidak mandiri *(independent).*
6		*Association*	Apa yang menghubungkan antara objek yang satu dengan objek lainnya.

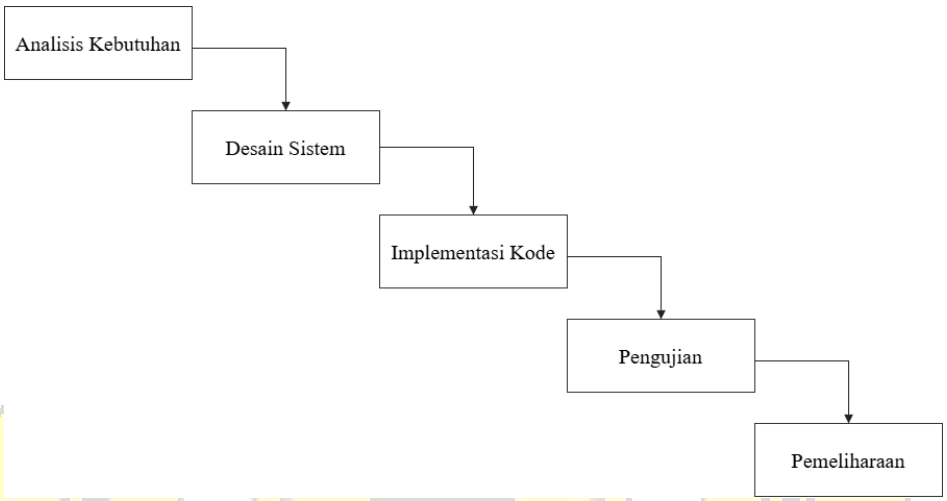
2.11. *Software Development Life Cycle (SDLC)*

Software Development Life Cycle (SDLC) merupakan kerangka kerja metodologis dalam rekayasa perangkat lunak yang menyediakan panduan sistematis untuk seluruh siklus hidup pengembangan sistem, mulai dari inisiasi hingga penggantian sistem (Prasetyo, 2024). Tujuan utama penerapan SDLC adalah untuk menghasilkan perangkat lunak berkualitas tinggi yang tidak hanya memenuhi ekspektasi pengguna, tetapi juga terselesaikan dalam estimasi waktu dan biaya yang direncanakan. Tahapan standar dalam siklus ini meliputi perencanaan *(planning)*, analisis kebutuhan, perancangan *(design),* implementasi kode, pengujian *(testing)*, serta pemeliharaan *(maintenance)* (Banin, 2021). Dalam penelitian ini, SDLC diterapkan untuk menjamin bahwa proses pengembangan aplikasi edukasi batik berbasis *mobile* berjalan secara terstruktur, efisien, dan terdokumentasi dengan baik.

2.12. Metode *Waterfall*

Metode *Waterfall* merupakan salah satu model pengembangan dalam SDLC yang bersifat linear dan sekuensial, dengan setiap fase harus diselesaikan sepenuhnya sebelum melangkah ke fase berikutnya. Alur kerjanya mengalir ke bawah seperti air terjun, dimulai dari analisis kebutuhan, desain sistem, implementasi kode, pengujian, hingga pemeliharaan (Adi Kurniyanti & Murdiani, 2022). Model ini sangat relevan digunakan dalam penelitian atau proyek yang

memiliki kebutuhan sistem yang jelas dan tidak banyak berubah sejak awal. Keunggulan utama metode ini adalah kemudahan dalam manajemen proyek dan dokumentasi yang rapi pada setiap tahapannya. Untuk alur kerja dari metode *waterfall* dapat dilihat pada Gambar 15.



Gambar 15. Alur Kerja Metode *Waterfall*

2.12.1. Analisis Kebutuhan

Analisis kebutuhan adalah tahapan kritis dalam model *Waterfall* Model yang memusatkan perhatian pada identifikasi, *elicitation*, dan dokumentasi kebutuhan sistem sebelum fase desain atau implementasi dimulai untuk menentukan fondasi bagi seluruh siklus pengembangan (Wahyudi dkk., 2021). Studi literatur terbaru menunjukkan bahwa kualitas kebutuhan seperti kelengkapan, konsistensi, dan *traceability* sangat memengaruhi keberhasilan proyek perangkat lunak ketika kebutuhan dikumpulkan dan divalidasi secara sistematis, maka risiko perubahan besar di tengah jalan atau *mismatch* antara sistem dan kebutuhan pengguna dapat diminimalkan.

2.12.2. Desain Sistem

Tahap Desain Sistem merupakan proses menerjemahkan kebutuhan yang telah didefinisikan pada fase analisis menjadi rancangan teknis yang jelas, terstruktur, dan dapat diimplementasikan oleh pengembang perangkat lunak. Pada tahap ini dilakukan perancangan arsitektur sistem, model data, alur proses,

antarmuka pengguna, serta spesifikasi komponen perangkat lunak yang akan dibangun, sehingga keseluruhan fungsi sistem dapat divisualisasikan sebelum proses coding dimulai (Setyawati & Adrimuna, 2022). Desain sistem berperan sebagai cetak biru *(blueprint)* yang memastikan bahwa solusi yang dikembangkan sesuai dengan kebutuhan pengguna, konsisten secara teknis, serta dapat dikelola dan diuji pada tahap berikutnya.

2.12.3. Implementasi

Tahap Implementasi merupakan fase penerjemahan rancangan sistem yang telah disusun pada tahap desain menjadi kode program menggunakan bahasa pemrograman, *framework*, dan teknologi yang telah ditentukan (Setyawati & Adrimuna, 2022). Pada tahap ini, setiap modul dan komponen perangkat lunak dibangun sesuai spesifikasi teknis yang telah didokumentasikan, kemudian disatukan menjadi suatu sistem yang fungsional. Implementasi dilakukan dengan mengikuti standar penulisan kode, prinsip pengembangan perangkat lunak, serta praktik terbaik seperti modularitas, efisiensi, dan *maintainability* agar sistem mudah dikelola dan dikembangkan pada tahap berikutnya.

2.12.4. Pengujian

Tahap Pengujian adalah proses evaluasi dan verifikasi sistem yang telah diimplementasikan untuk memastikan bahwa perangkat lunak berjalan sesuai dengan spesifikasi dan kebutuhan yang telah ditetapkan pada fase analisis (Wahyudi dkk., 2021). Pengujian meliputi berbagai jenis, seperti *unit testing*, *integration testing*, *system testing*, dan *acceptance testing*, yang bertujuan untuk menemukan kesalahan, *bug*, atau ketidaksesuaian fungsional. Proses ini memastikan kualitas perangkat lunak, meningkatkan keandalan, dan meminimalkan risiko kegagalan sistem ketika digunakan oleh pengguna akhir.

2.12.5. Pemeliharaan

Tahap Pemeliharaan *(maintenance)* pada model Waterfall adalah fase terakhir yang berfokus pada perbaikan, pembaruan, dan peningkatan sistem setelah perangkat lunak resmi digunakan oleh pengguna (Setyawati & Adrimuna, 2022). Aktivitas pada tahap ini meliputi koreksi kesalahan *(error correction)*, penyesuaian

terhadap perubahan kebutuhan, optimasi performa, serta penambahan fitur baru jika diperlukan. Pemeliharaan bertujuan untuk memastikan sistem tetap berfungsi dengan baik, andal, dan relevan sesuai perkembangan kebutuhan pengguna atau lingkungan operasional.

2.13. *Black Box Testing*

Black Box Testing adalah metode pengujian perangkat lunak yang berfokus pada evaluasi fungsi sistem tanpa melihat struktur internal atau kode program. Pendekatan ini menilai apakah input yang diberikan menghasilkan output sesuai dengan kebutuhan dan spesifikasi yang telah ditetapkan (Abdi, 2024). Metode *black box testing* mampu mengidentifikasi berbagai jenis kesalahan, seperti fungsi yang tidak berjalan dengan benar atau tidak tersedia, kekeliruan pada basis data maupun struktur data, kesalahan dalam akses data, masalah antarmuka, kesalahan pengguna, gangguan performa, serta kesalahan pada proses inisialisasi dan terminasi. Pendekatan ini berperan sebagai pelengkap pengujian *white box testing* karena dapat mengungkap jenis kesalahan yang tidak terdeteksi melalui pengujian berbasis struktur internal program (Aurellia, 2025).

Penerapan *Black Box Testing* biasanya dilakukan melalui beberapa jenis pendekatan seperti *equivalence partitioning*, *boundary value analysis, decision table testing*, hingga *state transition testing*. Setiap pendekatan memungkinkan penguji untuk merancang kasus uji secara sistematis berdasarkan variasi input dan kondisi yang mungkin terjadi pada sistem (Abdi, 2024).

2.14. Penelitian Sejenis

Dalam penyusunan penelitian ini, dilakukan tinjauan terhadap beberapa literatur terdahulu yang relevan dengan topik klasifikasi citra dan implementasi *deep learning* pada motif batik. Kajian ini bertujuan untuk mengetahui perkembangan metode terkini, hasil yang telah dicapai, serta sebagai bahan perbandingan untuk menentukan metode yang paling tepat diterapkan dalam penelitian ini. Rangkuman dari beberapa penelitian sejenis yang menjadi acuan utama disajikan secara ringkas dalam Tabel 5.

Tabel 5. Penelitian Sejenis

Peneliti	Judul	Metode	Hasil	Perbedaan
Khasanah, dkk (2023)	Implementasi *Data Augmentation Random Erasing* dan *GridMask* pada CNN untuk Klasifikasi Batik	CNN Arsitektur VGG-16	Hasil terbaik diperoleh pada *fine-tuning* VGG16 dengan GridMask (*rotate*=0) dengan akurasi 96,88%, lebih tinggi dibandingkan tanpa *augmentation* (91,67%) maupun *transfer learning* (85,42%). Implementasi *augmentation* terbukti membantu mengurangi *overfitting* meskipun akurasi menurun pada beberapa parameter.	Objek penelitian Arsitektur CNN Metode augmentasi data
Cahyati, dkk (2024)	Pengenalan Motif Songket Palembang Menggunakan *Convolutional Neural Network* dengan Arsitektur ResNet-50	CNN Arsitektur ResNet50	ResNet-50 diuji dengan beberapa *optimizer* pada klasifikasi motif Songket Palembang. Adam Optimizer memberikan hasil terbaik dengan akurasi 0,96, lebih tinggi dibandingkan SGD dan AdaGrad (0,92), serta menunjukkan *precision, recall*, dan *F1-score*	Objek penelitian Arsitektur CNN Implementasi

Peneliti	Judul	Metode	Hasil	Perbedaan
			yang lebih konsisten.	
Dani, dkk (2024)	Klasifikasi Motif Batik Yogyakarta Menggunakan Metode GLCM Dan CNN	CNN + GLCM	Kombinasi GLCM dan CNN MobileNetV2 berhasil mencapai akurasi 99% dalam klasifikasi motif Batik Yogyakarta	Objek penelitian Metode CNN standar Tidak menggunakan *transfer learning*
Yonathan, dkk (2025)	Modifikasi Arsitektur dalam *Convolutional Neural Network* untuk Klasifikasi Batik Lampung dan Batik Yogyakarta	CNN Arsitektur MobileNet V2, DenseNet 121, Xception	Model terbaik adalah Xception Modified dengan akurasi 100%, Presisi 97%, *Recall* 94%, *F1-Score* 92%, dan *Loss* 0,0066 pada *epoch* 30 & *learning rate* 0,0001. Sementara Xception (original) memperoleh akurasi 97,53%.	Objek penelitian Fokus pada perbandingan model Tidak diimplementasikan pada aplikasi mobile
Utama, dkk (2025)	Identifikasi Jenis Batik Menggunakan Metode *Convolutional Neural Network* (CNN) Berbasis Website	CNN Arsitektur MobileNet V2	Penelitian menerapkan CNN arsitektur MobileNetV2 untuk identifikasi jenis batik berbasis *website* Flask. Setelah proses *training* selama 10 *epoch* dengan *image processing* dan *augmentasi data*, model mencapai akurasi 92%, baik pada *training*	Objek penelitian Implementasi pada website Tidak menggunkan *transfer learning*

Peneliti	Judul	Metode	Hasil	Perbedaan
			maupun *confusion matrix*. Model berhasil diimplementasikan dalam sistem identifikasi batik berbasis *website*.	

Berdasarkan kajian terhadap penelitian-penelitian terdahulu, dapat disimpulkan bahwa sebagian besar studi terkait klasifikasi motif batik masih berfokus pada peningkatan performa model melalui perbandingan arsitektur CNN dan optimisasi parameter, namun belum diintegrasikan ke dalam aplikasi edukatif yang dapat dimanfaatkan langsung oleh pengguna umum. Selain itu, penelitian spesifik mengenai klasifikasi motif Batik Pekalongan masih sangat terbatas, padahal daerah tersebut memiliki kekayaan motif yang kompleks dan beragam. Di sisi lain, pemanfaatan arsitektur CNN yang ringan dan efisien seperti MobileNetV2 untuk implementasi model pada perangkat *mobile* juga masih jarang dilakukan. Oleh karena itu, penelitian ini memiliki kebaruan berupa pengembangan aplikasi edukatif berbasis *mobile* yang terintegrasi dengan model klasifikasi motif Batik Pekalongan menggunakan arsitektur MobileNetV2 melalui pendekatan *transfer learning*, sehingga tidak hanya memberikan kontribusi pada aspek teknis pemodelan, tetapi juga pada aspek edukasi dan pelestarian budaya secara praktis.

BAB III
METODE PENELITIAN

3.1. Waktu dan Tempat Penelitian

Penelitian Tugas Akhir ini direncanakan berlangsung selama lima bulan, mulai November 2025 hingga April 2026. Kegiatan penelitian dilaksanakan di Fakultas Teknik, Universitas Jenderal Soedirman, yang berlokasi di Jl. Raya Mayjen Sungkono KM 5, Dusun 2, Blater, Kecamatan Kalimanah, Kabupaten Purbalingga, Jawa Tengah 53371.

3.2. Data dan Alat Penelitian

Pelaksanaan penelitian ini didukung oleh dua komponen utama, yaitu data sebagai objek pembelajaran model dan alat komputasi sebagai perangkat pengolahan.

3.2.1. Data Penelitian

Penelitian ini menggunakan dua jenis sumber data, yakni data primer dan data sekunder. Data primer dikumpulkan melalui proses dokumentasi langsung terhadap koleksi kain di Museum Batik Pekalongan. Sementara itu, data sekunder diperoleh dari repositori digital Google Arts & Culture, laman resmi Museum Kemdikbud, serta rujukan dataset "Batik Utara Jawa" yang disusun oleh Faisal dkk. (2023) dengan total 805 citra. Dataset sekunder tersebut mencakup berbagai kategori motif seperti Jlamprang, Buketan, Mega Mendung, Tujuh Rupa, Liong, Singa Barong, dan Negative. Secara keseluruhan, penelitian ini mengolah 454 data yang terbagi ke dalam tujuh label kelas, yaitu Pagi-sore, Jlamprang, Jawa Hokokai, Tirtateja, Terang Bulan, Buketan, dan Lunglungan, dengan rincian distribusi sebagaimana tersaji pada Tabel 6.

Tabel 6. Data Penelitian

Kelas/Label	Dataset		Total
	Primer	Sekunder	
Buketan	35	76	111
Jlamprang	19	53	72
Lunglungan	16	47	63

Kelas/Label	Dataset		Total
	Primer	Sekunder	
Tirtateja	16	37	53
Hokokai	19	60	79
Pagisore	14	59	73
Terang Bulan	26	51	76
Non Batik	0	89	89
Total	**146**	**470**	**616**

3.2.2. Alat Penelitian

Alat yang digunakan dalam penelitian ini terdiri dari perangkat keras dan perangkat lunak pendukung. Perangkat keras yang digunakan adalah laptop Lenovo IdeaPad Slim 3 dengan spesifikasi prosesor AMD Ryzen 5 5500U dengan Radeon Graphics 2.10 GHz, RAM 8 GB, dan sistem operasi Windows 10 64-bit.

Perangkat lunak utama yang digunakan meliputi bahasa pemrograman Python dengan *framework* TensorFlow dan Keras untuk pelatihan dan evaluasi model *deep learning*. Google Colab digunakan sebagai lingkungan komputasi berbasis *cloud* untuk proses pelatihan model. Untuk pengembangan aplikasi *mobile*, digunakan Flutter sebagai *framework* UI dan Android Studio sebagai IDE untuk kompilasi, *debugging*, dan pengujian aplikasi pada perangkat Android. Sebagai basis data offline, digunakan SQLite yang disimpan langsung di dalam aplikasi untuk memuat informasi motif Batik Pekalongan serta data pengguna tanpa memerlukan koneksi internet.

3.3. Tahap Penelitian

Penelitian ini menggunakan pendekatan kuantitatif eksperimen dengan fokus pada pengujian kinerja model *Convolutional Neural Network* MobileNetV2 dalam melakukan klasifikasi motif Batik Pekalongan. Pendekatan kuantitatif dipilih karena efektivitas model dapat diukur secara objektif melalui metrik evaluasi seperti akurasi, presisi, *recall*, dan *F1-score*. Teknik *transfer learning* digunakan untuk meningkatkan performa model pada dataset yang terbatas, sedangkan pendekatan eksperimen dilakukan melalui proses pelatihan, pengujian, dan analisis

hasil klasifikasi. Model yang diperoleh kemudian diintegrasikan ke dalam aplikasi edukatif berbasis mobile sebagai media pembelajaran interaktif terkait motif Batik Pekalongan. Selain itu, penelitian ini menerapkan metode pengumpulan data serta metode pengembangan sistem untuk memastikan seluruh rangkaian kegiatan penelitian berlangsung secara terencana, sistematis, dan selaras dengan kebutuhan pengguna.

3.3.1. Metode Pengumpulan Data

Untuk memperoleh data yang representatif dan landasan teori yang kokoh, penelitian ini menggunakan beberapa teknik pengumpulan data sebagai berikut:

a. Studi Literatur

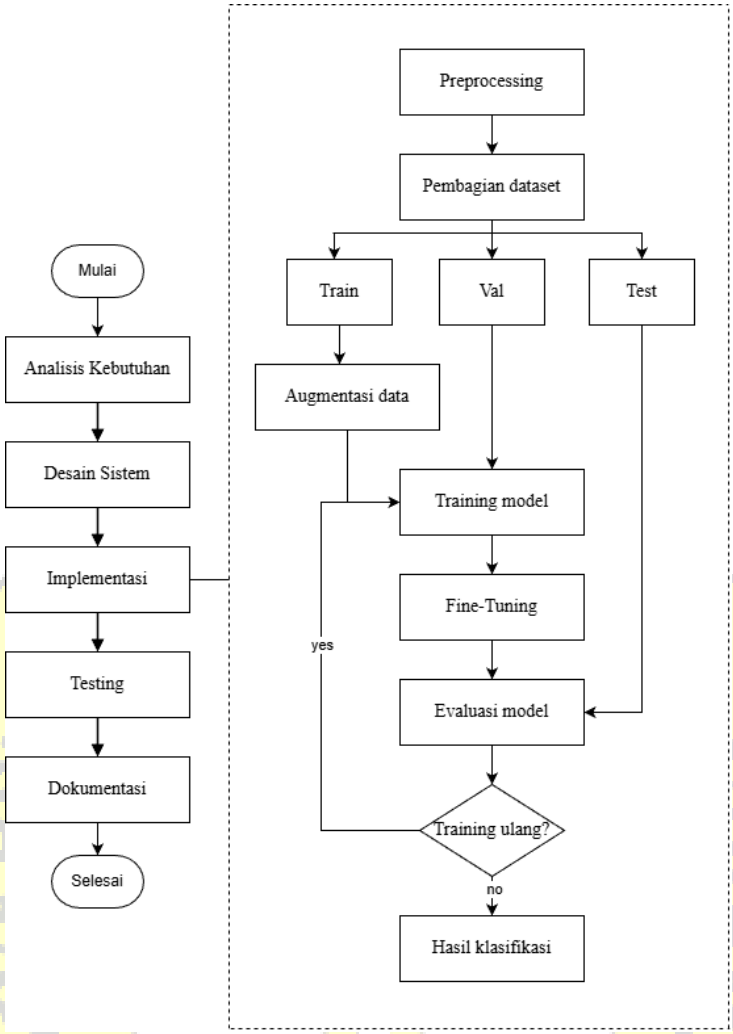
Melakukan tinjauan mendalam terhadap berbagai sumber pustaka ilmiah seperti jurnal, artikel, dan penelitian relevan sebelumnya untuk membangun landasan teoretis yang kuat. Tinjauan ini mencakup konsep *deep learning*, arsitektur CNN MobileNetV2, metode *transfer learning*, dan studi kasus klasifikasi citra pada domain serupa.

b. Akuisisi Data Citra

Pengumpulan data citra dilakukan melalui dua metode utama untuk memastikan keragaman variasi visual. Pertama, dokumentasi langsung terhadap koleksi fisik di Museum Batik Pekalongan. Kedua, eksplorasi repositori daring terverifikasi digunakan untuk melengkapi variasi corak dan warna yang tidak tersedia secara fisik di museum saat pengambilan data.

3.3.2. Metode Pengembangan Sistem

Metode pengembangan yang digunakan dalam pengembangan aplikasi ini adalah Model *Waterfall*. Model ini dipilih karena menyediakan alur kerja pengembangan perangkat lunak (SDLC) yang sistematis dan sekuensial, sangat cocok untuk proyek dengan kebutuhan fungsional yang telah terdefinisi secara jelas di awal. Model ini membingkai keseluruhan proses dari analisis hingga implementasi produk akhir, dengan metodologi penelitian *machine learning* dijalankan sebagai inti dari tahap implementasi. Keseluruhan alur kerja metode pengambangan sistem ini dapat divisualisasikan pada Gambar 16.



Gambar 16. *Flowchart* Penelitian

1) Analisis Kebutuhan

Tahap awal ini melibatkan proses identifikasi kebutuhan fungsional dan non-fungsional dari sistem. Kebutuhan fungsional meliputi fitur utama seperti identifikasi motif batik menggunakan kamera atau unggah galeri berbasis model MobileNetV2, penyajian informasi seperti sejarah motif, makna filosofis, ciri visual, serta galeri contoh variasi dari motif yang teridentifikasi. Kebutuhan nonfungsional mencakup aspek performa model, efisiensi sistem, dan kemudahan penggunaan. Data pendukung diperoleh melalui studi literatur serta observasi dari berbagai sumber, termasuk museum dan situs daring.

2) Perancangan Sistem

Tahap ini berfokus pada pembuatan rancangan awal sistem, mencakup pemodelan proses bisnis dan rancangan arsitektur sistem. Pemodelan dilakukan menggunakan *Unified Modeling Language* (UML), yang terdiri atas *use case diagram, activity diagram, sequence diagram,* dan *class diagram*. Selain itu, pada tahap ini juga dilakukan desain antarmuka pengguna (UI/UX) untuk aplikasi *mobile*.

3) Implementasi

Pada tahap ini dilakukan dua aspek utama, yaitu implementasi sistem aplikasi *mobile* dan implementasi model klasifikasi.

a) Implementasi Model CNN MobileNetV2

Tahap ini mengintegrasikan proses pengolahan data citra dan pelatihan model kecerdasan buatan ke dalam sistem. Rangkaian prosesnya meliputi:

1. *Preprocessing Data*

   Proses ini diawali dengan standardisasi ukuran, yaitu semua gambar dalam dataset akan diubah ukurannya *(resize)* menjadi dimensi 224x224 piksel untuk menyesuaikan dengan arsitektur input MobileNetV2. Selanjutnya, nilai setiap piksel pada gambar akan dinormalisasi dari rentang [0, 255] ke rentang [0, 1]. Terakhir, label kategori yang berupa teks akan dikonversi menjadi format numerik melalui *One-Hot Encoding* agar kompatibel dengan output dari fungsi aktivasi *softmax* sebagaimana ditunjukan pada persamaan (3) dan fungsi *loss categorical_crossentropy*.

2. Pembagian Dataset

   Dataset akan dibagi secara proporsional menjadi tiga bagian, data latih (70%) untuk melatih model, data validasi (20%) untuk memonitor performa dan melakukan *tuning hyperparameter* selama pelatihan, dan data uji (10%) yang disimpan terpisah untuk pengujian akhir model. Pembagian ini dilakukan secara acak *(random split)* untuk memastikan setiap kelas tetap terdistribusi secara seimbang sehingga model tidak bias terhadap kelas tertentu.

3. Augmentasi Data

   Untuk mengatasi keterbatasan jumlah dataset dan mencegah *overfitting*, teknik augmentasi akan diterapkan secara langsung pada data latih. Operasi yang dilakukan meliputi *rotation* (untuk mengantisipasi kemiringan kamera pengguna), *brightness adjustments*, serta *zoom* dan *crop* (untuk fokus pada detail motif). Teknik *flip* juga diterapkan untuk memperbanyak data latih mengingat motif batik umumnya memiliki simetri visual.

4. *Training* Model dan *Fine Tuning*

   Model dibangun dengan memuat arsitektur MobileNetV2 yang telah dilatih pada dataset *ImageNet*. Arsitektur ini secara internal menggunakan fungsi aktivasi ReLU6 sebagaimana ditunjukkan dalam persamaan (2) pada lapisan-lapisan konvolusinya untuk mempelajari fitur-fitur citra. Lapisan klasifikasi teratas dari model asli tidak disertakan, dan sebagian besar lapisan konvolusi dari model dasar ini akan dibekukan untuk mempertahankan pengetahuan fitur visual yang sudah ada. Selanjutnya, sebuah *classifier head* baru akan ditambahkan di atasnya, yang terdiri dari lapisan *GlobalAveragePooling2D*, lapisan *Dropout* untuk regularisasi, dan lapisan *Dense* dengan aktivasi *softmax* yang ditunjukan pada persamaan (3) untuk 8 kelas motif. Setelah arsitektur lengkap terbentuk, model kemudian dilatih melalui dua tahap. Tahap pertama adalah melatih hanya *classifier head* yang baru ditambahkan. Setelah performa model stabil, tahap kedua yaitu *fine-tuning* dilakukan dengan *unfreezing* beberapa lapisan teratas dari model MobileNetV2 dan melanjutkan pelatihan dengan *learning rate* yang sangat kecil untuk menyesuaikan fitur yang lebih spesifik dengan dataset batik. Model akan dikompilasi menggunakan *optimizer* Adam, yang dipilih karena kemampuannya dalam melakukan adaptasi *learning rate* secara efisien dan konvergensinya yang cepat, sebagaimana telah terbukti efektif pada penelitian sejenis.

5. Evaluasi Model

Performa model dievaluasi dengan membandingkan hasil prediksi dari data uji dengan label aslinya. Evaluasi ini menggunakan *Confusion Matrix* untuk menghitung metrik utama seperti Akurasi, Presisi, *Recall*, dan *F1-Score* sebagaimana ditunjukkan pada persamaan (4) hingga (7). Analisis juga akan mencakup pengamatan kurva *loss* dan akurasi selama pelatihan untuk mendiagnosis potensi *overfitting* atau *underfitting*.

Model yang telah dilatih kemudian diintegrasikan ke dalam *framework* Flutter agar pengguna dapat mengunggah gambar batik dan memperoleh hasil klasifikasi serta informasi edukatif secara langsung.

b) Implementasi Aplikasi Mobile

Pada tahap ini akan dilakukan proses pengkodean aplikasi menggunakan *framework* Flutter, karena *framework* ini mampu menghasilkan antarmuka yang responsif, modern, serta mendukung kompatibilitas di berbagai versi sistem operasi Android. Pada tahap ini juga akan dilakukan proses penghubungan aplikasi dengan model CNN yang telah dilatih, yang sebelumnya telah dikonversi ke dalam format TensorFlow Lite.

4) Pengujian

Pengujian dilakukan menggunakan metode *Black Box Testing*, dengan fokus pada pengujian setiap fungsi utama seperti proses unggah citra batik, klasifikasi motif menggunakan model yang telah diintegrasikan, serta penampilan informasi edukatif terkait sejarah dan makna motif.

5) Dokumentasi

Seluruh proses penelitian, mulai dari analisis kebutuhan, perancangan sistem, implementasi, pengujian, akan didokumentasikan secara sistematis dalam laporan tugas akhir. Dokumentasi ini mencakup penjelasan teknis arsitektur aplikasi edukatif, integrasi model CNN MobileNetV2 ke dalam sistem berbasis *mobile*, serta hasil pengujian performa model dan aplikasi.

BAB IV
HASIL DAN PEMBAHASAN

4.1. Analisis Kebutuhan

Tahap analisis kebutuhan dilakukan dengan analisis mandiri terhadap segala kebutuhan dari pengguna dan sistem. Analisis tersebut didasarkan pada kemungkinan masalah pengguna yang dapat diselesaikan dengan sistem dan fitur-fitur dari sistem yang dapat menyelesaikan masalah tersebut. Berikut penjabaran dari kebutuhan pengguna dan kebutuhan sistem.

4.2.1. Kebutuhan Pengguna

Pengguna yang berinteraksi langsung dengan sistem terdiri dari satu entitas, yaitu entitas yang memasukkan data berupa gambar kepada sistem dan mengharapkan keluaran berupa hasil klasifikasi dari model. Penjabaran dari spesifikasi kebutuhan pengguna ditunjukkan pada Tabel 7.

Tabel 7. Spesifikasi Kebutuhan Pengguna

Kode	Kebutuhan
URS.001	Pengguna dapat mengambil gambar motif batik secara langsung menggunakan kamera perangkat android untuk proses klasifikasi.
URS.002	Pengguna dapat mengunggah gambar motif batik yang tersimpan di dalam galeri perangkat ke dalam aplikasi.
URS.003	Pengguna dapat melihat hasil identifikasi jenis motif batik beserta nilai persentase akurasi dari model.
URS.004	Pengguna dapat mengakses fitur galeri motif yang menyajikan informasi detail serta referensi gambar batik yang tersedia oleh sistem.
URS.005	Pengguna dapat menjalankan fungsi utama klasifikasi pada aplikasi tanpa memerlukan koneksi jaringan internet.
URS.006	Pengguna dapat melihat petunjuk visual menggunakan kamera untuk memastikan posisi batik presisi.

4.2.2. Kebutuhan Sistem

Kebutuhan fungsional sistem dirancang untuk menjembatani interaksi antara input pengguna dengan *output* yang dihasilkan oleh model. Sistem wajib menyediakan fitur-fitur yang relevan dengan kebutuhan pengguna sekaligus memastikan model dapat terimplementasi secara optimal. Penjabaran lebih lanjut mengenai kebutuhan sistem ditunjukkan pada Tabel 8.

Tabel 8. Spesifikasi Kebutuhan Sistem

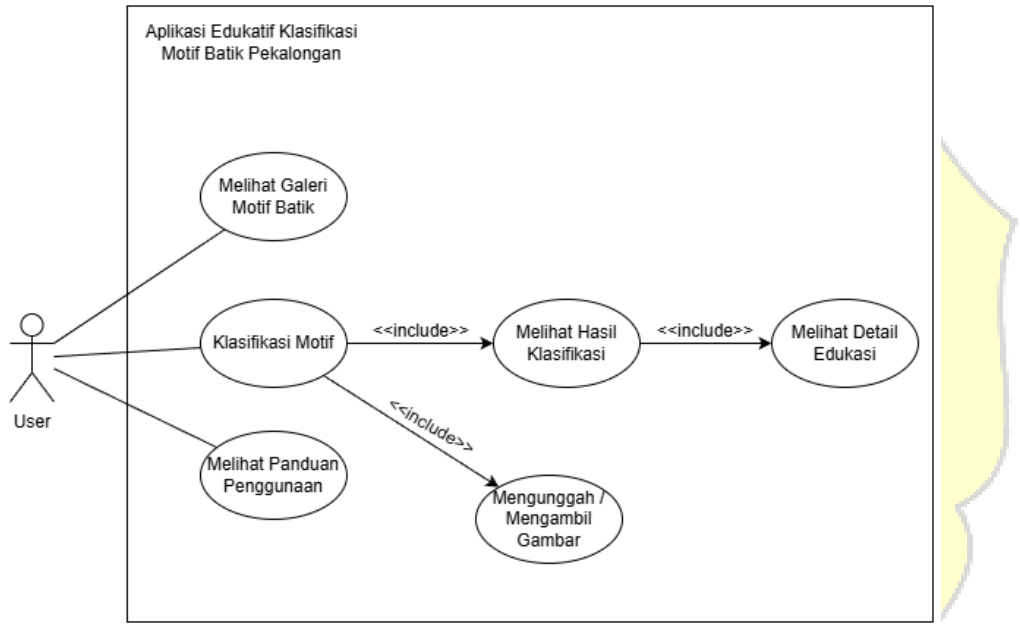
Kode	Kebutuhan
SRS.001	Sistem harus meminta dan memvalidasi izin akses kamera (*Android Permission*) sebelum mengaktifkan fitur pengambilan gambar langsung.
SRS.002	Sistem harus meminta izin akses media penyimpanan (*Read External Storage*) untuk mengambil gambar dari galeri lokal perangkat.
SRS.003	Sistem harus menyediakan basis data lokal yang berisi informasi detail untuk 7 jenis motif batik yang didukung.
SRS.004	Sistem harus mengintegrasikan TensorFlow Lite sebagai mesin inferensi untuk menjalankan model klasifikasi dalam format (.tflite).
SRS.005	Sistem harus melakukan *preprocessing* citra yang meliputi penyesuaian ukuran dan normalisasi nilai piksel sesuai dengan kebutuhan model sebelum proses inferensi dilakukan.
SRS.006	Sistem harus menampilkan label motif dengan nilai persentase (c*onfidence score*) tertinggi sebagai hasil utama klasifikasi.
SRS.007	Sistem harus tetap dapat menjalankan fungsi klasifikasi meskipun perangkat berada dalam kondisi *offline*.
SRS.008	Sistem harus memvalidasi format file gambar yang diunggah dan hanya menerima format .jpg, .jpeg, atau .png.
SRS.009	Sistem harus menerima masukan berupa citra dari kamera maupun galeri perangkat.
SRS.010	Sistem harus menampilkan pesan kesalahan apabila gambar tidak dapat diproses atau terjadi kegagalan klasifikasi.

4.2. Desain Sistem

Pada tahap desain sistem, aplikasi Klasifikasi Batik Pekalongan dimodelkan menggunakan standar *Unified Modeling Language* (UML), yang terdiri atas *use case diagram, activity diagram, sequence diagram*, dan *class diagram*.

4.2.1. *Use Case Diagram*

Use case diagram digunakan untuk mendefinisikan interaksi yang terjadi antara aktor dengan sistem serta memberikan gambaran mengenai fungsionalitas aplikasi dari sudut pandang pengguna. Adapun *use case diagram* pada aplikasi *Mbatic: Mobile Batik Classifier* digambarkan pada Gambar 17.

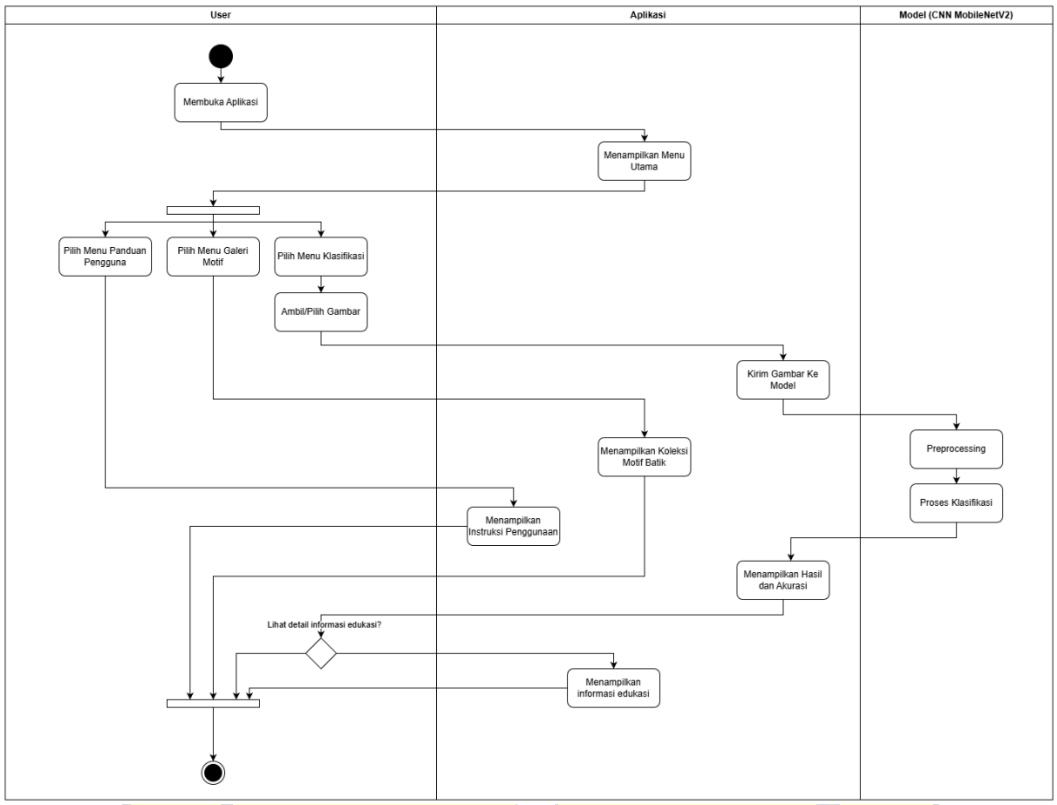


Gambar 17. *Use Case Diagram* Aplikasi Klasifikasi Batik Pekalongan

Pada Gambar 17, terdapat satu aktor utama yaitu *User* yang dapat melakukan beberapa aktivitas utama, yaitu melihat galeri motif batik, melakukan klasifikasi motif, serta melihat panduan penggunaan aplikasi. Fungsionalitas utama berpusat pada *use case* Klasifikasi Motif. Dalam proses klasifikasi, terdapat relasi <<*include*>> yang menunjukkan bahwa *use case* Klasifikasi Motif mencakup proses Mengunggah/Mengambil Gambar sebagai tahap awal sebelum inferensi dilakukan. Setelah sistem memproses gambar menggunakan model klasifikasi, pengguna dapat Melihat Hasil Klasifikasi yang juga memiliki relasi <<*include*>> terhadap fitur Melihat Detail Edukasi yang berisi informasi mengenai motif batik

yang teridentifikasi. Di luar alur utama tersebut, pengguna juga dapat mengakses fitur pendukung secara mandiri, yaitu Melihat Galeri Motif Batik sebagai sarana eksplorasi motif batik serta Melihat Panduan Penggunaan untuk memahami cara operasional aplikasi.

4.2.2. *Activity Diagram*



Gambar 18. *Activity Diagram* Klasifikasi Batik Pekalongan

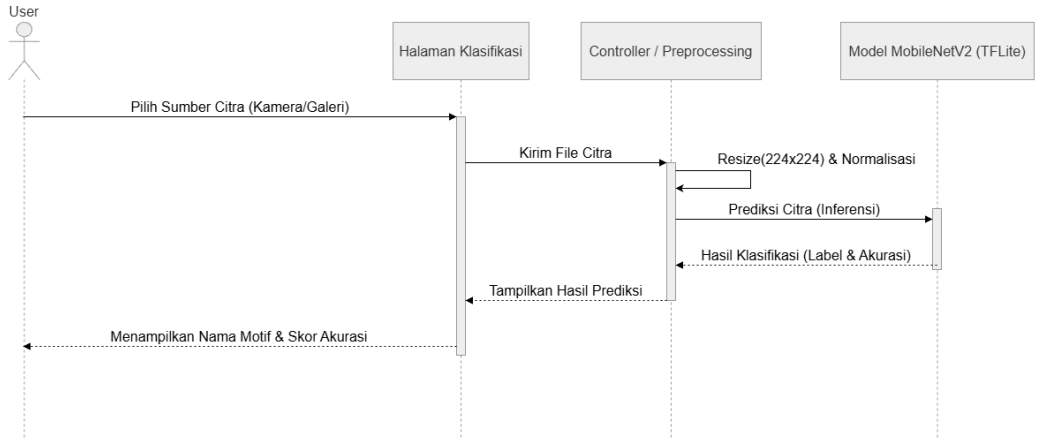
Activity diagram tersebut menggambarkan alur interaksi antara *User*, Aplikasi, dan Model CNN MobileNetV2 dalam sistem *Mbatic: Mobile Batik Classifier*. Proses dimulai ketika pengguna membuka aplikasi dan sistem menampilkan menu utama. Pengguna kemudian dapat memilih menu panduan penggunaan, galeri motif, atau klasifikasi. Jika pengguna memilih menu klasifikasi, pengguna mengambil atau memilih gambar sebagai input. Gambar tersebut dikirim ke model untuk melalui tahap *preprocessing* dan proses klasifikasi. Model kemudian menghasilkan prediksi berupa label motif beserta nilai probabilitas, yang selanjutnya ditampilkan oleh aplikasi kepada pengguna. Setelah hasil klasifikasi

ditampilkan, pengguna dapat memilih untuk melihat detail informasi edukasi terkait motif yang teridentifikasi atau mengakhiri proses. Secara keseluruhan, diagram ini menunjukkan bahwa proses klasifikasi menjadi alur utama sistem, sedangkan fitur galeri dan panduan berfungsi sebagai fitur pendukung.

4.2.3. *Sequence Diagram*

Sequence diagram digunakan untuk menggambarkan interaksi antar objek dalam sistem secara berurutan berdasarkan urutan waktu. Berikut adalah rincian *sequence diagram* pada aplikasi *Mobile Batik Classifier*:

a. *Sequence Diagram* Klasifikasi Motif Batik

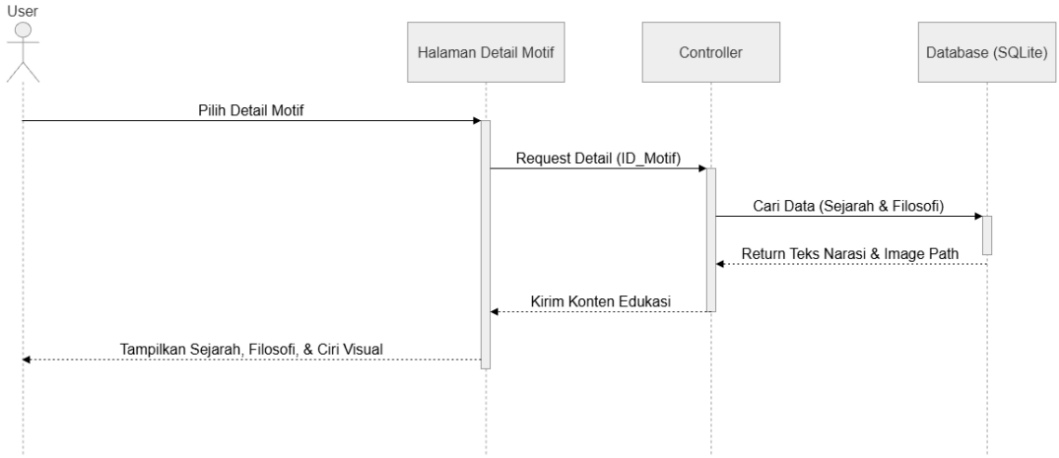


Gambar 19. *Sequence Diagram* Klasifikasi Motif Batik

Proses dimulai ketika pengguna memilih sumber citra pada halaman klasifikasi. Halaman klasifikasi kemudian mengirimkan file citra ke bagian *Controller* untuk diproses lebih lanjut. Pada tahap ini, sistem melakukan penyesuaian ukuran gambar menjadi 224×224 piksel serta normalisasi nilai piksel agar sesuai dengan kebutuhan model yaitu rentang -1 sampai 1.

Setelah tahap *preprocessing* selesai, citra dikirim ke Model MobileNetV2 untuk dilakukan proses inferensi. Model kemudian menghasilkan keluaran berupa label prediksi dan *confidence score*, yang dikembalikan ke *Controller*. Selanjutnya, hasil tersebut diteruskan ke halaman klasifikasi untuk ditampilkan kepada pengguna dalam bentuk nama motif dan persentase prediksi.

b. *Sequence Diagram* Melihat Informasi Motif Batik

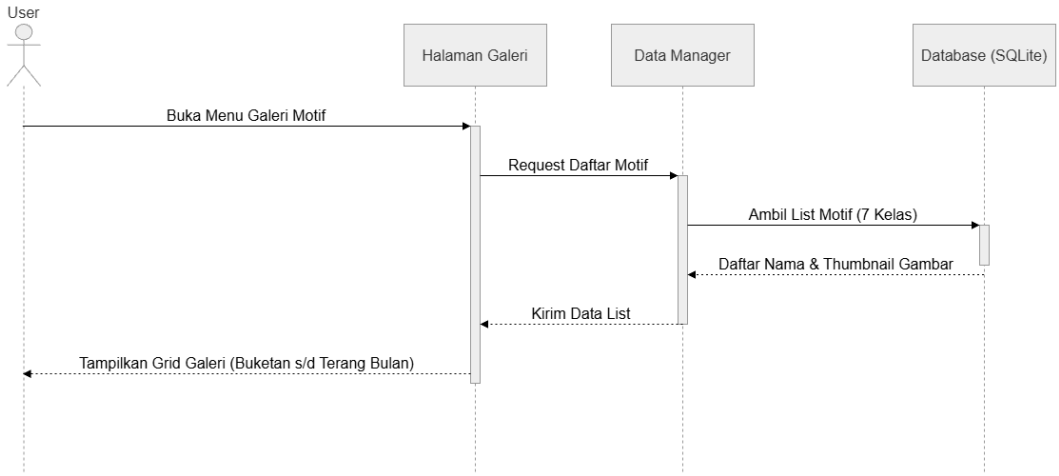


Gambar 20. *Sequence Diagram* Melihat Informasi Edukasi Batik

Proses dimulai saat pengguna memilih detail motif pada aplikasi. Permintaan tersebut diterima oleh Halaman Detail Motif, yang kemudian mengirimkan request berupa ID motif ke *Controller*. Selanjutnya, *Controller* melakukan pencarian data pada *database* yang berisi informasi sejarah, filosofi, serta elemen visual dari motif batik yang dipilih.

Setelah data ditemukan, *database* mengembalikan teks narasi dan *image path* kepada *Controller*. Data tersebut kemudian diteruskan ke Halaman Detail Motif untuk ditampilkan kepada pengguna dalam bentuk informasi sejarah, filosofi, dan ciri visual motif batik.

c. *Sequence Diagram* Melihat Galeri Motif Batik

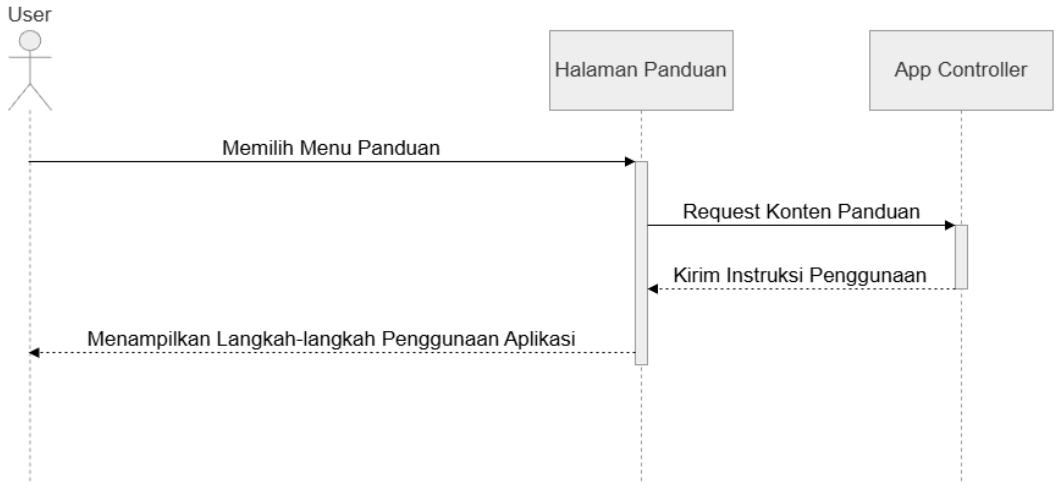


Gambar 21. *Sequence Diagram* Melihat Galeri Motif Batik

Proses dimulai saat pengguna membuka menu galeri motif. Permintaan tersebut diterima oleh Halaman Galeri, yang kemudian mengirimkan permintaan daftar motif ke *Controller*. Selanjutnya, *Controller* mengambil data dari *database* yang berisi daftar tujuh kelas motif batik beserta nama dan thumbnail gambar masing-masing motif.

Setelah data diperoleh, *database* SQLite mengembalikan daftar nama motif dan *thumbnail* kepada *Controller*, yang kemudian meneruskannya ke Halaman Galeri. Halaman galeri selanjutnya menampilkan data tersebut dalam bentuk grid galeri sehingga pengguna dapat melihat dan memilih motif yang tersedia.

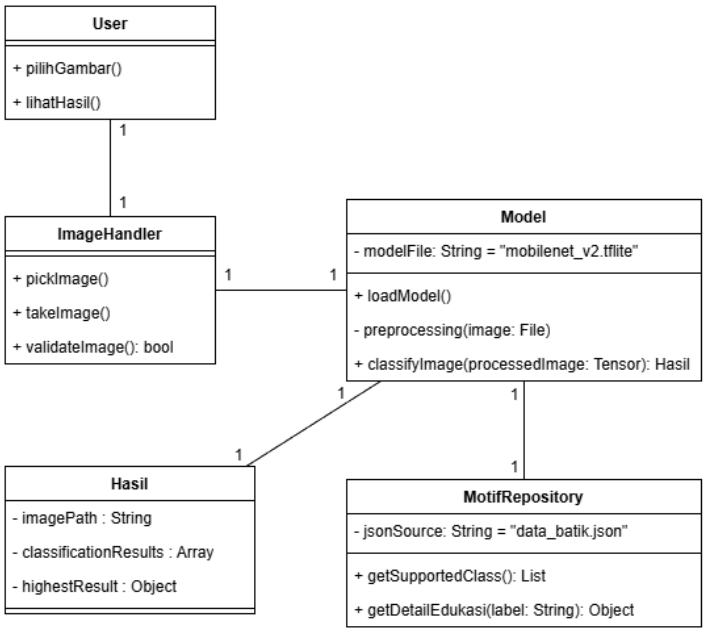
d. *Sequence Diagram* Melihat Panduan Penggunaan



Gambar 22. *Sequence Diagram* Melihat Panduan Penggunaan

Proses dimulai saat pengguna memilih menu panduan pada aplikasi. Permintaan tersebut diterima oleh Halaman Panduan, yang kemudian mengirimkan permintaan konten panduan ke *Controller*. Selanjutnya, *Controller* mengirimkan kembali instruksi penggunaan aplikasi kepada halaman panduan. Setelah konten diterima, Halaman Panduan menampilkan langkah-langkah penggunaan aplikasi kepada pengguna. Proses ini bersifat informatif dan tidak melibatkan model klasifikasi maupun pengolahan data yang kompleks.

4.2.4. *Class Diagram*



Gambar 23. *Class Diagram* Aplikasi Klasifikasi Batik

Diagram ini terdiri atas lima kelas utama, yaitu *User*, *ImageHandler*, *Model*, *Hasil*, dan *MotifRepository*. Kelas *User* merepresentasikan aktor yang berinteraksi dengan sistem. Kelas ini memiliki dua operasi utama, yaitu *pilihGambar()* untuk memilih citra sebagai input dan *lihatHasil()* untuk menampilkan hasil klasifikasi. *User* berasosiasi dengan kelas *ImageHandler* dalam proses pengelolaan gambar. Kelas *ImageHandler* bertanggung jawab dalam menangani *input* citra dari perangkat. Kelas ini menyediakan metode *pickImage()* dan *takeImage()* untuk mengambil gambar dari galeri atau kamera, serta *validateImage()* untuk memvalidasi kelayakan file gambar sebelum diproses lebih lanjut. Setelah valid, gambar diteruskan ke kelas *Model*. Kelas *Model* merupakan komponen inti yang menangani proses klasifikasi menggunakan model CNN MobileNetV2. Kelas ini memiliki atribut modelFile yang menyimpan nama file model dalam format .tflite. Operasi yang tersedia meliputi *loadModel()* untuk memuat model, *preprocessing()* untuk menyesuaikan *input* citra dengan spesifikasi model, serta *classifyImage()* untuk menghasilkan prediksi berupa objek Hasil. Kelas ini berasosiasi dengan kelas *Hasil* sebagai keluaran dari proses klasifikasi.

Kelas *Hasil* berfungsi menyimpan informasi hasil klasifikasi, yang terdiri atas *imagePath*, *classificationResults* dalam bentuk *array* probabilitas, serta *highestResult* sebagai prediksi dengan nilai tertinggi. Objek dari kelas ini kemudian digunakan untuk ditampilkan kepada pengguna. Selain itu, kelas MotifRepository bertanggung jawab dalam pengelolaan data edukasi motif batik yang bersumber dari *database* SQLite. Kelas ini menyediakan metode *getSupportedClass()* untuk mengambil daftar kelas motif yang didukung dan *getDetailEdukasi()* untuk mengambil informasi detail berdasarkan label hasil klasifikasi. Hubungan antar kelas menunjukkan bahwa *User* berinteraksi dengan *ImageHandler*, kemudian gambar diproses oleh *Model*, yang menghasilkan objek Hasil. Selanjutnya, informasi tambahan mengenai motif dapat diperoleh melalui *MotifRepository*. Struktur ini memperlihatkan pemisahan tanggung jawab yang jelas antara pengelolaan input, proses komputasi model, penyimpanan hasil, dan penyediaan data edukasi, sehingga sistem menjadi lebih terorganisasi dan modular.

4.2.5. *Pseudocode* Model *Convolutional Neural Network*

Tahapan selanjutnya dalam desain sistem adalah merancang logika komputasi dalam bentuk *pseudocode*. Algoritma sistem dibagi menjadi dua modul utama, yaitu *preprocessing* untuk menyesuaikan citra input sesuai format MobileNetV2, serta klasifikasi untuk melakukan inferensi dan menentukan label motif batik berdasarkan probabilitas tertinggi. Rincian rancangan logika untuk kedua modul tersebut diuraikan sebagai berikut:

```
PROCEDURE PelatihanModelCNN(dataTrain, dataVal)
    // Tahap 1: Inisialisasi Base Model (MobileNetV2)
    baseModel = Load_MobileNetV2(include_top=False, weights='imagenet')
    FreezeAllLayers(baseModel)

    // Tahap 2: Membangun Custom Classifier Head
    model = Sequential()
    model.Add(baseModel)
    model.Add(GlobalAveragePooling2D())
    model.Add(Dense(256, activation='relu', regularizer='L2'))
    model.Add(Dropout(0.45))
    model.Add(Dense(8, activation='softmax'))
```

```
 // Tahap 3: Pelatihan Awal
    Compile(model, optimizer='Adam', learning_rate=3e-4,
loss='CategoricalCrossentropy')
    callbacks = [EarlyStopping(patience=7), ReduceLROnPlateau()]

    LAKUKAN Pelatihan(model, dataTrain, dataVal, epochs=50, callbacks)

    // Tahap 4: Fine-Tuning
    UnfreezeLayers(baseModel, dari_lapisan=130_hingga_akhir)

    // Turunkan learning rate untuk fine-tuning
    Compile(model, optimizer='Adam', learning_rate=1e-5)
    callbacks_fine = [EarlyStopping(patience=6), ReduceLROnPlateau()]

    LAKUKAN PelatihanLanjutan(model, dataTrain, dataVal, epochs=40,
callbacks_fine)

    RETURN model
END PROCEDURE
```

Listing Program 1. *Pseudocode* Keseluruhan Model

Setelah model CNN berhasil dilatih dan diekspor ke dalam format TensorFlow Lite (.tflite), tahapan rancangan logika selanjutnya diimplementasikan pada sisi aplikasi mobile yang dibangun menggunakan *framework* Flutter. Sebelum citra dari kamera atau galeri dapat dianalisis oleh model, sistem harus melakukan *preprocessing*. Proses ini bertujuan untuk menstandarkan dimensi citra menjadi 224x224 piksel dan mengonversinya menjadi *array byte float32* agar sesuai dengan format masukan yang diwajibkan oleh arsitektur MobileNetV2. Rancangan logika *preprocessing* tersebut dapat dilihat pada Listing Program 2.

```
PROCEDURE PreprocessingCitra(imageFile)
    IF imageFile IS NULL THEN
        RETURN NULL
    END IF

    imageData = ReadAsBytes(imageFile)
    originalImage = DecodeImage(imageData)

    // Sesuaikan dengan input MobileNetV2
    resizedImage = ResizeCropSquare(originalImage, size=224)
```

```
    // Konversi ke format yang diterima TFLite
    inputFloatArray = ConvertToByteListFloat32(resizedImage, 224)

    RETURN inputFloatArray
END PROCEDURE
```

Listing Program 2. *Pseudocode Preprocessing* Citra

Selanjutnya citra masukan yang telah berhasil dikonversi menjadi *array float* kemudian diteruskan ke dalam modul klasifikasi untuk dilakukan proses inferensi. Pada tahap ini, interpreter TensorFlow Lite akan memproses *array* tersebut dan menghasilkan keluaran berupa nilai probabilitas (*confidence score*) untuk kedelapan kelas. Algoritma kemudian dirancang untuk melakukan iterasi pencarian nilai probabilitas tertinggi, yang akan ditetapkan sebagai label prediksi akhir untuk ditampilkan kepada pengguna aplikasi. Rancangan logika klasifikasi ini diuraikan pada Listing Program 3.

```
PROCEDURE KlasifikasiMotif(inputFloatArray)
    // Inisialisasi array output untuk 8 kelas motif
    outputArray = Array(8).fill(0)

    // Jalankan model TFLite
    RunInterpreter(inputFloatArray, outputArray)

    maxScore = -1
    maxIndex = 0

    // Cari nilai probabilitas paling tinggi
    FOR i = 0 TO Length(outputArray) - 1 DO
        IF outputArray[i] > maxScore THEN
            maxScore = outputArray[i]
            maxIndex = i
        END IF
    END FOR

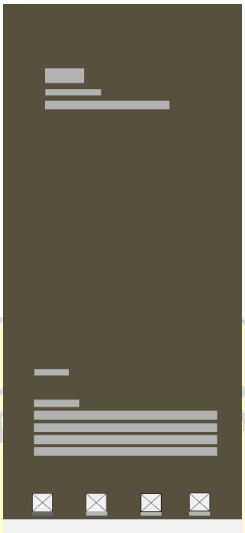
    predictedLabel = labels[maxIndex]

    RETURN predictedLabel, maxScore
END PROCEDURE
```

Listing Program 3. *Pseudocode* Klasifikasi

4.2.6. Rancangan Antarmuka

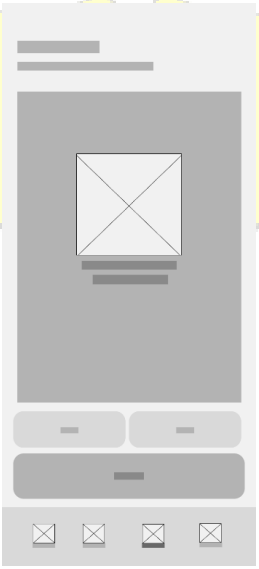
a. Halaman Menu Utama



Gambar 24. Rancangan Antarmuka Halaman Menu Utama

Halaman menu utama merupakan tampilan awal yang pertama kali dilihat oleh pengguna. Pada Gambar 24, rancangan halaman ini akan menampilkan nama aplikasi, deskripsi singkat, serta latar belakang berupa gambar motif batik. Selain itu, terdapat kartu informasi yang berisi penjelasan singkat mengenai batik. Pada bagian bawah disediakan *navigation bar* yang memudahkan pengguna untuk mengakses fitur utama seperti galeri, klasifikasi, dan panduan.

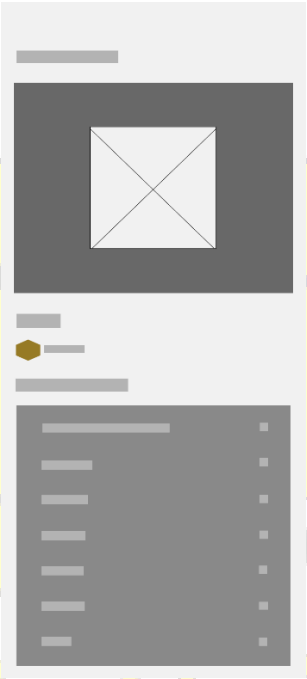
b. Halaman Klasifikasi



Gambar 25. Rancangan Antarmuka Halaman Klasifikasi

Halaman klasifikasi dirancang untuk memfasilitasi pengguna dalam melakukan proses identifikasi motif batik. Pada Gambar 25, terdapat rancangan antarmuka untuk halaman klasifikasi ini terdapat area untuk mengambil gambar melalui kamera atau memilih gambar dari galeri perangkat.

c. Halaman Hasil Klasifikasi



Gambar 26. Rancangan Antarmuka Halaman Hasil Klasifikasi

Gambar 26 menampilkan rancangan antarmuka halaman hasil klasifikasi sebagai keluaran dari proses identifikasi yang telah dilakukan. Pada halaman ini akan ditampilkan gambar yang telah diproses beserta hasil prediksi berupa nama motif batik. Selain itu, disajikan informasi tambahan seperti tingkat keakuratan serta deskripsi singkat untuk membantu pengguna dalam memahami hasil klasifikasi yang diberikan oleh sistem.

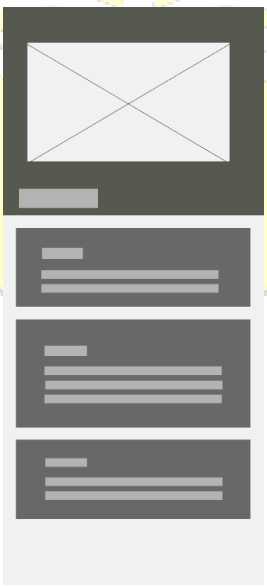
d. Halaman Galeri



Gambar 27. Rancangan Antarmuka Halaman Galeri

Gambar 27 adalah rancangan antarmuka halaman galeri yang berfungsi sebagai media eksplorasi berbagai motif batik yang tersedia dalam aplikasi. Motif ditampilkan dalam bentuk *grid* berupa kartu-kartu yang memuat gambar dan nama motif. Desain ini memudahkan pengguna dalam menelusuri dan memilih motif yang ingin dilihat lebih lanjut.

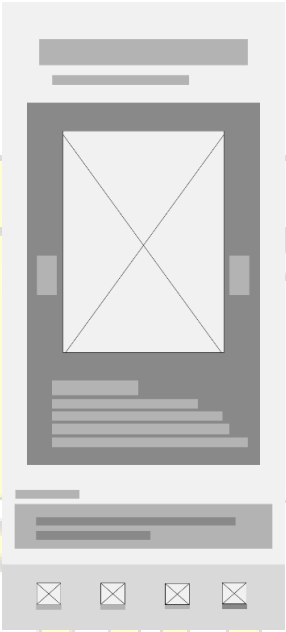
e. Halaman Detail Motif



Gambar 28. Rancangan Antarmuka Halaman Detail Motif

Halaman detail motif menampilkan informasi lengkap mengenai motif batik yang dipilih. Pada bagian atas ditampilkan gambar motif diikuti dengan nama dan deskripsi singkat. Informasi tambahan seperti sejarah, filosofi, dan karakteristik motif disajikan dalam beberapa bagian terpisah agar mudah dibaca dan dipahami.

f. Halaman Panduan Penggunaan



Gambar 29. Rancangan Antarmuka Halaman Panduan Pengguna

Gambar 29 menampilkan rancangan antarmuka halaman panduan penggunaan yang berisi langkah-langkah penggunaan aplikasi. Pada halaman ini terdapat ilustrasi yang disertai penjelasan singkat pada setiap langkah, serta navigasi untuk berpindah antar panduan. Halaman ini dirancang untuk membantu pengguna, terutama pengguna baru, agar dapat memahami cara menggunakan aplikasi dengan mudah.

4.3. Implementasi

Tahap implementasi merupakan realisasi dari rancangan sistem yang telah disusun sebelumnya. Tahap implementasi ini akan dipisahkan menjadi dua tahap utama yaitu implementasi kode model dan implementasi kode aplikasi mobile. Penjelasan mengenai tiap tahapan dipaparkan sebagai berikut.

4.3.1. Pengumpulan Dataset

Pada penelitian ini, dataset citra batik dikumpulkan dari dua sumber utama, yaitu data primer berupa dokumentasi langsung di Museum Batik Pekalongan dan data sekunder yang diperoleh dari Google Arts & Culture serta dataset Batik Utara Jawa. Dataset tersebut mencakup 7 kelas motif batik Pekalongan dan 1 kelas non batik dengan total 616 citra. Distribusi data per kelas meliputi Pagi-Sore sebanyak 73 citra, Jlamprang 72 citra, Jawa Hokokai 79 citra, Tirtateja 53 citra, Terang Bulan 76 citra, Buketan 111 citra, dan Lunglungan 63 citra.

4.3.2. *Preprocessing*

Preprocessing dilakukan dengan 3 tahapan, yaitu *resizing, one-hot encoding*, dan *rescalling*.

a) *Resizing*

Resizing dilakukan untuk menyamaratakan ukuran seluruh gambar, karena dataset citra batik memiliki ukuran yang bervariasi. Perbedaan ukuran ini dapat mempengaruhi akurasi model MobileNetV2. Proses *resizing* ini ditunjukan pada Listing Program 4.

```
IMG_SIZE = (224, 224)
BATCH_SIZE = 32
train_ds = tf.keras.utils.image_dataset_from_directory(
    '/content/drive/MyDrive/Dataset Batik/Split6/train',
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    label_mode='categorical'
)
```

***Listing* Program 4.** *Image Resizing* dan *One-Hot Encoding*

Pada *Listing* Program 4, dataset citra yang tersimpan di Google Drive dimuat menggunakan fungsi *image_dataset_from_directory*. Seluruh gambar secara otomatis diubah ukurannya menjadi 224 x 224 piksel agar sesuai dengan spesifikasi input yang dibutuhkan oleh arsitektur MobileNetV2. Selain itu, parameter *label_mode* diatur ke *categorical* sehingga setiap label akan direpresentasikan dalam bentuk vektor biner sesuai dengan jumlah kelas yang digunakan. Perbandingan dimensi citra sebelum *resizing* ditunjukkan pada Tabel 9.

Tabel 9. Resolusi Citra Sebelum *Resizing*

Nama File Sampel	Lebar (Piksel)	Tinggi (Piksel)
sekunder_buketan_168.jpg	590	640
sekunder_buketan_47.jpg	572	640
sekunder_buketan_78.jpg	1583	1600
…	...	...
primer_IMG_20251229_104622.jpg	4000	3000
primer_IMG_20251229_104632.jpg	4000	3000

Berdasarkan Tabel 9, terlihat bahwa resolusi citra asli memiliki rentang berbeda seperti 572 x 640 piksel hingga resolusi tinggi mencapai 4000 x 3000 piksel. Variasi ukuran ini dipengaruhi oleh perbedaan sumber akuisisi data. Karena ketidakseragaman ini, maka dibutuhkan proses standardisasi agar seluruh citra memiliki format yang kompatibel sebelum masuk ke tahap ekstraksi fitur. Hasil statistik deskriptif setelah *resizing* ditunjukkan pada Tabel 10.

Tabel 10. Hasil Akhir Setelah *Resizing*

Parameter	Nilai Minimum	Nilai Maksimum	Rata-Rata (*Mean*)
Tinggi	224	224	224.00
Lebar	224	224	224.00

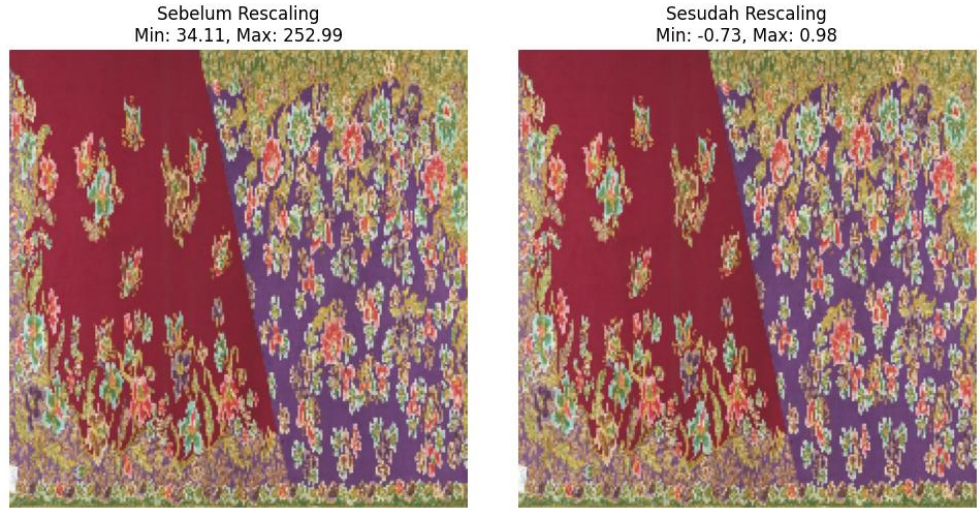
b) *Rescaling*

Rescaling dilakukan untuk menormalkan nilai piksel citra sebelum dimasukkan ke dalam model. Secara umum, nilai piksel gambar digital berada pada rentang 0 hingga 255. Rentang ini dapat menyebabkan proses pelatihan menjadi kurang stabil karena perbedaan skala antar nilai yang terlalu jauh. Oleh karena itu, dilakukan normalisasi agar nilai piksel berada pada rentang yang lebih kecil dan seragam. Tahapan *rescaling*, ditunjukkan pada *Listing* Program 5.

```
rescaling_layer = layers.Rescaling(1./127.5, offset=-1)
model = models.Sequential([
    layers.InputLayer(input_shape=(224, 224, 3)),
    rescaling_layer,
    base_model,
    # ... lapisan selanjutnya
])
```

Listing Program 5. *Rescaling Image*

Proses normalisasi dilakukan menggunakan layer *Rescaling* dengan faktor skala 1./127.5 dan *offset* -1. Operasi ini mengubah rentang nilai piksel dari 0–255 menjadi rentang -1 hingga 1. Layer *rescaling* kemudian ditempatkan di bagian awal model menggunakan arsitektur *Sequential*. Setelah data masuk melalui *InputLayer*, citra terlebih dahulu dinormalisasi oleh *rescaling* layer sebelum diteruskan ke base model dan lapisan-lapisan berikutnya. Hasil dari operasi *rescaling* ini ditunjukkan pada Gambar 24.



Gambar 30. Hasil *Rescaling Image*

Gambar 24 menunjukkan hasil eksekusi dari layer *rescaling* pada salah satu sampel citra. Secara visual, tidak terdapat penurunan kualitas atau perubahan warna antara citra sebelum dan sesudah *rescaling*, yang menandakan fitur citra tetap terjaga utuh. Namun, secara representasi numerik, layer tersebut berhasil menormalisasi rentang nilai matriks piksel. Pada citra asli, nilai piksel berada pada rentang 34.11 hingga 252.99. Setelah melewati proses *rescaling* dengan faktor skala 1/127.5 dan offset -1, nilai matriks piksel berhasil dinormalisasi menjadi rentang -0.73 hingga 0.98.

4.3.3. Pembagian Dataset

Proses *splitting data* dilakukan untuk membagi dataset citra batik ke dalam 3 bagian yaitu data *train*, data *val*, dan data *test*. Pembagian ini bertujuan agar model dapat dilatih, divalidasi, dan diuji secara terpisah untuk menghindari *overfitting*. Setiap gambar dalam masing-masing kelas akan dipisahkan dengan rincian 70%

untuk data *train*, 20% untuk data *validation*, dan 10% untuk data *testing*. Proses *splitting data* ditunjukan pada *Listing* Program 6.

```
split_ratio = (0.7, 0.20, 0.10)
for kelas in os.listdir(src_path):
    random.shuffle(files)

    total = len(files)
    n_train = int(split_ratio[0] * total)
    n_val   = int(split_ratio[1] * total)

    train_files = files[:n_train]
    val_files   = files[n_train:n_train+n_val]
    test_files  = files[n_train+n_val:]

    salin_list(train_files, 'train')
    salin_list(val_files, 'val')
    salin_list(test_files, 'test')
```

***Listing* Program 6.** *Splitting Dataset*

Pada *Listing* Program 6 dilakukan proses pembagian dataset citra batik sesuai dengan *split_rasio*. Setiap file citra dalam masing-masing kelas terlebih dahulu diacak menggunakan *random shuffle* agar distribusi data merata. Selanjutnya, jumlah data pada setiap kelas dihitung dan dibagi sesuai rasio yang telah ditentukan. File yang telah dipisahkan kemudian disalin ke folder *train, val*, dan *test* menggunakan fungsi *salin_list*. Hasil dari splitting data ditunjukkan pada tabel 11.

Tabel 11. Hasil *Splitting Data*

Kelas	*Train*	*Val*	*Test*
Buketan	77	22	12
Jlamprang	50	14	8
Lunglungan	44	12	7
Tirta Teja	37	10	6
Hokokai	55	15	9
Pagi-Sore	51	14	8
Terang Bulan	53	15	8
Non Batik	62	17	10

4.3.4. Penanganan *Imbalance Data*

Penanganan *imbalance* data dilakukan melalui dua pendekatan yang berbeda, yaitu kombinasi *class weight* dan augmentasi data, serta *Synthetic Minority Over-sampling Technique* (SMOTE). Selanjutnya masing-masing

pendekatan akan digunakan pada model CNN MobileNetV2 untuk dibandingkan hasil akurasinya.

a. *Class Weight*

Class weight digunakan untuk mengatasi ketidakseimbangan jumlah data antar kelas, di mana beberapa kelas motif batik memiliki jumlah citra lebih banyak atau lebih sedikit dibandingkan kelas lainnya. Implementasi perhitungan *class weight* ditunjukkan pada *Listing* Program 7.

```
y_train = np.concatenate([y for x, y in train_ds], axis=0)
y_train_label = np.argmax(y_train, axis=1)

unik = np.unique(y_train_label)
bobot = class_weight.compute_class_weight(
  class_weight='balanced',
  classes=unik,
  y=y_train_label
)
bobot_kelas = dict(zip(unik, bobot))
print("Class Weights:", bobot_kelas)
```

***Listing* Program 7.** *Class Weight*

Berdasarkan *Listing* Program 7, proses perhitungan *class weight* diawali dengan menggabungkan seluruh label dari dataset latih menggunakan fungsi *np.concatenate*. Label yang masih dalam bentuk *one-hot encoding* kemudian dikonversi menjadi indeks kelas menggunakan fungsi *np.argmax*, sehingga setiap sampel hanya memiliki satu label kelas. Selanjutnya, fungsi *np.unique* digunakan untuk mengidentifikasi seluruh kelas yang terdapat pada dataset. Setelah kelas-kelas unik diperoleh, fungsi *compute_class_weight* dengan parameter *class_weight='balanced'* digunakan untuk menghitung bobot masing-masing kelas secara otomatis berdasarkan distribusi jumlah data pada setiap kelas. Pada fungsi *compute_class_weight*, perhitungan bobot dilakukan dengan membagi jumlah total sampel dengan hasil perkalian antara jumlah kelas dan jumlah sampel pada kelas ke-i, sehingga kelas dengan jumlah data lebih sedikit akan memperoleh bobot yang lebih besar dibandingkan kelas mayoritas.

Hasil bobot yang diperoleh kemudian dipasangkan dengan label kelas menggunakan fungsi zip, sehingga terbentuk *dictionary* bobot_kelas. Nilai *class weight* yang dihasilkan dapat dilihat pada Tabel 12.

Tabel 12. Hasil Pembobotan Kelas

Nama Kelas	Nilai Bobot
Buketan	0.6580882352941176
Hokokai	1.3984375
Jlamprang	1.2094594594594594
Lunglungan	1.3984375
Pagi-Sore	0.7336065573770492
Terang Bulan	0.69921875
Tirtateja	1.4916666666666667
Non Batik	1.3161764705882353

Berdasarkan Tabel 12, bisa dilihat bahwa kelas dengan jumlah data paling sedikit akan mendapat bobot lebih besar dari kelas dengan jumlah data paling banyak. Contohnya pada kelas Tirtateja yang memiliki total 53 data citra mendapat nilai bobot 1.4917 yang didapatkan dari perhitungan berikut:

$$w_{minoritas} = \frac{616}{8 \times 53} = \frac{616}{424} \approx 1.491$$

Sementara pada kelas Buketan yang memiliki total 111 data citra mendapat nilai bobot sebesar 0.6580 yang didapatkan dari perhitungan berikut:

$$w_{mayoritas} = \frac{616}{8 \times 111} = \frac{616}{888} \approx 0.658$$

b. Augmentasi Data

Proses augmentasi data diterapkan untuk memperkaya variasi citra dalam dataset batik dan mencegah *overfitting* pada model. Metode transformasi yang digunakan yaitu pencerminan (*flip*), perputaran (*rotation*), pergeseran (*translation*), pembesaran/pengecilan (*zoom*), penyesuaian kontras (*contrast*). Implementasi augmentasi ini didefinisikan menggunakan Sequential API dari TensorFlow yang ditunjukkan pada *Listing* Program 8.

```
data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.05),
    layers.RandomZoom(0.1),
```

```
    layers.RandomContrast(0.1)
], name="augmentasi_ringan")
```

***Listing* Program 8.** Augmentasi Data

Pada *Listing* Program 8, *RandomFlip* digunakan untuk mencerminkan citra secara horizontal ke kiri-kanan. Pada *RandomRotation*, citra akan diputar secara acak dengan batas maksimal 5% dari putaran penuh 360 derajat. *RandomZoom* digunakan untuk memperbesar atau memperkecil citra secara acak hingga 10%. Sementara *RandomContrast* mengubah tingkat kontras citra secara acak dengan batas sebesar 10%. Hasil dari augmentasi citra ini ditunjukkan pada Gambar 25.



Gambar 31. Hasil Augmentasi Citra

c. SMOTE (*Synthetic Minority Over-sampling Technique*)

Selain dengan kombinasi *class weight* dan augmentasi, akan dilakukan juga teknik *oversampling* untuk menambah jumlah data pada kelas minoritas menggunakan SMOTE. Implementasi SMOTE dapat dilihat pada *Listing* Program 9.

```
from imblearn.over_sampling import SMOTE

smote = SMOTE(sampling_strategy='auto', random_state=42)
X_smote, y_smote = smote.fit_resample(X_train, y_train)
print("Sebelum SMOTE:", X_train.shape)
print("Sesudah SMOTE:", X_smote.shape)
```

***Listing* Program 9**. *Oversampling* SMOTE

Berdasarkan *Listing* Program 9, proses *oversampling* dimulai dengan membentuk objek SMOTE menggunakan parameter *sampling_strategy='auto'* untuk menyeimbangkan jumlah data antar kelas serta *random_state=42* agar hasil yang diperoleh bersifat konsisten dan dapat diulang kembali. Pada proses SMOTE, untuk setiap sampel pada kelas minoritas akan dicari beberapa tetangga terdekat,

kemudian dipilih salah satu tetangga untuk dihitung selisih nilainya terhadap sampel asli. Selanjutnya, sampel sintetis dibentuk dengan menambahkan sampel asli kelas minoritas dengan nilai acak yang dikalikan terhadap selisih tersebut, sehingga menghasilkan data baru yang berada di antara dua sampel minoritas. Setelah itu, fungsi *fit_resample()* diterapkan pada data latih *X_train* dan label *y_train* sehingga menghasilkan dataset baru *X_smote* dan *y_smote*. Jumlah dataset tersebut dapat dilihat pada Gambar 32.

```
Sebelum SMOTE: (358, 1280)
Sesudah SMOTE: (544, 1280)
```

Gambar 32. Hasil SMOTE

4.3.5. Perancangan Arsitektur Model

Perancangan arsitektur model pada penelitian ini dilakukan melalui dua metode yang berbeda. Metode pertama menggunakan arsitektur CNN berbasis MobileNetV2 dengan penerapan teknik augmentasi dan *class weight,* sementara metode kedua menggunakan CNN berbasis MobileNetV2 dengan dataset yang telah melalui proses *oversampling* menggunakan SMOTE. Rincian kedua metode tersebut dirincikan sebagai berikut.

a. Arsitektur CNN MobileNetV2

Setelah melalui tahapan *preprocessing,* langkah selanjutnya adalah melakukan pemodelan dengan menggunakan arsitektur MobileNetV2. Pemodelan pada penelitian ini menerapkan teknik *transfer learning*, di mana model memanfaatkan bobot yang telah dilatih sebelumnya pada dataset ImageNet. Proses pemodelan beserta penambahan *custom layers* ditunjukkan pada *Listing* Program 10.

```
base_model = tf.keras.applications.MobileNetV2(
    input_shape=(224, 224, 3),
    include_top=False,
    weights='imagenet'
)
base_model.trainable = False
inputs = layers.Input(shape=(224,224,3))

x = data_augmentation(inputs)
x = tf.keras.applications.mobilenet_v2.preprocess_input(x)
x = base_model(x, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.BatchNormalization()(x)
x = layers.Dense(256,
activation='relu',kernel_regularizer=tf.keras.regularizers.l2(5e-5))(x)
x = layers.Dropout(0.45)(x)
outputs = layers.Dense(8, activation='softmax')(x)

model = models.Model(inputs, outputs)
```

***Listing* Program 10.** Perancangan Arsitektur CNN MobileNetV2

Berdasarkan *Listing* Program 10, model MobileNetV2 dipanggil dengan ukuran input 224×224×3, tanpa menggunakan lapisan klasifikasi bawaan *include_top=False*, serta memanfaatkan bobot hasil pelatihan pada dataset ImageNet *weights='imagenet'*. Selanjutnya baris kode *base_model.trainable = False* digunakan untuk membekukan bobot MobileNetV2 agar tidak ikut diperbarui saat proses *training*. Input citra kemudian melewati proses augmentasi data *data_augmentation* dan dinormalisasi menggunakan *preprocess_input*. Hasil ekstraksi fitur dari MobileNetV2 selanjutnya diringkas menggunakan *GlobalAveragePooling2D*, kemudian distabilkan menggunakan *Batch Normalization*.

Pada tahap klasifikasi, ditambahkan layer *Dense* berukuran 256 neuron dengan aktivasi ReLU serta regularisasi L2 untuk mengurangi *overfitting*, kemudian *Dropout* sebesar 0.45. Output akhir menggunakan *Dense* dengan 8 neuron dan aktivasi *softmax* untuk menghasilkan prediksi ke dalam 8 kelas batik. Model kemudian dibangun menggunakan *models.Model(inputs, outputs)*.

b. Arsitektur CNN MobileNetV2 + SMOTE

Pada arsitektur ini, MobileNetV2 tidak digunakan sebagai model klasifikasi secara langsung, melainkan dimanfaatkan sebagai *feature extractor* untuk mengekstraksi fitur dari citra batik. Fitur yang dihasilkan kemudian digunakan sebagai *input* untuk proses *oversampling* menggunakan SMOTE serta pelatihan model klasifikasi menggunakan beberapa lapisan *fully connected*. Implementasi arsitektur ini ditunjukkan pada *Listing* Program 11.

```
base_model = tf.keras.applications.MobileNetV2(
  input_shape=(224, 224, 3),
  include_top=False,
  weights='imagenet',
  pooling='avg'
)
def ekstraksi_fitur(dataset):
  daftar_fitur = []
  daftar_label = []
  for gambar, label in dataset:
    gambar = (gambar / 127.5) - 1
    fitur = base_model.predict(gambar, verbose=0)
    daftar_fitur.append(fitur)
    daftar_label.append(label)
  X = np.concatenate(daftar_fitur)
  y = np.concatenate(daftar_label)

  if len(y.shape) > 1:
    y = np.argmax(y, axis=1)
  return X, y
X_train, y_train = ekstraksi_fitur(train_ds)
X_val, y_val = ekstraksi_fitur(val_ds)

model = models.Sequential([
    layers.Input(shape=(X_smote.shape[1],)),
    layers.Dense(64, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.5),
    layers.Dense(32, activation='relu'),
    layers.Dropout(0.4),
    layers.Dense(jml_kelas, activation='softmax')
])
```

***Listing* Program 11.** Perancangan Arsitektur CNN + SMOTE

Pada *Listing* Program 11, MobileNetV2 digunakan sebagai base model dengan parameter *include_top=False* sehingga hanya berfungsi sebagai ekstraktor fitur, serta menggunakan bobot *pretrained* dari ImageNet. Parameter *pooling='avg'* digunakan untuk menerapkan *global average pooling* secara otomatis sehingga *output* MobileNetV2 berupa vektor fitur 1 dimensi. Hal ini yang memungkinkan *output* dari MobileNetV2 dapat digunakan untuk *input* pada teknik SMOTE.

Setelah itu, dibuat fungsi *ekstraksi_fitur(dataset)* yang bertugas mengambil fitur dari setiap *batch* gambar pada dataset. Di dalam fungsi tersebut, gambar dinormalisasi terlebih dahulu menggunakan rumus $(gambar\ /\ 127.5) - 1$ agar sesuai dengan skala *input* MobileNetV2. Kemudian *base_model.predict()* digunakan untuk menghasilkan fitur dari gambar, lalu fitur dan label disimpan ke dalam list daftar_fitur dan daftar_label. Setelah seluruh data diproses, fitur dan label digabungkan menggunakan *np.concatenate* menjadi variabel X dan y. Apabila label masih berbentuk *one-hot encoding*, maka dilakukan konversi ke bentuk indeks kelas menggunakan *np.argmax*.

Fungsi tersebut kemudian diterapkan pada dataset latih dan validasi sehingga diperoleh X_*train, y_train* serta *X_val, y_val* yang berisi fitur hasil ekstraksi. Setelah fitur diperoleh dan dapat dilakukan SMOTE, dibangun model klasifikasi menggunakan *Sequential* yang terdiri dari input sesuai jumlah fitur , dua layer *Dense* berukuran 64 dan 32 neuron dengan aktivasi ReLU, *BatchNormalization* untuk menstabilkan proses *training*, serta *Dropout* sebesar 0.5 dan 0.4 untuk mengurangi *overfitting*. Lapisan output menggunakan *Dense* dengan aktivasi *softmax* sesuai jumlah kelas untuk menghasilkan prediksi klasifikasi.

Pada tahap ini, arsitektur MobileNetV2 tidak disertakan secara langsung ke dalam model *Sequential* karena proses ekstraksi fitur telah dilakukan sebelumnya secara terpisah. Input yang masuk ke dalam model (*X_train*) bukan lagi berupa piksel gambar mentah, melainkan vektor fitur hasil ekstraksi. Oleh karena itu, model yang dibangun berfokus pada *fully connected layer* sebagai pengklasifikasi akhir.

4.3.6. *Training* Model Awal

Setelah arsitektur model selesai didefinisikan, langkah berikutnya adalah melakukan kompilasi dan pelatihan model awal.

a. *Training* Model CNN dengan Augmentasi

Pada tahap pelatihan model ini, pembaruan bobot hanya dilakukan pada lapisan kustom yang telah ditambahkan, sementara bobot dari *pre-trained* model MobileNetV2 dibiarkan tetap beku agar fitur dasar yang telah dipelajari dari ImageNet tidak rusak. Proses eksekusi pelatihan model awal ditunjukkan pada *Listing* Program 12.

```
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=3e-4),
    loss = tf.keras.losses.CategoricalCrossentropy(label_smoothing=0.05),
    metrics=['accuracy']
)
early_stop = tf.keras.callbacks.EarlyStopping(
    monitor='val_loss',
    patience=7,
    restore_best_weights=True
)
lr_scheduler = tf.keras.callbacks.ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.3,
    patience=3,
    min_lr=1e-6
)
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=50,
    class_weight=bobot_kelas,
    callbacks=[early_stop, lr_scheduler]
)
```

***Listing* Program 12.** Pelatihan Awal Model CNN

Berdasarkan *Listing* Program 12, Model dikompilasi menggunakan *optimizer Adam* dengan *learning rate* sebesar 3×10^{-4}, fungsi *loss CategoricalCrossentropy* dengan *label smoothing* 0.05 untuk mengurangi *overconfidence* model, serta metrik evaluasi berupa akurasi. Selanjutnya digunakan *callback EarlyStopping* yang memantau nilai *val_loss* dengan patience 7 untuk menghentikan pelatihan apabila

performa validasi tidak membaik, serta mengembalikan bobot terbaik. *Callback ReduceLROnPlateau* juga diterapkan untuk menurunkan learning rate secara otomatis sebesar 0.3 apabila *val_loss* tidak mengalami penurunan selama 3 *epoch*, dengan batas minimum *learning rate* 1×10^{-6}. Proses pelatihan dilakukan menggunakan *model.fit* selama maksimal 50 *epoch* dengan data *training train_ds* dan validasi *val_ds*, serta menerapkan *class_weight=bobot_kelas* untuk menangani ketidakseimbangan kelas. Hasil *training* disimpan dalam variabel *history* untuk keperluan evaluasi lebih lanjut. Dari *listing* program tersebut, didapatkan hasil *training* model yang dapat dilihat pada Tabel 13.

Tabel 13. Hasil *Training* Model CNN

Epoch	***Train Acc***	***Train Loss***	***Val Acc***	***Val Loss***	***LR***
41	0.8073	0.7395	0.8351	0.8325	3e-4
42	0.7989	0.7514	0.8351	0.8308	3e-4
43	0.8101	0.7655	0.8247	0.8278	3e-4
44	0.8324	0.7196	0.8144	0.8249	3e-4
45	0.8128	0.7215	0.8144	0.8236	3e-4
46	0.8464	0.7192	0.8351	0.8225	3e-4
47	0.8520	0.6672	0.8454	0.8222	3e-4
48	0.7961	0.7111	0.8454	0.8228	3e-4
49	0.8492	0.6702	0.8351	0.8199	3e-4
50	**0.8324**	**0.6992**	**0.8454**	**0.8123**	3e-4

Berdasarkan Tabel 13, Model masih menunjukkan peningkatan performa, yang ditunjukkan oleh penurunan nilai *loss* secara konsisten pada data validasi, maka pelatihan dilanjutkan hingga model mencapai *plateau*. Ringkasan hasil dari pelatihan lanjutan dapat dilihat pada Tabel 14.

Tabel 14. *Extended Training*

Epoch	***Train Acc***	***Train Loss***	***Val Acc***	***Val Loss***	***LR***
84	**0.8799**	**0.6021**	**0.8557**	**0.7571**	2.7e-5
85	0.9022	0.5630	0.8557	0.7588	2.7e-5
86	0.8771	0.5918	0.8557	0.7582	2.7e-5

Epoch	*Train Acc*	*Train Loss*	*Val Acc*	*Val Loss*	*LR*
87	0.9050	0.5819	0.8557	0.7578	2.7e-5
88	0.9022	**0.5517**	0.8557	0.7585	8.1e-6
89	0.8827	0.5825	0.8557	0.7585	8.1e-6
90	0.9022	0.5756	0.8557	0.7582	8.1e-6
91	**0.9190**	0.5586	0.8557	0.7578	2.43e-6

Berdasarkan tabel 14, model terbaik dipilih pada *epoch* 84 karena menghasilkan *validation loss* terendah sebesar 0.7571 dengan akurasi validasi yang stabil sebesar 85.57%. Meskipun *epoch* 91 memiliki akurasi *training* lebih tinggi, peningkatan tersebut tidak diikuti oleh peningkatan performa pada data validasi sehingga mengindikasikan adanya gejala *overfitting* ringan.

b. *Training* CNN MobileNetV2 + SMOTE

```
model.compile(
  optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
  loss='sparse_categorical_crossentropy',
  metrics=['accuracy']
)
early_stop = tf.keras.callbacks.EarlyStopping(
    monitor='val_loss',
    patience=5,
    restore_best_weights=True
)
lr_scheduler = tf.keras.callbacks.ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.3,
    patience=3,
    min_lr=1e-6
)
history = model.fit(
    X_smote, y_smote,
    validation_data=(X_val, y_val),
    epochs=100,
    batch_size=32,
    callbacks=[early_stop, lr_scheduler],
    verbose=1
)
```

***Listing* Program 13.** Pelatihan Awal Model CNN + SMOTE

Pada *Listing* Program 13, secara umum mengikuti kode pada *Listing* Program 13, namun dengan beberapa penyesuaian parameter untuk mengoptimalkan hasil *oversampling* SMOTE. Perbedaan utama terletak pada penggunaan fungsi *loss sparse_categorical_crossentropy* karena label data kini berbentuk indeks *integer*, serta peningkatan nilai *learning rate awal* menjadi 1×10^{-3}. Selain itu, durasi pelatihan maksimal ditingkatkan menjadi 100 *epoch* dengan pengaturan *patience* pada *EarlyStopping* yang lebih ketat, yaitu sebesar 5 *epoch*. Proses pelatihan kali ini menggunakan dataset hasil penyeimbangan kelas, yakni *X_smote* dan *y_smote*. Hasil *training* model CNN dengan teknik SMOTE ini ditunjukkan pada Tabel 15.

Tabel 15. Hasil *Training* Model CNN + SMOTE

Epoch	***Train Acc***	***Train Loss***	***Val Acc***	***Val Loss***	***Learning Rate***
27	**0.8952**	**0.4629**	**0.8866**	**0.5617**	3e-4
28	0.8842	0.4556	0.8763	0.5575	3e-4
29	0.9136	0.4237	0.8763	0.5580	3e-4
30	0.9007	0.4439	0.8660	0.5443	3e-4
31	0.8934	0.4257	0.8763	0.5333	3e-4
32	0.8989	0.4312	0.8763	0.5445	3e-4
33	0.9044	0.4058	0.8660	0.5495	3e-4
34	0.9228	0.4052	0.8557	0.5506	3e-4
35	0.9228	0.4116	0.8660	0.5413	9e-5
36	0.9154	0.4159	0.8660	0.5320	9e-5
37	0.9044	0.4071	0.8660	0.5290	9e-5
38	0.9026	0.4297	0.8660	**0.5281**	9e-5
39	0.9136	0.4064	0.8660	0.5283	9e-5
40	0.8842	0.4582	0.8763	0.5298	9e-5
41	**0.9430**	**0.3513**	0.8763	0.5319	9e-5
42	0.9062	0.3890	0.8763	0.5319	2.7e-5
43	0.9357	0.3608	0.8763	0.5321	2.7e-5

Dari tabel 16, Model terbaik dipilih pada *epoch* 27 karena menghasilkan akurasi validasi tertinggi sebesar 88.66%, yang menunjukkan kemampuan generalisasi

terbaik terhadap data baru. Meskipun *epoch* 41 memiliki akurasi *training* tertinggi, performa pada data validasi tidak mengalami peningkatan signifikan sehingga tidak dipilih sebagai model terbaik.

4.3.7. *Fine-Tuning*

Setelah tahap pelatihan awal berhasil menstabilkan bobot pada lapisan klasifikasi kustom, langkah optimalisasi selanjutnya adalah melakukan *fine-tuning* agar model dapat beradaptasi lebih spesifik terhadap detail pola pada dataset motif batik. Pada tahap ini, sebagian lapisan atas dari *base model* dibuka sehingga bobotnya dapat diperbarui bersamaan dengan lapisan kustom selama proses pelatihan berlangsung. Tahap *fine-tuning* ini hanya diterapkan pada model CNN MobileNetV2, sedangkan pada SMOTE proses pelatihan dilakukan menggunakan fitur hasil ekstraksi *(embedding)* sehingga *base model* tidak dilatih ulang. Implementasi tahapan ini ditunjukkan pada *Listing* Program 14.

```
base_model.trainable = True

for layer in base_model.layers[:-30]:
    layer.trainable = False

for layer in base_model.layers:
    if isinstance(layer, tf.keras.layers.BatchNormalization):
        layer.trainable = False

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
    loss=tf.keras.losses.CategoricalCrossentropy(label_smoothing=0.1),
    metrics=['accuracy']
)
callbacks_fine = [
    tf.keras.callbacks.EarlyStopping(
        monitor='val_loss',
        patience=6,
        restore_best_weights=True
    ),
    tf.keras.callbacks.ReduceLROnPlateau(
        monitor='val_loss',
        factor=0.3,
        patience=3,
        min_lr=1e-7
    ) ]
```

```
history_fine = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=40,
    class_weight=bobot_kelas,
    callbacks=callbacks_fine
)
```

***Listing* Program 14.** *Fine-tuning*

Pada *Listing* Program 14, proses *fine-tuning* dimulai dengan mengaktifkan seluruh lapisan pada arsitektur dasar MobileNetV2 agar dapat dilatih kembali. Namun, untuk menjaga kemampuan model dalam mengenali fitur dasar seperti tepi dan bentuk sederhana, 130 lapisan pertama tetap dibekukan sehingga hanya lapisan-lapisan akhir yang diperbarui selama pelatihan. Model kemudian dikompilasi ulang menggunakan *optimizer* Adam dengan *learning rate* yang sangat kecil, yaitu 1e-5, agar perubahan pada bobot pralatih tidak terlalu drastis. Tahap *fine-tuning* dijalankan maksimal 20 *epoch*. Selama proses ini, alur data *train* dan penggunaan *class weight* tetap dipertahankan agar kemampuan model dalam mengenali semua kelas motif menjadi lebih seimbang dan akurat. Hasil dari *fine-tuning* ditunjukkan pada Tabel 16.

Tabel 16. Hasil *Fine-tuning*

Epoch	***Train Acc***	***Train Loss***	***Val Acc***	***Val Loss***	***Learning Rate***
1	0.8939	0.8316	0.8351	0.9846	1.0e-05
2	0.9162	0.8225	0.8454	0.9777	1.0e-05
3	0.8966	0.8078	0.8454	0.9792	1.0e-05
4	0.8799	0.8254	0.8454	0.9866	1.0e-05
5	0.8771	0.8481	0.8454	0.9958	1.0e-05
6	0.8994	0.8149	0.8454	0.9962	3.0e-06
7	0.8911	0.8267	0.8454	0.9945	3.0e-06
8	0.8966	0.8130	0.8557	0.9912	3.0e-06

Pada Tabel 17, proses *fine-tuning* dilakukan selama 8 *epoch* dengan penurunan *learning rate* secara bertahap dari 1.0×10^{-5} hingga 3.0×10^{-6}. Performa terbaik dicapai pada *epoch* ke-8, dengan nilai *train accuracy* sebesar

0,8966 dan *validation accuracy* sebesar 0,8557. Stabilitas performa tersebut juga diikuti oleh nilai *loss* yang konsisten, di mana *train loss* tercatat sebesar 0,8130 dan *validation loss* sebesar 0,9912 pada awal hingga mencapai kondisi yang lebih stabil pada akhir pelatihan.

Meskipun proses *fine-tuning* menunjukkan performa yang stabil, namun hasilnya mengalami penurunan pada *validation accuracy* serta peningkatan *validation loss* dibandingkan dengan *base model* sebelumnya. Oleh karena itu, model hasil *fine-tuning* tidak digunakan sebagai model akhir. Sebagai gantinya, model terbaik ditetapkan berdasarkan *base model* yang memiliki performa generalisasi lebih baik. Perbandingan performa kedua model tersebut ditunjukkan pada Tabel 17.

Tabel 17. Perbandingan Performa Model

Tahapan	***Epoch* Terbaik**	***Train Accuracy***	***Train Loss***	***Validation Accuracy***	***Validation Loss***
Base Model	*Epoch* 84	0.8799	0.6021	0.8557	0.7571
Fine-tuning	*Epoch* 2	0.9162	0.8225	0.8454	0.9777

4.3.8. Evaluasi Model

Tahap selanjutnya adalah evaluasi terhadap performa model. Sebelum itu, dilakukan prediksi terlebih dahulu pada data test yang belum pernah dipelajari model sebelumnya. Implementasi prediksi data test ditunjukan pada *Listing* Program 15.

```
test_loss, test_acc = model.evaluate(test_ds)
print(f"Test Accuracy: {test_acc:.4f}")

y_test_true = []
y_test_pred = []

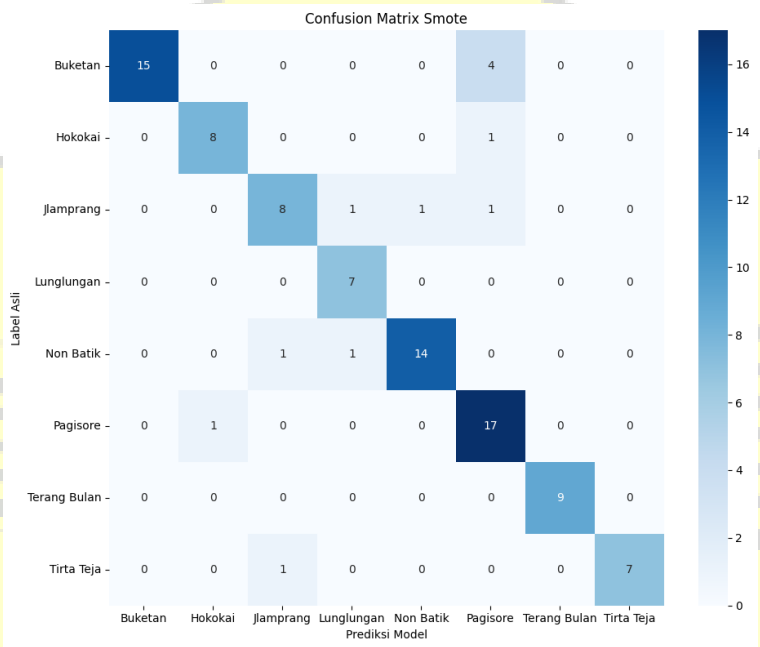
for images, labels in test_ds:
    preds = model.predict(images, verbose=0)
    y_test_true.extend(np.argmax(labels, axis=1))
    y_test_pred.extend(np.argmax(preds, axis=1))

print(classification_report(y_test_true, y_test_pred,
                            target_names=class_names))
```

***Listing* Program 15.** Prediksi Pada *Data Testing*

Berdasarkan *Listing* Program 15, evaluasi diawali dengan mengukur akurasi dan *loss* menggunakan fungsi *model.evaluate* pada *test_ds*. Selanjutnya, program melakukan iterasi untuk mengekstrak label asli (*y_test_true*) dan membandingkannya dengan hasil prediksi model (*y_test_pred*). Data perbandingan tersebut kemudian digunakan untuk menghasilkan laporan klasifikasi serta *confusion matrix*. Hasil pengujian pada data *testing* masing-masing disajikan pada Gambar 33 hingga Gambar 35.

a. Hasil Pengujian Pada Model CNN MobileNetV2 dengan SMOTE



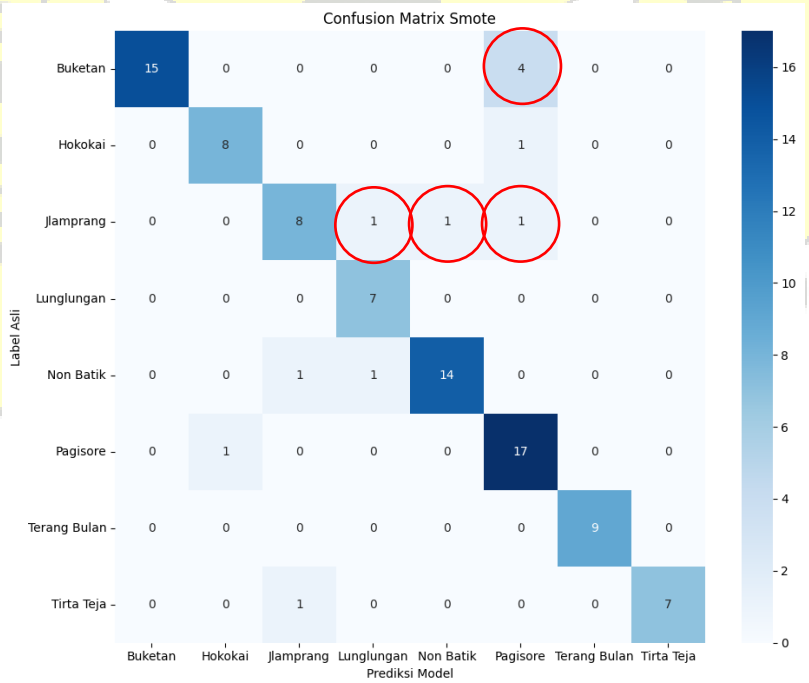
Gambar 33. *Confusion Matrix* CNN + SMOTE

Berdasarkan hasil pengujian menggunakan *confusion matrix*, model MobileNetV2 yang diintegrasikan dengan teknik SMOTE menunjukkan performa yang sangat baik dalam mengklasifikasikan motif batik. Kelas Terang Bulan dan Lunglungan berhasil diidentifikasi dengan akurasi sempurna (100%), yang mengindikasikan bahwa model mampu menangkap fitur unik dari kedua motif tersebut tanpa kesalahan. Penggunaan SMOTE terbukti efektif dalam memberikan kekuatan prediksi pada kelas-kelas dengan sampel terbatas seperti Tirta Teja dan Hokokai, di mana model tetap mampu mengenali pola secara akurat meskipun data aslinya tidak sebanyak kelas mayoritas. Rincian mengenai hasil evaluasi tersebut ditunjukkan pada Tabel 18.

Tabel 18. *Classification Report* CNN + SMOTE

	Precision	***Recall***	***F1-score***	***Support***
Buketan	1.00	0.79	0.88	19
Hokokai	0.89	0.89	0.89	9
Jlamprang	0.80	0.73	0.76	11
Lunglungan	0.78	1.00	0.88	7
Non Batik	0.93	0.90	0.90	16
Pagi-Sore	0.74	0.94	0.83	18
Terang Bulan	1.00	1.00	1.00	9
Tirta Teja	1.00	0.88	0.93	8
Accuracy			0.88	97
Macro avg	0.89	0.89	0.88	97
Weghted avg	0.89	0.88	0.88	97

Meskipun beberapa kelas berhasil diklasifikasi dengan akurasi yang baik, namun masih terdapat beberapa kesalahan yang perlu diperhatikan, terutama pada interaksi antara motif Buketan dan Pagisore.

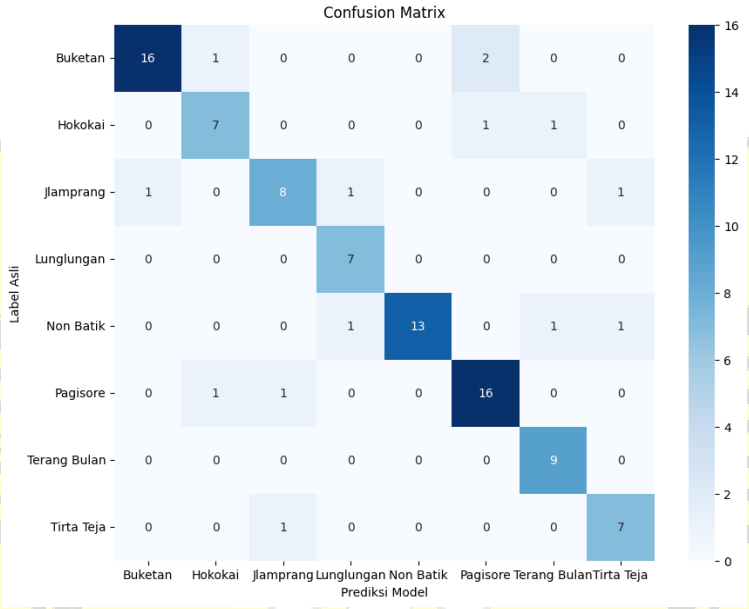


Gambar 34. Kesalahan Klasifikasi Model CNN + SMOTE

Terdeteksi sebanyak 4 sampel motif Buketan yang salah diklasifikasikan sebagai Pagisore, yang kemungkinan besar disebabkan oleh kemiripan fitur visual pada elemen ornamen bunga atau tekstur latar belakang *isen-isen* yang identik di

antara keduanya. Selain itu, motif Jlamprang menunjukkan sedikit penyebaran kesalahan prediksi ke beberapa kelas lain seperti Lunglungan dan Non-Batik, yang menunjukkan kompleksitas pola geometris pada dataset yang terbatas. Secara keseluruhan, matriks ini memvalidasi bahwa model tidak hanya condong pada kelas tertentu, melainkan telah memiliki kemampuan generalisasi yang baik berkat penyeimbangan distribusi data melalui teknik SMOTE.

b. Hasil Pengujian Pada Model CNN MobileNetV2 dengan Augmentasi & *Class Weight*



Gambar 35. *Confusion Matrix* CNN + Augmentasi dan *Class Weight*

Berdasarkan Gambar 35, hasil pengujian pada model MobileNetV2 dengan augmentasi data dan *class weight* menunjukkan performa klasifikasi yang cukup stabil namun dengan distribusi kesalahan yang lebih tersebar dibandingkan saat menggunakan SMOTE. Model ini tetap menunjukkan keunggulan pada kelas Terang Bulan dan Lunglungan dengan tingkat akurasi mencapai 100%, yang menandakan bahwa fitur visual pada kedua motif tersebut sangat mudah dibedakan bagi arsitektur MobileNetV2. Pada kelas Buketan, model mencatatkan jumlah prediksi benar yang sedikit lebih tinggi (16 sampel), namun terdapat degradasi performa pada kelas-kelas lain seperti Hokokai (6 sampel), Pagi-sore (16 sampel) dan Non Batik (13 sampel), di mana muncul kesalahan prediksi baru ke arah kelas

Terang Bulan dan Tirta Teja yang tidak ditemukan pada model sebelumnya. Rincian perbandigan *confusion matrix* kedua model dapat dilihat pada gambar 36.



Gambar 36. Perbandingan *Confussion Matrix*

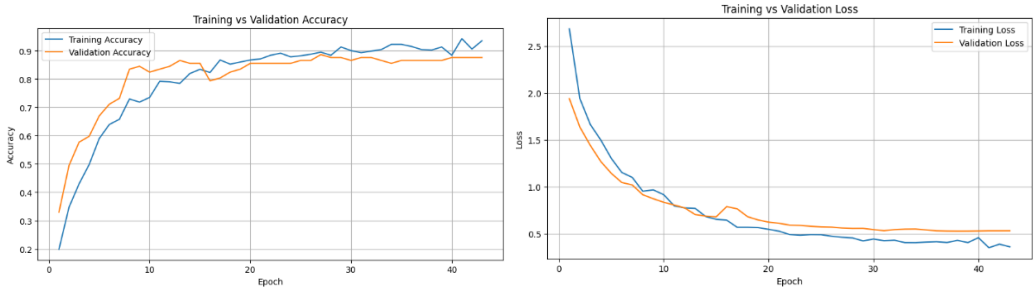
Untuk memberikan gambaran yang lebih jelas mengenai performa model, ditunjukkan *classification report* untuk model CNN dengan augmentasi dan *class weight* yang ditunjukkan pada tabel 19.

Tabel 19. *Classification Report* CNN + Augmentasi dan *Class Weight*

	Precision	***Recall***	***F1-score***	***Support***
Buketan	0.94	0.84	0.89	19
Hokokai	0.78	0.78	0.78	9
Jlamprang	0.80	0.73	0.76	11
Lunglungan	0.78	1.00	0.88	7
Non Batik	1.00	0.81	0.90	16
Pagi-Sore	0.84	0.89	0.86	18
Terang Bulan	0.82	1.00	0.90	9
Tirta Teja	0.78	0.88	0.82	8
Accuracy			0.86	97
Macro avg	0.84	0.87	0.85	97
Weghted avg	0.86	0.86	0.86	97

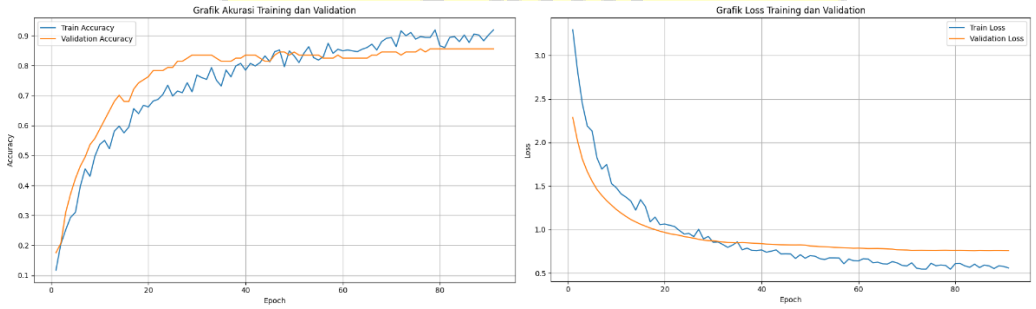
c. Perbandingan Akhir Pada Data Validasi

Sebagai pembanding akhir, akan dibandingkan *training* dan *validation*, persebaran akurasi dan *loss* pada kedua model yang dapat dilihat pada Gambar 37 dan Gambar 38.



Gambar 37. Grafik Akurasi dan *Loss* pada Model CNN + SMOTE

Berdasarkan Gambar 37, model CNN dengan penerapan SMOTE menunjukkan proses pembelajaran yang lebih cepat dan stabil dibandingkan model CNN dengan augmentasi dan *class weight*. Pada model CNN + SMOTE, nilai *training accuracy* meningkat signifikan sejak awal pelatihan, dari 0,1985 pada *epoch* pertama hingga mencapai lebih dari 0,70 pada *epoch* ke-10. Selain itu, *validation accuracy* juga mengalami peningkatan yang konsisten yang dimulai dari 0,3299 hingga mencapai nilai tertinggi sekitar 0,8866 pada *epoch* ke-27. Penurunan nilai *training loss* dan *validation loss* juga terlihat lebih cepat, di mana *validation loss* turun dari 1,9417 pada *epoch* pertama hingga mendekati 0,53 pada *epoch* akhir, menandakan bahwa model mampu melakukan generalisasi dengan cukup baik. Sementara untuk grafik akurasi dan *loss* pada Model CNN dengan augmentasi ditunjukkan pada Gambar 38.



Gambar 38. Grafik Akurasi dan *Loss* pada Model CNN Augmentasi

Berdasarkan Gambar 38, pada model CNN dengan augmentasi, peningkatan performa berlangsung lebih lambat. *Training accuracy* awal berada pada nilai 0,1173 dan meningkat secara bertahap hingga mencapai sekitar 0,83 pada *epoch* ke-50. *Validation accuracy* pada model ini juga meningkat, namun cenderung stabil pada kisaran 0,83–0,85. Penurunan *validation loss* pada model augmentasi memang terjadi secara konsisten, tetapi nilai akhirnya masih lebih tinggi dibandingkan model SMOTE, yaitu sekitar 0,81 pada *epoch* ke-50. Hal ini menunjukkan bahwa model CNN dengan augmentasi membutuhkan *epoch* lebih banyak untuk mencapai performa yang mendekati stabil.

Untuk memberikan gambaran yang lebih jelas mengenai perbedaan hasil pelatihan kedua model tersebut, ringkasan perbandingan metrik antara model CNN berbasis augmentasi dan SMOTE dirangkum dalam Tabel 20.

Tabel 20. Perbandingan Kedua Model

No	*Train Acc* Awal	*Train Loss* Awal	*Val Acc* Awal	*Val Loss* Awal	*Best Epoch*	*Best Val Acc*	*Best Val Loss*	*Best Train Acc*	*Best Train Loss*
1	0,1173	3,2933	0,1753	2,2859	26	0,8557	0,7587	0,8966	0,5842
2	0,1985	2,6864	0,3299	1,9417	27	0,8866	0,5617	0,8952	0,4629

Berdasarkan perbandingan pada Tabel 21, model CNN dengan penerapan SMOTE (Nomor 2) menunjukkan performa yang lebih unggul dibandingkan model CNN berbasis augmentasi (Nomor 1). Secara keseluruhan, dapat disimpulkan bahwa penerapan SMOTE memberikan pengaruh positif terhadap proses pembelajaran model, khususnya dalam meningkatkan *validation accuracy* dan menurunkan *validation loss* secara lebih efektif. Hal ini konsisten dengan temuan oleh (Joloudari dkk., 2023) yang melaporkan bahwa model CNN yang dipadukan dengan SMOTE mampu mengatasi masalah ketidakseimbangan data dan menghasilkan performa klasifikasi yang lebih unggul berdasarkan metrik akurasi pada berbagai dataset yang diuji. Hal ini mengindikasikan bahwa distribusi data yang lebih seimbang mampu membantu model mempelajari pola pada kelas

minoritas dengan lebih baik, sehingga performa validasi menjadi lebih optimal dibandingkan model CNN dengan augmentasi dan *class weight*.

Sementara nilai *loss* yang tinggi pada pengujian Tabel 21 dipengaruhi oleh beberapa faktor, yaitu ketidakseimbangan data, jumlah kelas yang banyak dan juga sedikitnya jumlah data. Kondisi ini sejalan dengan penelitian (Buda dkk., 2018) dalam *A Systematic Study of the Class Imbalance Problem in Convolutional Neural Networks*, yang menunjukkan bahwa ketidakseimbangan data yang tinggi dapat membuat model cenderung bias terhadap kelas mayoritas, sehingga nilai *loss* keseluruhan meningkat meskipun akurasi tampak tinggi karena model lebih banyak benar pada kelas mayoritas.

Selanjutnya, (Fachrie dkk., 2025) juga menjelaskan bahwa jumlah kelas dalam dataset merupakan salah satu faktor yang mempengaruhi performa klasifikasi pada masalah multikelas. Semakin banyak jumlah kelas, terutama dalam kombinasi dengan ketidakseimbangan kelas dan *class overlap*, dapat mengurangi performa (termasuk *recall* dan kinerja klasifikasi) karena kemungkinan overlap antar kelas meningkat dan memperumit tugas prediksi model. Selain itu, penggunaan *class weight* juga turut memengaruhi nilai *loss* karena memberikan penalti lebih besar pada kesalahan prediksi kelas minoritas. Hal ini diperkuat oleh (Razali dkk., 2025) yang menyatakan bahwa teknik seperti *class weights* memang membantu menyeimbangkan kontribusi kelas minoritas dalam fungsi *loss*, namun permasalahan tetap dapat terjadi apabila ketidakseimbangan data tidak ditangani pada tingkat data.

d. Perbandingan Pada Data Test

Berdasarkan hasil pengujian pada data *testing*, dilakukan perbandingan antara dua model yang diusulkan, yaitu CNN dengan data augmentation dan *class weighting*, serta CNN dengan SMOTE. Hasil perbandingan ditunjukkan pada Tabel 21.

Tabel 21. Perbandingan Kedua Model Pada Data *Testing*

Model	***Accuracy***	***Precision***	***Recall***	***F1-Score***
CNN + Augmentasi + *Class Weight*	0.8305	0.8615	0.8339	0.8309
CNN + SMOTE	0.8475	0.8642	0.8311	0.8336

Secara umum, kedua model menunjukkan performa yang seimbang, namun terdapat perbedaan pada beberapa metrik evaluasi. Model CNN dengan SMOTE menghasilkan nilai *accuracy* sebesar 0.8475, sedikit lebih tinggi dibandingkan model CNN dengan augmentasi dan *class weight* yang memperoleh *accuracy* sebesar 0.8305. Hal ini menunjukkan bahwa SMOTE memberikan sedikit peningkatan pada tingkat ketepatan prediksi secara keseluruhan. Rincian performa dapat dilihat pada *classification report* Tabel 22 dan Tabel 23.

Tabel 22. *Classification Report* Model SMOTE Pada Data Test

	Precision	***Recall***	***F1-Score***	***Support***
Buketan	0.80	0.92	0.86	13
Hokokai	1.00	0.60	0.75	5
Jlamprang	1.00	0.83	0.91	6
Lunglungan	0.57	0.80	0.67	5
Non Batik	1.00	0.78	0.88	9
Pagisore	0.91	1.00	0.95	10
Terang Bulan	0.83	0.71	0.77	7
Tirta Teja	0.80	1.00	0.89	4
Accuracy			0.85	59
Macro avg	0.86	0.83	0.83	59
Weghted avg	0.87	0.85	0.85	59

Pada model CNN dengan SMOTE, diperoleh nilai *accuracy* sebesar 0.85 dengan *macro F1-score* sebesar 0.83. Pada model ini, beberapa kelas menunjukkan performa precision yang tinggi seperti Hokokai (1.00), Jlamprang (1.00), dan Non Batik (1.00), namun terdapat penurunan recall pada kelas tertentu seperti Hokokai (0.60) dan Terang Bulan (0.71). Hal ini menunjukkan bahwa meskipun model

cukup akurat dalam prediksi yang benar, masih terdapat beberapa sampel yang tidak berhasil dikenali dengan baik oleh model.

Tabel 23. *Classification Report* Model Augmentasi + *Class Weight* Data Test

	Precision	***Recall***	***F1-Score***	***Support***
Buketan	0.73	0.85	0.79	13
Hokokai	1.00	0.60	0.75	5
Jlamprang	1.00	0.83	0.91	6
Lunglungan	0.62	1.00	0.77	5
Non Batik	1.00	0.78	0.88	9
Pagisore	0.90	0.90	0.90	10
Terang Bulan	0.83	0.71	0.77	7
Tirta Teja	0.80	1.00	0.89	4
Accuracy			0.83	59
Macro avg	0.86	0.83	0.83	59
Weghted avg	0.86	0.83	0.83	59

Sementara pada Tabel 24, memperoleh accuracy sebesar 0.83 dengan *macro F1-score* yang juga sebesar 0.83. Meskipun nilai *accuracy* sedikit lebih rendah dibandingkan SMOTE, model ini menunjukkan performa yang lebih stabil pada beberapa kelas. Contohnya, kelas Lunglungan memiliki *recall* yang lebih tinggi yaitu 1.00, yang menunjukkan bahwa seluruh data pada kelas tersebut berhasil dikenali dengan baik oleh model. Namun, ini juga disebabkan karena jumlah data pada setiap kelas yang relatif kecil sehingga perubahan nilai *recall* dan *precision* antar kelas menjadi sensitif terhadap perubahan jumlah prediksi yang benar maupun salah.

Berdasarkan hasil perbandingan tersebut, model CNN + SMOTE dipilih sebagai model yang lebih unggul secara keseluruhan karena menghasilkan nilai *accuracy* dan *F1-Score* yang lebih tinggi pada data *testing*, meskipun kedua model masih menunjukkan keterbatasan pada kelas dengan jumlah sampel yang terbatas.

4.3.9. Implementasi Aplikasi *Mobile*

Pada tahap ini dilakukan implementasi aplikasi *mobile* berbasis Flutter sebagai media pengenalan dan klasifikasi motif batik. Proses implementasi mencakup pembangunan struktur aplikasi, penyusunan komponen pendukung,

serta integrasi layanan seperti *database* SQLite dan model TensorFlow Lite untuk melakukan proses identifikasi motif batik.

a. Implementasi Model Data

Pada tahap implementasi model data, dilakukan perancangan struktur objek yang digunakan untuk merepresentasikan data pada aplikasi. Model data ini berfungsi sebagai penghubung antara *database* dan antarmuka pengguna, sehingga data dapat dikelola dan ditampilkan secara terstruktur. Adapun model yang diimplementasikan meliputi model data batik, fakta batik, gambar, serta panduan penggunaan aplikasi.

1) Batik Model

```
class BatikModel {
  final int id;
  final String name;
  final String description;
  final String history;
  final String filosofi;
  final String ciri;
  final String gambar;
  BatikModel({
    required this.id,
    required this.name,
    required this.description,
    required this.history,
    required this.filosofi,
    required this.ciri,
    required this.gambar,
  });

  factory BatikModel.fromMap(Map<String, dynamic> map) {
    return BatikModel(
      id: map['id'],
      name: map['name'],
      description: map['description'],
      history: map['history'],
      filosofi: map['filosofi'],
      ciri: map['ciri'],
      gambar: map['gambar'],
    );
  }
}
```

***Listing* Program 16**. *BatikModel*

Listing Program 16 merupakan implementasi *BatikModel* yang digunakan untuk merepresentasikan data batik dalam aplikasi. Model ini berfungsi sebagai struktur objek yang menampung informasi detail batik yang diperoleh dari *database*, dengan atribut utama yaitu id, *name*, *description*, *history*, filosofi, ciri, dan gambar yang masing-masing digunakan untuk menyimpan identitas batik, nama motif, deskripsi, sejarah, filosofi, ciri khas, serta gambar batik. Selain itu, terdapat *method factory BatikModel.fromMap()* yang berfungsi untuk mengonversi data bertipe *Map<String, dynamic>* menjadi objek *BatikModel*.

2) Fakta Model

```
class FaktaBatik {
  final String text;

  FaktaBatik({required this.text});
}

// Data List Fakta
final List<FaktaBatik> faktaBatikList = [
  FaktaBatik(
    text: "Batik diakui UNESCO sebagai Warisan Budaya Tak Benda Dunia pada 2
Oktober 2009.",
  ),
  ...
  FaktaBatik(
    text: "Lilin (malam) pada batik berfungsi sebagai perintang warna agar tidak
masuk ke serat kain tertentu.",
  ),
];
```

***Listing* Program 17.** *FaktaBatik*

Pada *Listing* Program 17 ditunjukkan implementasi *FaktaModel* yang digunakan untuk menyimpan informasi berupa fakta-fakta singkat mengenai batik dalam aplikasi yang akan ditampilkan pada halaman *Home*. Model ini direpresentasikan melalui kelas *FaktaBatik* yang memiliki atribut text untuk menampung isi fakta dalam bentuk teks. Selain itu, pada *listing* ini juga dibuat sebuah list bernama *faktaBatikList* yang berisi kumpulan objek *FaktaBatik*.

3) Hasil Prediksi Model

```
class HasilPrediksiModel {
  final String label;
  final double confidence;
  final String description;
  final Map<String, double> allProbabilities;

  HasilPrediksiModel({
    required this.label,
    required this.confidence,
    required this.description,
    required this.allProbabilities,
  });

  factory HasilPrediksiModel.fromMap(Map<String, dynamic> map) {
    return HasilPrediksiModel(
      label: map['label'],
      confidence: map['confidence'],
      description: map['description'],
      allProbabilities: Map<String, double>.from(map['allProbabilities']),
    );
  }
}
```

***Listing* Program 18**. *HasilPrediksiModel*

Listing Program 18 menunjukkan definisi model data *HasilPrediksiModel* yang digunakan untuk menyimpan hasil prediksi dari proses klasifikasi. Kelas ini memiliki atribut label untuk menyimpan nama kelas hasil prediksi, *confidence* untuk menyimpan tingkat keyakinan model dalam bentuk nilai probabilitas, description untuk menyimpan deskripsi terkait kelas batik yang diprediksi, serta *allProbabilities* yang berisi daftar probabilitas dari seluruh kelas dalam bentuk pasangan nama kelas dan nilainya. Selain itu, disediakan konstruktor dengan parameter *required* untuk memastikan semua data terisi. Kelas ini juga memiliki metode *fromMap* yang berfungsi mengonversi data dari bentuk *Map<String, dynamic>* menjadi objek *HasilPrediksiModel.*

4) *Image* Model

```
class ImageModel {
  final int idImage;
  final int batikId;
  final String imagePath;
```

```
  ImageModel({
    required this.idImage,
    required this.batikId,
    required this.imagePath,
  });

  factory ImageModel.fromMap(Map<String, dynamic> map) {
    return ImageModel(
      idImage: map['id_image'],
      batikId: map['batik_id'],
      imagePath: map['image_path'],
    );
  }
}
```

***Listing* Program 19.** *ImageModel*

Pada *Listing Program* 19, ditunjukkan pembuatan kelas *ImageModel* yang digunakan untuk menampung data gambar batik yang tersimpan di dalam *database*. Kelas ini memiliki beberapa atribut yaitu *idImage* sebagai identitas unik gambar, *batikId* sebagai penanda relasi antara gambar dengan data batik tertentu, serta *imagePath* yang berisi lokasi atau alamat file gambar yang akan ditampilkan pada aplikasi. Selain itu, terdapat method *fromMap()* yang berfungsi untuk mengubah data hasil *query database* berbentuk *Map<String, dynamic>* menjadi objek *ImageModel*.

5) Panduan Model

```
class PanduanModel {
  final String title;
  final String description;
  final String image;
  PanduanModel({
    required this.title,
    required this.description,
    required this.image,
  });
}
final List<PanduanModel> panduanList = [
  PanduanModel(
    title: 'Cahaya Terang & Merata',
    description: 'Pastikan kain batik ...',
```

```
    image: 'assets/images/panduan1.png',),
PanduanModel(
    title: 'Posisi Tegak Lurus',
    description: 'Ambil foto secara tegak lurus dari atas...',
    image: 'assets/images/panduan2.png',
  ),
  PanduanModel(
    title: 'Fokus pada Satu Motif',
    description: 'Pastikan hanya satu motif batik ...',
    image: 'assets/images/panduan3.png',
  ),
];
```

***Listing* Program 20.** *PanduanModel*

Pada *Listing* Program 20, ditunjukkan implementasi *PanduanModel* yang digunakan untuk merepresentasikan data panduan penggunaan aplikasi dalam bentuk informasi terstruktur. Model ini dibuat dalam kelas *PanduanModel* yang memiliki atribut *title*, *description*, dan *image*, dimana masing-masing digunakan untuk menyimpan judul panduan, deskripsi atau penjelasan langkah, serta gambar pendukung panduan. Selain itu, pada *listing* ini juga terdapat list *panduanList* yang berisi kumpulan objek *PanduanModel* sebagai data statis yang akan ditampilkan pada halaman panduan aplikasi

b. Implementasi Kode *Services*

Pada bagian ini dilakukan implementasi *service* yang berfungsi sebagai penghubung antara aplikasi dengan komponen pendukung sistem, seperti *database* dan model klasifikasi. Implementasi *service* pada aplikasi ini mencakup *DatabaseService* untuk mengakses data batik yang tersimpan pada SQLite, serta *TfliteService* untuk melakukan proses klasifikasi menggunakan model TensorFlow Lite dalam menentukan motif batik.

1) *DatabaseService*

```
class DatabaseInstance {
  final String _databaseName = "mbatik.db";
  Database? _database;
  Future<Database> get database async {
    if (_database != null) return _database!;
    _database = await _initDatabase();
    return _database!;}
```

***Listing* Program 21.** Deklarasi *class*

Listing Program 21 menunjukkan deklarasi *class DatabaseInstance* sebagai *service* untuk mengelola koneksi SQLite pada aplikasi. *Class* ini menyimpan nama *database mbatik.db* melalui variabel *_databaseName* serta *instance database* pada variabel *_database* agar tidak dilakukan pembukaan berulang. Melalui *getter database*, aplikasi akan memanggil fungsi *_initDatabase()* jika *database* belum diinisialisasi. Proses inisialisasi *database* tersebut ditunjukkan pada *Listing* Program 22.

```
Future<Database> _initDatabase() async {
  Directory documentsDirectory = await getApplicationDocumentsDirectory();
  String path = join(documentsDirectory.path, _databaseName);
  bool db = await File(path).exists();
  if (!db) {
    ByteData data = await rootBundle.load(join("assets", _databaseName));
    List<int> bytes = data.buffer.asUint8List(
      data.offsetInBytes, data.lengthInBytes,
    );
    await File(path).writeAsBytes(bytes, flush: true);
  }
  return openDatabase(path);
}
```

***Listing* Program 22.** Inisialisasi *Database*

Pada *Listing* Program 22 menunjukkan fungsi *_initDatabase()* yang digunakan untuk menginisialisasi *database* SQLite. Fungsi ini mengambil direktori penyimpanan lokal, membentuk path *database* berdasarkan *_databaseName*, lalu mengecek keberadaan file *database* menggunakan *File(path).exists()*. Jika *database* belum tersedia, file *mbatik.db* akan disalin dari folder *assets* ke penyimpanan lokal menggunakan *rootBundle.load()* dan *writeAsBytes()*. Setelah file tersedia, *database* dibuka menggunakan *openDatabase(path)* agar dapat digunakan untuk proses *query* dan pengambilan data. Implementasi pengambilan data batik secara keseluruhan ditunjukkan pada *Listing* Program 23.

```
Future<List<BatikModel>> getAllBatik() async {
  final db = await database;
  final result = await db.query(tableBatik);
  return result.map((e) => BatikModel.fromMap(e)).toList();
}
```

***Listing* Program 23.** *GetAllBatik*

Pada *Listing* Program 23 Fungsi *getAllBatik()* bekerja dengan memanggil *instance database* SQLite untuk melakukan *query* pada *tableBatik*. Hasil pencarian yang berupa *Map* kemudian dikonversi menjadi objek *BatikModel* melalui metode *fromMap()*. Proses ini memastikan data yang diambil dari *database* dapat dikelola secara terstruktur untuk ditampilkan pada antarmuka daftar batik di aplikasi Mbatic.

2) *TFLiteService*

```
class TfliteService {
  Interpreter? _interpreter;
  bool _isModelLoaded = false;

  final List<String> _labels = [
    "Buketan", "Hokokai", "Jlamprang",
    "Lunglungan", "Pagisore",
    "Terang Bulan", "Tirta Teja",
  ];
```

***Listing* Program 24.** Deklarasi *Class TfLiteServices*

Pada *Listing* Program 24 ditunjukkan deklarasi *class TfliteService* yang digunakan sebagai *service* untuk menjalankan proses klasifikasi motif batik menggunakan model TensorFlow Lite pada aplikasi. *Class* ini memiliki variabel *_interpreter* sebagai objek eksekusi model, *_isModelLoaded* sebagai penanda penanda apakah model sudah berhasil dimuat, serta list *_labels* yang berisi nama kelas batik sebagai output prediksi. Label tersebut digunakan untuk menentukan hasil klasifikasi berdasarkan nilai *confidence* tertinggi. Setelah deklarasi *class TfliteService* dibuat, tahap berikutnya adalah memuat model TensorFlow Lite agar dapat digunakan untuk proses klasifikasi. Rincian kode tersebut dapat dilihat pada *Listing* Program 25

```
Future<void> initModel() async {
    try {
      final assetData = await rootBundle.load('assets/batik.tflite');
      final buffer = assetData.buffer.asUint8List();
      _interpreter = Interpreter.fromBuffer(buffer);
      _isModelLoaded = true;
      print("Model Berhasil");
    } catch (e) {
      _isModelLoaded = false;
      print("Gagal memuat model: $e");
    }}
```

***Listing* Program 25.** Inisiasi Model TfLite

Fungsi tersebut memuat file *batik.tflite* dari folder *assets* menggunakan *rootBundle.load()*, mengubahnya menjadi *buffer (Uint8List)*, lalu membuat interpreter melalui *Interpreter.fromBuffer(buffer)*. Setelah model berhasil dimuat, variabel *_isModelLoaded* diubah menjadi *true* sebagai penanda bahwa model siap digunakan. Setelah model berhasil dimuat melalui fungsi *initModel()*, tahap berikutnya adalah melakukan proses klasifikasi gambar menggunakan fungsi *predict()* sebagaimana ditunjukkan pada *Listing* Program 26.

```
Future<Map<String, dynamic>?> predict(File imageFile) async {
  if (!_isModelLoaded || _interpreter == null) return null;

  final imageData = await imageFile.readAsBytes();
  img.Image? originalImage = img.decodeImage(imageData);
  if (originalImage == null) return null;

  img.Image resizedImage = img.copyResizeCropSquare(
    originalImage,
    size: 224,
  );

  var input = _imageToByteListFloat32(resizedImage, 224);
  var output = List<double>.filled(8, 0).reshape([1, 8]);
  _interpreter!.run(input, output);
  List<double> scores = List<double>.from(output[0
  int maxIndex = 0;
  double maxScore = scores[0];

  for (int i = 1; i < scores.length; i++) {
    if (scores[i] > maxScore) {
      maxScore = scores[i];
      maxIndex = i;
    } }

  return {
    "label": _labels[maxIndex],
    "confidence": maxScore,
  };
}
```

***Listing* Program 26.** Fungsi Prediksi

Listing Program 26 menunjukkan bahwa fungsi prediksi terlebih dahulu memastikan model sudah siap melalui pengecekan *_isModelLoaded* dan *_interpreter*. Selanjutnya, gambar input didekode menggunakan *img.decodeImage()*, kemudian diubah ukurannya menjadi 224×224 piksel agar sesuai dengan *input* model. Gambar tersebut dikonversi menjadi data numerik bertipe *float* melalui *_imageToByteListFloat32()*, lalu diproses menggunakan *_interpreter.run(input, output)*. Hasil *output* berupa skor probabilitas setiap kelas digunakan untuk menentukan label dengan *confidence* tertinggi, sehingga menghasilkan prediksi motif batik beserta nilai *confidence*. Proses output tersebut ditunjukkan pada *Listing* Program 27.

```
var output = List<double>.filled(8, 0).reshape([1, 8]);
_interpreter!.run(input, output);

List<double> scores = List<double>.from(output[0]);
int maxIndex = 0;
double maxScore = -1;
for (int i = 0; i < scores.length; i++) {
  if (scores[i] > maxScore) {
    maxScore = scores[i];
    maxIndex = i;
  }
}
```

***Listing* Program 27.** Proses *Output*

Listing Program 27 menunjukkan proses pengolahan *output* hasil klasifikasi model TensorFlow Lite. *Output* disiapkan dalam bentuk list berukuran 8 kelas dengan dimensi [1,8], kemudian diproses menggunakan *_interpreter.run(input, output)*. Nilai prediksi yang dihasilkan disimpan dalam *list scores*, lalu dilakukan pencarian nilai tertinggi untuk menentukan indeks kelas dengan *confidence* terbesar. Hasil ini digunakan sebagai label akhir klasifikasi motif batik pada aplikasi.

c. Implementasi Antarmuka Pengguna (*User Interface*)

Pada tahap ini dilakukan implementasi antarmuka pengguna yang berfungsi sebagai tampilan utama aplikasi serta media interaksi antara pengguna dengan sistem. Implementasi antarmuka ini mencakup beberapa halaman, yaitu halaman *Home*, halaman Galeri, halaman Klasifikasi, dan halaman Panduan Penggunaan.

1) Halaman *Home*

```
void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      initialRoute: '/',
      routes: {
        '/': (context) => const HomeView(),
        '/galeri': (context) => const GaleriView(),
        '/klasifikasi': (context) => const KlasifikasiView(),
        '/panduan': (context) => const PanduanView(),
      },
    );
  }
}
```

***Listing* Program 28.** *Routing* dan Inisiasi Aplikasi pada *main.dart*

Listing Program 28 menunjukkan file *main.dart* yang berfungsi sebagai titik awal aplikasi Flutter. Pada fungsi *main()*, aplikasi dijalankan menggunakan *runApp()* dengan memanggil *widget* utama *MyApp*. *Class MyApp* merupakan *StatelessWidget* yang membangun struktur aplikasi melalui *MaterialApp*. Parameter *debugShowCheckedModeBanner: false* digunakan untuk menghilangkan label *debug* pada tampilan aplikasi. Selanjutnya, *initialRoute: '/'* menetapkan halaman awal aplikasi yaitu *HomeView*. Bagian *routes* digunakan untuk mendefinisikan navigasi antar halaman sehingga aplikasi dapat berpindah halaman menggunakan sistem *routing* berdasarkan nama *route* yang telah ditentukan. Setelah proses *routing* pada *main.dart* didefinisikan, halaman utama aplikasi diatur melalui file *home_view.dart*. Implementasi halaman utama tersebut ditunjukkan pada *Listing* Program 29.

```
class HomeView extends StatefulWidget {
  const HomeView({super.key});
  @override
  State<HomeView> createState() => _HomeViewState();
}

class _HomeViewState extends State<HomeView> {
  int _selectedIndex = 0;
  List<Widget> _buildPages() {
    return const [
      HomeContent(),
      GaleriView(),
      KlasifikasiView(),
      PanduanView(),
    ];
  }

  void _onItemTapped(int index) {
    setState(() => _selectedIndex = index);
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: IndexedStack(
        index: _selectedIndex,
        children: _buildPages(),
      ),
      bottomNavigationBar: CustomNavBar(
        selectedIndex: _selectedIndex,
        onItemTapped: _onItemTapped,
      ),
    );
  }
}
```

***Listing* Program 29.** Implementasi Navigasi Menu pada *home_view.dart*

Listing Program 29 menunjukkan *class HomeView* yang menggunakan *StatefulWidget* untuk mengelola perpindahan halaman melalui menu navigasi bawah. Variabel *_selectedIndex* digunakan untuk menyimpan indeks halaman yang sedang aktif, sedangkan fungsi *_buildPages()* berisi daftar halaman yang dapat ditampilkan. Tampilan halaman diatur menggunakan *IndexedStack* agar *state* setiap halaman tetap tersimpan saat berpindah menu. Selanjutnya, *widget CustomNavBar*

digunakan sebagai *bottom navigation bar* untuk mengubah halaman berdasarkan pilihan pengguna.

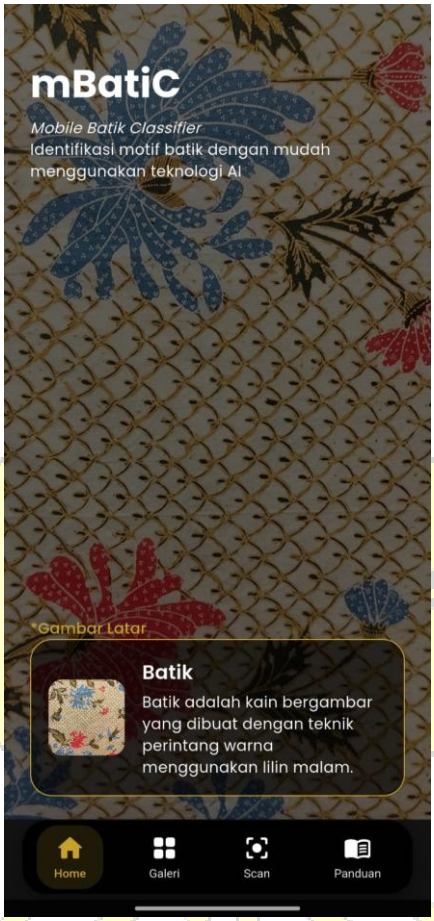
```
class HomeContent extends StatefulWidget {
  const HomeContent({super.key});
  @override
  State<HomeContent> createState() => _HomeContentState();
}

class _HomeContentState extends State<HomeContent> {
  final db = DatabaseInstance();
  List<BatikModel> daftarBatik = [];
  bool isLoading = true;
  @override
  void initState() {
    super.initState();
    _loadData();
  }

  void _loadData() async {
    final data = await db.getAllBatik();
    if (mounted) {
      setState(() {
        daftarBatik = data;
        isLoading = false;
      });
    }
  }
}
```

***Listing* Program 30.** Menampilkan Daftar Batik dan Navigasi

Listing Program 30 menunjukkan *class HomeContent* yang berfungsi untuk mengambil dan menampilkan data batik dari *database*. Objek *DatabaseInstance* digunakan untuk mengakses *database*, sedangkan variabel *daftarBatik* digunakan untuk menyimpan hasil *query*. Pada fungsi *initState()*, aplikasi memanggil *_loadData()* untuk mengambil data batik menggunakan *getAllBatik()*. Setelah data berhasil diperoleh, *state* diperbarui melalui *setState()* dan variabel *isLoading* diubah menjadi false sebagai penanda bahwa proses pemuatan data telah selesai. Hasil implementasi halaman utama yang menampilkan daftar batik tersebut dapat dilihat pada Gambar 39.



Gambar 39. Tampilan Halaman Utama

Pada halaman ini berisi judul dan deskripsi singkat aplikasi, pengenalan batik Pekalongan, contoh motif batik, tombol menuju halaman galeri, *fun fact*, serta informasi Museum Batik sebagai sumber data. Selain itu, halaman ini juga dilengkapi navigation bar di bagian bawah yang terdiri dari menu *Home*, Galeri, *Scan*, dan Panduan.

2) Halaman Galeri

```
class GaleriView extends StatefulWidget {
  const GaleriView({super.key});
  @override
  State<GaleriView> createState() => _GaleriViewState();
}

class _GaleriViewState extends State<GaleriView> {
  final db = DatabaseInstance();
  List<BatikModel> daftarMotif = [];
  bool isLoading = true;
```

***Listing* Program 31.** Deklarasi *Class* Galeri

Listing Program 31 menunjukkan deklarasi *class GaleriView* yang digunakan untuk menampilkan halaman galeri motif batik pada aplikasi. *Class* ini menggunakan *StatefulWidget* karena halaman membutuhkan perubahan *state* saat data dimuat dari *database*. Pada bagian *state*, objek *DatabaseInstance* digunakan untuk mengakses *database* SQLite, sedangkan variabel *daftarMotif* berfungsi untuk menyimpan daftar motif batik yang akan ditampilkan. Variabel *isLoading* digunakan sebagai indikator proses *load* data. Setelah *class* galeri dideklarasikan dilakukan inisiasi untuk load data motif dari *database*. Proses inisialisasi dan pengambilan data tersebut ditunjukkan pada *Listing* Program 32.

```
@override
void initState() {
  super.initState();
  loadData();
}

void loadData() async {
  final data = await db.getAllBatik();
  setState(() {
    daftarMotif = data;
    isLoading = false;
  });
}
```

***Listing* Program 32.** Inisiasi dan Load Data

Pada fungsi *initState()*, metode *loadData()* dipanggil setelah *super.initState()* agar proses pengambilan data dilakukan sejak awal *widget* dibuat. Fungsi *loadData()* bersifat *asynchronous (async)* dan mengambil seluruh data batik dari *database* menggunakan *db.getAllBatik()*. Setelah data berhasil diperoleh, fungsi *setState()* digunakan untuk memperbarui variabel *daftarMotif* dengan data yang didapat serta mengubah status *isLoading* menjadi *false*, sehingga tampilan dapat diperbarui dan data dapat ditampilkan pada halaman. Implementasi kode untuk menampilkan data tersebut dapat dilihat pada *Listing* Program 33.

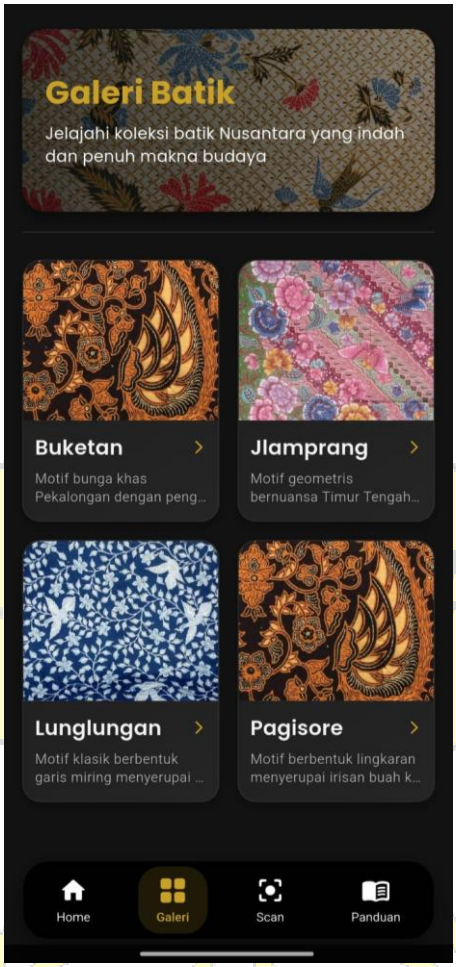
```
GridView.builder(
  itemCount: daftarMotif.length,
  gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(
    crossAxisCount: 2,
  ),
```

```
  itemBuilder: (context, index) {
    final motif = daftarMotif[index];

    return InkWell(
      onTap: () async {
        final gambarList = await db.getGambarByBatikId(motif.id);
        Navigator.push(
          context,
          MaterialPageRoute(
            builder: (_) => DetailGaleri(motif: motif, gambarList: gambarList),
          ),
        );
      },
      child: Text(motif.name),
    );
  },
)
```

***Listing* Program 33.** Menampilkan Data Galeri

Selanjutnya, *Listing* Program 33 menunjukkan proses menampilkan data motif batik menggunakan *GridView.builder*. Data yang ditampilkan berasal dari list *daftarMotif* dengan jumlah item sesuai itemCount. Tampilan dibuat dalam bentuk grid dua kolom melalui *SliverGridDelegateWithFixedCrossAxisCount*. Setiap item ditampilkan menggunakan *InkWell* agar dapat ditekan. Ketika salah satu motif dipilih, aplikasi akan mengambil daftar gambar terkait menggunakan *db.getGambarByBatikId(motif.id)* dan menavigasikan pengguna ke halaman *DetailGaleri* untuk menampilkan detail motif beserta gambar-gambarnya. Sehingga tampilan akhir untuk halaman galeri ditunjukkan pada gambar 38.



Gambar 40. Tampilan Halaman Galeri

Gambar 37 menunjukkan tampilan Halaman Galeri yang berisi judul halaman dan deskripsi singkat, serta daftar motif batik yang ditampilkan dalam bentuk *grid card*. Setiap *card* menampilkan gambar motif, nama motif, dan deskripsi singkat, serta dapat diklik untuk menuju halaman detail motif.

3) Halaman Detail Galeri

```
class DetailGaleri extends StatelessWidget {
  final BatikModel motif;
  final List<String> gambarList;

  const DetailGaleri({
    super.key,
    required this.motif,
    required this.gambarList,
  });
```

***Listing* Program 34.** Deklarasi *Class* Detail Galeri

Listing Program 34 menunjukkan deklarasi *class DetailGaleri* yang digunakan untuk menampilkan halaman detail motif batik. *Class* ini menerima parameter motif bertipe *BatikModel* sebagai data utama motif yang dipilih, serta *gambarList* berupa daftar gambar pendukung yang akan ditampilkan pada halaman detail. Selanjutnya, proses penampilan informasi detail batik ditunjukkan pada *Listing* Program 34.

```
Column(
  children: [
    Text(motif.description),
    Text(motif.history),
    Text(motif.filosofi),
    Column(
      children: motif.ciriList.map((ciri) => Text(ciri)).toList(),
    ),
  ],
)
```

***Listing* Program 35.** Menampilkan Data Detail Batik

Listing Program 35 menunjukkan proses penampilan informasi detail motif batik menggunakan *widget Column*. Data yang ditampilkan meliputi deskripsi, sejarah, filosofi, serta daftar ciri khas (*ciriList*) yang ditampilkan secara dinamis menggunakan *map()*. Selanjutnya, penampilan kumpulan gambar motif batik ditunjukkan pada *Listing* Program 36.

```
GridView.builder(
  shrinkWrap: true,
  itemCount: gambarList.length,
  gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(
    crossAxisCount: 3,
  ),
  itemBuilder: (context, index) {
    return GestureDetector(
      onTap: () {
        Navigator.push(
          context,
          MaterialPageRoute(
            builder: (_) => FullscreenGallery(
              images: gambarList,
              initialIndex: index,
            ),
          ),
```

```
);
      },
      child: Image.asset(gambarList[index]),
    );
  },
)
```

***Listing* Program 36.** Menampilkan Galeri Gambar

Listing Program 36 menampilkan galeri gambar menggunakan *GridView.builder* dengan *layout* 3 kolom. Setiap gambar ditampilkan melalui Image.asset dan dibungkus *GestureDetector* agar dapat ditekan. Ketika gambar dipilih, aplikasi akan membuka halaman *FullscreenGallery* untuk menampilkan gambar dalam mode layar penuh sesuai indeks yang dipilih. Hasil implementasi halaman detail galeri tersebut dapat dilihat pada Gambar 41.



Gambar 41. Tampilan Halaman Detail Galeri

Gambar 41 menunjukkan tampilan Halaman Detail Motif Batik yang berisi gambar motif, nama motif, dan deskripsi singkat. Selain itu, halaman ini menampilkan informasi tambahan seperti asal daerah, filosofi, dan ciri khas motif, serta beberapa gambar contoh motif terkait di bagian bawah.

4) Halaman Klasifikasi

```
@override
void initState() {
  super.initState();
  _tfliteService.initModel();
}

Future<void> _pickImage(ImageSource source) async {
  final pickedFile = await _picker.pickImage(source: source, imageQuality: 85);

  if (pickedFile != null) {
    setState(() {
      _image = File(pickedFile.path);
    });
  }
}
```

***Listing* Program 37.** Fungsi Ambil Gambar

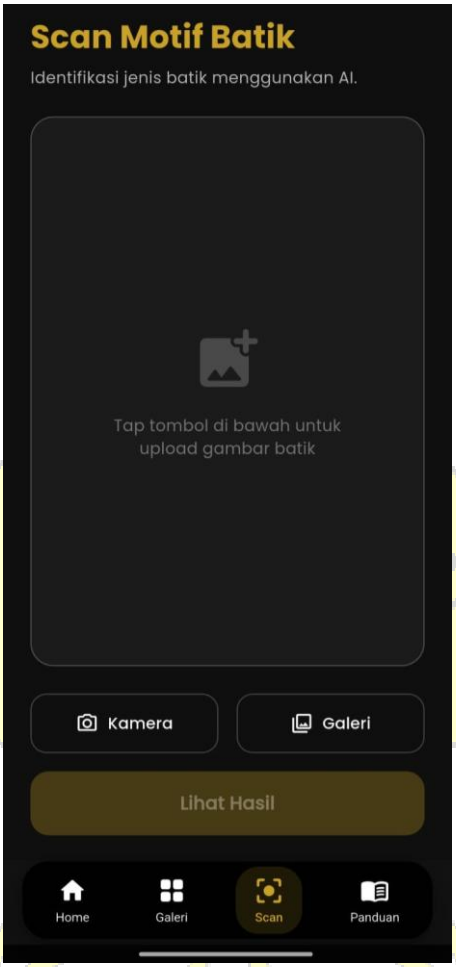
Pada *Listing* Program 37, akan dipanggil fungsi ambil gambar yang digunakan untuk melakukan inisialisasi model dan memilih gambar dari perangkat. Pada fungsi *initState()*, method *_tfliteService.initModel()* dipanggil untuk memuat model TensorFlow Lite saat halaman pertama kali dijalankan. Selanjutnya, fungsi *_pickImage()* digunakan untuk mengambil gambar dari sumber tertentu *ImageSource*, seperti kamera atau galeri, menggunakan *ImagePicker*. Parameter *imageQuality*: 85 digunakan untuk mengompresi kualitas gambar agar ukuran file lebih kecil. Jika gambar berhasil dipilih *pickedFile != null*, maka file gambar disimpan ke variabel *_image* menggunakan *setState()* sehingga tampilan aplikasi diperbarui dan gambar dapat ditampilkan pada gambar 42. "Setelah gambar berhasil dipilih dan ditampilkan pada halaman klasifikasi, proses identifikasi motif batik dilakukan dengan menjalankan fungsi prediksi menggunakan model TensorFlow Lite. Implementasi proses klasifikasi ditunjukkan pada *Listing* Program 38.

```
final hasil = await _tfliteService.predict(_image!);

if (hasil != null) {
  Navigator.push(
    context,
    MaterialPageRoute(
      builder: (_) => HasilView(
        image: _image!.path,
        namaMotif: hasil['label'],
        akurasi: hasil['confidence'],
        deskripsi: "Motif terdeteksi: ${hasil['label']}",
        allProbabilities: hasil['allProbabilities'],
      ),
    ),
  );
}
```

***Listing* Program 38.** Proses Klasifikasi dan Navigasi

Listing Program 35 menunjukkan proses klasifikasi motif batik dan navigasi ke halaman hasil. Setelah pengguna memilih gambar, aplikasi memanggil fungsi *_tfliteService.predict(_image!)* untuk melakukan inferensi menggunakan model TensorFlow Lite. Jika hasil prediksi tidak bernilai *null*, aplikasi akan berpindah ke halaman HasilView menggunakan *Navigator.push()*. Data yang dikirim meliputi path gambar, nama motif hasil prediksi, nilai *confidence*, deskripsi singkat, serta seluruh probabilitas tiap kelas untuk ditampilkan pada halaman hasil klasifikasi. Hasil dari halaman klasifikasi tersebut dapat dilihat pada gambar 42.



Gambar 42. Tampilan Halaman Klasifikasi

Gambar 42 menunjukkan tampilan Halaman Klasifikasi yang berisi area pratinjau gambar, tombol Kamera untuk mengambil foto, tombol Galeri untuk memilih gambar dari perangkat, serta tombol Lihat Hasil untuk menjalankan proses klasifikasi motif batik. Pada bagian bawah halaman juga terdapat *navigation bar* untuk berpindah menu.

5) Halaman Hasil Klasifikasi

```
final sortedLabels = hasil.allProbabilities.entries.toList()
  ..sort((a, b) => b.value.compareTo(a.value));

Text(
  hasil.label,
  style: const TextStyle(fontWeight: FontWeight.bold),
);
```

```
Text(
  "Akurasi ${(hasil.confidence * 100).toStringAsFixed(1)}%",
);

Column(
  children: sortedLabels.map((entry) {
    return Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text("${entry.key} - ${(entry.value * 100).toStringAsFixed(1)}%"),
        LinearProgressIndicator(value: entry.value),
      ],
    );
  }).toList(),
);

Text(
  hasil.description,
);
```

***Listing* Program 39.** Implementasi Hasil Klasifikasi

Listing Program 39 menunjukkan implementasi tampilan hasil klasifikasi motif batik. Data probabilitas dari seluruh kelas diambil dari *hasil.allProbabilities* kemudian diurutkan menurun menggunakan *sortedLabels*. Hasil prediksi utama ditampilkan melalui *hasil.label* dan nilai *confidence* ditampilkan dalam bentuk persentase. Selanjutnya, setiap kelas beserta probabilitasnya divisualisasikan menggunakan *LinearProgressIndicator*. Pada bagian akhir, *hasil.description* ditampilkan sebagai deskripsi motif batik yang terdeteksi. Tampilan halaman hasil klasifikasi tersebut dapat dilihat pada gambar 43.



Gambar 43. Tampilan Halaman Hasil Klasifikasi

Gambar 43 menunjukkan tampilan Halaman Hasil Klasifikasi yang berisi judul halaman, gambar batik yang dianalisis, nama motif dengan probabilitas tertinggi beserta deskripsi singkat, serta rincian nilai probabilitas untuk seluruh kelas motif batik yang tersedia.

6) Halaman Panduan

```
Widget _buildPageView() {
    return Expanded(
      child: PageView.builder(
        controller: pageController,
        itemCount: panduanList.length,
        onPageChanged: (index) => setState(() => currentPage = index),
        itemBuilder: (context, index) {
          final panduan = panduanList[index];
          return _buildPanduanCard(panduan);
        },
```

```
      ),
    );
  }
```

***Listing* Program 40.** Implementasi Halaman Panduan

Tampilan panduan dalam bentuk halaman yang dapat digeser menggunakan *PageView.builder*. *Widget Expanded* berfungsi agar *PageView* mengisi ruang yang tersedia pada layout. *PageView.builder* menampilkan data berdasarkan jumlah item pada *panduanList*, serta menggunakan *pageController* untuk mengatur perpindahan halaman. Ketika pengguna berpindah halaman, fungsi *onPageChanged* akan memperbarui nilai *currentPage* menggunakan *setState*, sehingga aplikasi dapat mengetahui halaman panduan yang sedang aktif. Setiap halaman kemudian ditampilkan dalam bentuk kartu melalui fungsi *_buildPanduanCard(panduan)*.

```
Column(
  crossAxisAlignment: CrossAxisAlignment.start,
  children: [
    Text(
      panduan.title,
      style: const TextStyle(
        fontSize: 20,
        fontWeight: FontWeight.bold,
        color: Color(0xFFC9A227),
      ),
    ),
    const SizedBox(height: 12),
    Text(
      panduan.description,
      style: TextStyle(
        fontSize: 14,
        color: Colors.white.withOpacity(0.8),
        height: 1.5,
      ),
    ),
  ],
);
```

***Listing* Program 41.** Tampilan Detail Panduan

Listing Program 41 menunjukkan tampilan detail isi panduan dalam bentuk *Column* yang menampilkan judul dan deskripsi (*panduan.description*). Judul dibuat

lebih menonjol menggunakan pengaturan ukuran font dan ketebalan huruf, sedangkan deskripsi ditampilkan dengan ukuran lebih kecil agar mudah dibaca. Hasil implementasi halaman panduan tersebut dapat dilihat pada Gambar 44.



Gambar 44. Tampilan Halaman Panduan

Gambar 44 menunjukkan tampilan Halaman Panduan yang berisi judul dan deskripsi singkat, ilustrasi penggunaan aplikasi, serta langkah-langkah penggunaan dalam bentuk teks. Pada bagian bawah halaman juga terdapat *navigation bar* untuk berpindah menu.

4.4. *Testing*

Pengujian fungsionalitas pada aplikasi mBatiC dilakukan menggunakan metode *Black Box Testing.* Pengujian *Blackbox* dilakukan untuk menguji fungsionalitas aplikasi tanpa melihat struktur internal kode program. Pengujian ini berfokus pada kesesuaian antara input yang diberikan dengan output yang

dihasilkan oleh sistem. Tujuan dari pengujian *Blackbox* adalah untuk memastikan bahwa setiap fitur dalam aplikasi mBatic dapat berjalan dengan baik sesuai dengan kebutuhan dan spesifikasi yang telah dirancang. Hasil pengujian dari *blackbox testing* dapat dilihat pada tabel 24.

Tabel 24. Hasil Pengujian *Blackbox*

Kode	Skenario Pengujian	Hasil yang diharapkan	Hasil Pengujian
URS.001, SRS.001, SRS.009	Pengguna membuka fitur kamera untuk klasifikasi, lalu sistem memeriksa izin kamera.	Sistem meminta izin kamera jika belum diberikan. Jika izin diberikan, kamera aktif dan siap digunakan.	Sesuai
URS.002, SRS.002, SRS.009	Pengguna memilih fitur upload dari galeri tanpa izin akses media penyimpanan.	Sistem meminta izin akses media penyimpanan terlebih dahulu.	Sesuai
URS.002, SRS.002	Pengguna mengunggah gambar batik dari galeri setelah izin diberikan.	Sistem berhasil mengambil gambar dari galeri dan menampilkannya pada halaman klasifikasi.	Sesuai
URS.003, SRS.006	Pengguna melakukan klasifikasi menggunakan kamera atau galeri.	Sistem menampilkan hasil prediksi berupa label motif batik dan nilai *confidence score* tertinggi.	Sesuai
URS.004, SRS.003	Pengguna membuka menu galeri motif batik pada aplikasi.	Sistem menampilkan daftar motif batik beserta informasi detail dan referensi gambar yang tersedia di *database* lokal.	Sesuai
URS.005, SRS.004, SRS.007	Pengguna menjalankan klasifikasi dalam kondisi perangkat tanpa koneksi internet.	Sistem tetap dapat menjalankan inferensi model dan menampilkan hasil prediksi secara *offline*.	Sesuai
URS.006	Pengguna membuka fitur kamera untuk klasifikasi motif batik.	Sistem menampilkan petunjuk visual (panduan posisi) agar pengguna dapat memposisikan batik dengan tepat.	Sesuai
SRS.004	Sistem dijalankan pertama kali dan melakukan proses load	Model berhasil dimuat dan aplikasi siap digunakan untuk klasifikasi.	Sesuai

Kode	Skenario Pengujian	Hasil yang diharapkan	Hasil Pengujian
	model TensorFlow Lite (.tflite).		
SRS.005	Pengguna memilih gambar dengan resolusi besar, lalu sistem melakukan *preprocessing*.	Sistem mengubah ukuran citra sesuai *input* model dan melakukan normalisasi piksel sebelum klasifikasi.	Sesuai
SRS.008	Pengguna mengunggah gambar dengan format .jpg/.jpeg/.png.	Sistem menerima file dan melanjutkan proses klasifikasi.	Sesuai
SRS.010	Pengguna mengunggah file rusak atau gambar tidak valid sehingga inferensi gagal.	Sistem menampilkan pesan kesalahan bahwa gambar tidak dapat diproses atau terjadi kegagalan inferensi.	Sesuai
SRS.006	Sistem menghasilkan output probabilitas beberapa kelas.	Sistem menampilkan hanya label dengan probabilitas tertinggi sebagai hasil utama.	Sesuai
URS.003, SRS.006	Pengguna melakukan klasifikasi beberapa kali dengan gambar berbeda.	Sistem menampilkan hasil prediksi terbaru sesuai gambar terakhir yang diproses.	Sesuai

Berdasarkan hasil *Black Box Testing* pada Tabel 24, dapat disimpulkan bahwa seluruh fitur utama aplikasi mBatiC telah berfungsi dengan baik sesuai kebutuhan pengguna dan spesifikasi sistem. Aplikasi mampu menangani izin akses kamera dan penyimpanan, melakukan klasifikasi citra melalui kamera maupun galeri, serta menampilkan hasil prediksi motif dengan c*onfidence score* tertinggi. Dengan demikian, aplikasi mBatiC dinyatakan telah memenuhi aspek fungsionalitas dan siap digunakan sebagai media klasifikasi dan edukasi motif Batik Pekalongan.

BAB V
KESIMPULAN DAN SARAN

5.1. Kesimpulan

Berdasarkan penelitian yang sudah dilakukan, dapat ditarik kesimpulan sebagai berikut:

1. Penelitian ini berhasil mengembangkan aplikasi edukasi berbasis *mobile* untuk mengklasifikasikan 7 motif batik Pekalongan dan 1 kelas non-batik menggunakan arsitektur CNN MobileNetV2. Proses pembangunan aplikasi dimulai dari tahap analisis kebutuhan, perancangan sistem, implementasi sistem yang mengintegrasikan model CNN MobileNetV2, hingga pengujian fungsionalitas menggunakan metode *blackbox testing*. Aplikasi dikembangkan menggunakan *framework* Flutter untuk sisi antarmuka pengguna, SQLite digunakan sebagai basis data *offline* yang disimpan langsung di dalam aplikasi dan Python dengan *framework* TensorFlow dan Keras untuk pelatihan dan evaluasi model.
2. Berdasarkan hasil pengujian model, CNN MobileNetV2 mampu mencapai performa klasifikasi yang baik pada kedua skenario yang diujikan. Model terbaik diperoleh pada skenario CNN dengan penerapan SMOTE, yang menghasilkan nilai *validation accuracy* tertinggi sebesar 0,8866 pada *epoch* ke-27, serta menunjukkan nilai *validation loss* yang lebih rendah dibandingkan model lainnya. Sementara itu, model CNN dengan augmentasi data dan *class weighting* memperoleh *validation accuracy* terbaik sebesar 0,8557 sehingga performanya berada di bawah model CNN + SMOTE. Keunggulan model CNN + SMOTE juga terkonfirmasi pada tahap pengujian menggunakan data *testing*, di mana model ini memperoleh nilai akurasi (0.8475) dan *F1-Score* (0.8336) yang lebih tinggi dibandingkan akurasi (0.8305) dan *F1-Score* (0.8309) model CNN dengan augmentasi dan *class weight*. Dengan demikian, model MobileNetV2 yang dikombinasikan dengan SMOTE terbukti lebih optimal dalam meningkatkan performa klasifikasi motif batik Pekalongan secara keseluruhan.

5.2. Saran

Berdasarkan penelitian yang telah dilakukan, berikut beberapa rekomendasi yang dapat diterapkan untuk meningkatkan kinerja model:

1. Penelitian selanjutnya disarankan untuk menggunakan jumlah data yang lebih besar dan lebih seimbang pada setiap kelas, terutama pada kelas dengan jumlah sampel yang sedikit, agar model dapat belajar lebih optimal dan meningkatkan kemampuan generalisasi.
2. Pengembangan model dapat dilakukan dengan mencoba arsitektur CNN lain yang lebih dalam atau lebih baru, seperti EfficientNet atau ResNet, untuk membandingkan performa terhadap MobileNetV2.
3. Disarankan untuk melakukan optimasi *hyperparameter* secara lebih mendalam, seperti *learning rate*, *batch size*, dan *fine-tuning* agar diperoleh hasil yang lebih optimal.

DAFTAR PUSTAKA

Abdi, A. M. (2024). Tinjauan Langkah Pengujian Perangkat Lunak Dengan Metode Black Box Menggunakan Teknik Equivalance Partitions. *JISCO (Journal of Information System and Computing) ISSN*, *2*(2), 59–65. https://doi.org/https://doi.org/10.30631/jisco.v2i2.103

Adi Kurniyanti, V., & Murdiani, D. (2022). Perbandingan Model Waterfall Dengan Prototype Pada Pengembangan System Informasi Berbasis Website. *Jurnal Syntax Fusion*, *2*(08), 669–675. https://doi.org/10.54543/fusion.v2i08.210

Alnuaimi, A. F. A. H., & Albaldawi, T. H. K. (2024). An Overview of Machine Learning Classification Techniques. *BIO Web of Conferences*, *97*, 00133. https://doi.org/10.1051/bioconf/20249700133

Alzubaidi, L., Zhang, J., Humaidi, A. J., Al-Dujaili, A., Duan, Y., Al-Shamma, O., Santamaría, J., Fadhel, M. A., Al-Amidie, M., & Farhan, L. (2021). Review of deep learning: concepts, CNN architectures, challenges, applications, future directions. *Journal of Big Data*, *8*(1), 53. https://doi.org/10.1186/s40537-021-00444-8

Ambarwari, A., Husni, M., & Mahayana, D. (2023). Perkembangan Metode Klasifikasi Citra Penginderaan Jauh dalam Perspektif Revolusi Ilmiah Thomas Kuhn. *Jurnal Filsafat Indonesia*, *6*. https://doi.org/https://doi.org/10.23887/jfi.v6i3.53865

Amylia, A., Andini, A. P., MutiaA, M. R., & Najihah, N. N. (2025). Klasifikasi Kendaraan Menggunakan Convolutional Neural Network untuk Sistem Gerbang Tol Otomatis di Kota Pintar. *JURNAL KOMPUTER DAN TEKNOLOGI INFORMASI*, *3*(1). https://doi.org/10.26714/jkti.v3i1.13957

Andrianto, M. R., Firdaus, F., Pratama, E., Purnama, R. S., & Puspaningrum, E. Y. (2025). Klasifikasi Motif Batik Tradisional Mataraman Menggunakan CNN. *Seminar Nasional Informatika Bela Negara (SANTIKA)*. https://doi.org/https://doi.org/10.33005/santika.v5i1.567

Angelika, A. (2024). User Perception And Preference Study On Batik Motifs For Use In Upcycled Products. *Proceeding Bali-Bhuwana Waskita: Global Art Creativity Conference*, *4*, 179–191. https://doi.org/10.31091/bbwp.v4i1.477

Anhar, A., & Putra, R. A. (2023). Perancangan dan Implementasi Self-Checkout System pada Toko Ritel menggunakan Convolutional Neural Network (CNN). *ELKOMIKA: Jurnal Teknik Energi Elektrik, Teknik Telekomunikasi, & Teknik Elektronika*, *11*(2), 466. https://doi.org/10.26760/elkomika.v11i2.466

Aurellia, A. (2025). Pemanfaatan UML Dalam Perancangan Sistem Informasi Produk Kreatif Daur Ulang Sampah Berbasis Web. *Jurnal Informatika dan Teknik Elektro Terapan*, *13*(3S1). https://doi.org/10.23960/jitet.v13i3S1.8073

Azmi, K., Defit, S., & Sumijan. (2023). Implementasi Convolutional Neural Network (CNN) Untuk Klasifikasi Batik Tanah Liat Sumatera Barat. *Jurnal Unitek*, *16*(1). https://doi.org/https://doi.org/10.52072/unitek.v16i1.504

Bagaskara, T. O. (2023). Model Prediksi Menggunakan Teknik Machine Learning untuk Penjualan terhadap Produksi Kain Jumputan pada Pengerajin Batiq Colet Jumputan Palembang. *PETIR*, *16*(2), 189–199. https://doi.org/10.33322/petir.v16i2.2187

Bahmani, L., Minaei, S., Mahdavian, A., Banakar, A., & Firouz, M. S. (2025). A comparative study of convolutional neural networks and traditional feature extraction techniques for adulteration detection in ground beef. *Sensing and Bio-Sensing Research*, *48*, 100774. https://doi.org/10.1016/j.sbsr.2025.100774

Banin, M. M. (2021). Perancangan Sistem Informasi Untuk Mengontrol Sistem Pembelian, Persediaan Dan Penjualan Dengan Menggunakan Metode System Development Life Cycle (SDLC). *Integrasi : Jurnal Ilmiah Teknik Industri*, *6*(2), 89. https://doi.org/10.32502/js.v6i2.3994

Barat, A. I. R. H., Astuti, W. S., Hartama, D., Windarto, A. P., & Wanto, A. (2025). Enhanced Flower Image Classification Using MobileNetV2 With Optimized Performance. *JITK (Jurnal Ilmu Pengetahuan dan Teknologi Komputer)*, *11*(1), 180–191. https://doi.org/10.33480/jitk.v11i1.6497

Buda, M., Maki, A., & Mazurowski, M. A. (2018). A systematic study of the class imbalance problem in convolutional neural networks. *Neural Networks*, *106*, 249–259. https://doi.org/10.1016/j.neunet.2018.07.011

Chandra, T., & Ari Dartono, F. (2021). Palihan Nagari Sebagai Sumber Ide Dalam Penciptaan Motif Batik Pagi-Sore. *Jurnal Dimensi*, *2*. https://doi.org/https://doi.org/10.5583/jurnaldimensi.v2i02.35

Deni Akbar, M., Martanto, & Arie Wijaya, Y. (2022). Klasifikasi Motif Batik Jawa Menggunakan Algoritma K-Nearest Neighbors (KNN). *JURSIMA Jurnal Sistem Informasi dan Manajemen*, *10*. https://doi.org/https://doi.org/10.47024/js.v10i2.412

Dewinta Sari Rahmaputri. (2023). Analisis Semiotika Terhadap Keanekaragaman Motif Batik Pekalongan Hasil Akulturasi Budaya. *Jurnal Dimensi Seni Rupa dan Desain*, *20*(1), 91–106. https://doi.org/10.25105/dim.v20i1.16943

Fachrie, M., Musdholifah, A., & Pulungan, R. (2025). Effectiveness of data resampling and ensemble learning in multiclass imbalance learning. *Artificial Intelligence Review*, *58*(12). https://doi.org/10.1007/s10462-025-11357-w

Fahmi, M. N. (2023). Implementasi Mechine Learning menggunakan Python Library : Scikit-Learn (Supervised dan Unsupervised Learning). *Sains Data Jurnal Studi Matematika dan Teknologi*, *1*(2), 87–96. https://doi.org/10.52620/sainsdata.v1i2.31

Faturrahman, R., Hariyani, Y. S., & Hadiyoso, S. (2023). Klasifikasi Jajanan Tradisional Indonesia berbasis Deep Learning dan Metode Transfer Learning. *ELKOMIKA: Jurnal Teknik Energi Elektrik, Teknik Telekomunikasi, & Teknik Elektronika*, *11*(4), 945. https://doi.org/10.26760/elkomika.v11i4.945

Fitri Widiastuti, Setiawati, R., & Yuniarti, Y. (2024). *Batik Sebagai Identitas Lokal: Mengangkat Kembali Motif-Motif Khas KEYWORD Batik Motif Sale value Pangkal Babu*. *4*(6). https://doi.org/10.59818/jpm.v4i6.913

Galih Pramuja Inngam Fanani, Yana Safitri, Muhammad Amirul Mu'min, Setiawan Ardi Wijaya, Tri Stiyo Famuji, & Novi Tristanti. (2025). Pengenalan Citra Batik Tradisional Menggunakan Deep Learning untuk Klasifikasi Motif Daerah. *Scientific : Journal of Computer Science and Informatics*, *2*(1). https://doi.org/https://doi.org/10.34304/scientific.v2i1.336

Ghandi, S., Ma'ruf Nugroho, I., & Ramadhan, Y. R. (2024). Penerapan Metode Convolutional Neural Network (CNN) Dalam Aplikasi Pendeteksi Penyakit Daun Tanaman Kentang Berbasis Android. *Jurnal Mahasiswa Teknik Informatika*, *8*(5). https://doi.org/10.36040/jati.v8i5.10769

Indraswari, R., Herulambang, W., & Rokhana, R. (2022). *Deteksi Penyakit Mata Pada Citra Fundus Menggunakan Convolutional Neural Network (CNN) Ocular Disease Detection on Fundus Images Using Convolutional Neural Network (CNN)*. *21*(2), 378–389. https://doi.org/https://doi.org/10.33633/tc.v21i2.6162

Joloudari, J. H., Marefat, A., Nematollahi, M. A., Oyelere, S. S., & Hussain, S. (2023). Effective Class-Imbalance Learning Based on SMOTE and Convolutional Neural Networks. *Applied Sciences (Switzerland)*, *13*(6). https://doi.org/10.3390/app13064006

Kalyani, J. L., Clement, N., Felicia, A. W., Jeson, A. D., Thaddeus, K. A., Wilsen, S., & Rahmi, Y. N. (2023). Penggunaan Bahasa Pemrograman Python Untuk Memvisualisasikan Data Peluang Selamat Dari Kecelakaan Titanic. *Jurnal Publikasi Teknik Informatika*, *2*(2), 66–79. https://doi.org/10.55606/jupti.v2i2.1735

Kurniawan, K., Ceasaro, B., & Sucipto, S. (2024). Perbandingan Fungsi Aktivasi Untuk Meningkatkan Kinerja Model LSTM Dalam Prediksi Ketinggian Air Sungai. *Jurnal Edukasi dan Penelitian Informatika (JEPIN)*, *10*(1), 134. https://doi.org/10.26418/jp.v10i1.72866

Kusrianto, A. (2024). *Batik Filosofi, Motif dan Kegunaan*. Penerbit Andi. https://books.google.co.id/books?id=_5P3EAAAQBAJ

Lisbijanto, H. (2013). *Batik*. Graha Ilmu.

Marques, C. S., Malafaia, E., Pereira, S., Santos, V. F., & Dufourq, E. (2025). A Review of Machine Learning Applications For Identification And Classification Problems An Paleontology. *Ecological Informatics*, *91*, 103329. https://doi.org/10.1016/j.ecoinf.2025.103329

Maziyah, S., & Alamsyah. (2023). Perjalanan Panjang Patola Menjadi Jlamprang: Transformasi Motif Tenun Menjadi Motif Batik. *KALPATARU*, *30*(1), 61–74. https://ejournal.brin.go.id/kalpataru/article/view/2702

Maziyah, S., Alamsyah, A., & Tri Lestari, D. (2021). Jlamprang Batik Motive : Representation of the Natural and Cultural Environment of Pekalongan City. *E3S Web of Conferences*, *317*, 01005. https://doi.org/10.1051/e3sconf/202131701005

Mbaba, P., Anwar, K., Rahayu, S., & Eka Kartawati, A. (2022). Penerapan Metode Convolution Neural Networks Untuk Mengidentifikasi Wajah Kelelahan. Dalam *JNATIA* (Vol. 1, Nomor 1). https://doi.org/https://doi.org/10.24843/JNATIA.2022.v01.i01.p12

Misela, M., Wona, A., Aulia Asyifa, S., Virgianti, R., Hamid, M. N., Handoko, I. M., Wayan, N., Septiani, P., & Lestari, M. (2023). Klasifikasi Batik Indonesia Menggunakan Convolutional Neural Network (CNN). Dalam *172 JURTI* (Vol. 7, Nomor 2). https://doi.org/https://doi.org/10.30872/jurti.v7i2.13694

Muslim, M., Puspita Sari, R., & Rahmayuda, S. (2022). Implementasi Framework Flutter Pada Sistem Informasi Perpustakaan Masjid (Studi Kasus: Masjid di Kota Pontianak). *Coding : Jurnal Komputer dan Aplikasi*, *10*(1). https://doi.org/https://doi.org/10.26418/coding.v10i01.52178

Musman, A., & Arini, A. B. (2011). *Batik: warisan adiluhung Nusantara*. G-Media. https://books.google.co.id/books?id=HbReMwEACAAJ

Nafisah, B., Mayliana, E., & Sukanadi, I. M. (2021). Batik Buketan Motif Bunga Nasional Indonesia Dan Penerapannya Dalam Busana Bernuansa Vintage. *Style : Journal of Fashion Design*, *1*(1), 24. https://doi.org/10.26887/style.v1i1.2109

Nesta Suandana, I., Asriyanik, A., & Apriandari, W. (2024). Pemanfaatan CNN (Convolution Neural Network) Dan MobileNet V2 Dalam Klasifikasi Rempah-Rempah Lokal Di Indonesia. *JATI (Jurnal Mahasiswa Teknik Informatika)*, *8*(5), 10109–10116. https://doi.org/10.36040/jati.v8i5.10873

Ni'mah, Q. M., & Dartono, F. A. (2021). Perancangan Batik Kontemporer Motif Pagi Sore Untuk Busana Ready To Wear Dengan Sumber Ide "Bohemian." *Ekspresi Seni : Jurnal Ilmu Pengetahuan dan Karya Seni*, *23*(2), 407. https://doi.org/10.26887/ekspresi.v23i2.2018

Noe, A., Hidayat, A., Yogaswara, N., Handayani, D., Kustanto, P., & Hartanti, D. (2025). Implementasi Algoritma Random Forest Dalam Sistem Seleksi Karyawan Terbaik. *Jurnal Manajemen Informatika Jayakarta*, *5*(3), 263–274. https://doi.org/10.52362/jmijayakarta.v5i3.2081

Nuraryo, I., Vita, E., Meisyaroh, S., Yogananta, J. M., Savina, N., & Utari, A. D. (2023). "BATIK" Cultural Heritage to Construct City Branding: A Case of Pekalongan. *Proceedings of the 3rd International Conference on Innovations in Social Sciences Education and Engineering (ICoISSEE-3)*. https://conference.loupiasconference.org/index.php/icoissee3/article/view/437

Nurhidayat, R., & Dewi, K. E. (2023). Penerapan Algoritma K-Nearest Neighbor Dan Fitur Ekstraksi N-Gram Dalam Analisis Sentimen Berbasis Aspek. *Komputa : Jurnal Ilmiah Komputer dan Informatika*, *12*(1), 91–100. https://doi.org/10.34010/komputa.v12i1.9458

Paramanandana, S., Lutfi, I., & Ayundasari, L. (2021). Penetrasi Jepang Dan Dampaknya Terhadap Batik Pekalongan, 1930-1945. *AGASTYA: JURNAL SEJARAH DAN PEMBELAJARANNYA*, *11*(2), 120–135. https://doi.org/10.25273/ajsp.v11i2.8948

Prasetyo, D. (2024). Rancang Bangun Sistem Informasi Manajemen Persediaan Barang Menggunakan Metode System Development Life Cycle (SDLC) Di SMA Negeri 1 SoE. *Jurnal Digit*, *14*(2), 100. https://doi.org/10.51920/jd.v14i2.400

Ramadhani, K. F., & Tarigan, M. (2025). Implementasi Metode Convolutional Neural Network Untuk Klasifikasi Penyakit Daun Mangga Menggunakan Arsitektur EfficientnetV2-S dan ResNet50. *JATI (Jurnal Mahasiswa Teknik Informatika)*, *9*(3). https://doi.org/https://doi.org/10.36040/jati.v9i3.13571

Raup, A., Ridwan, W., Khoeriyah, Y., Supiana, S., & Zaqiah, Q. Y. (2022). Deep Learning dan Penerapannya dalam Pembelajaran. *JIIP - Jurnal Ilmiah Ilmu Pendidikan*, *5*(9), 3258–3267. https://doi.org/10.54371/jiip.v5i9.805

Razali, M. N., Arbaiy, N., Lin, P. C., & Ismail, S. (2025). Optimizing Multiclass Classification Using Convolutional Neural Networks with Class Weights and Early Stopping for Imbalanced Datasets. *Electronics (Switzerland)*, *14*(4). https://doi.org/10.3390/electronics14040705

Riska Rismaya, Dwi Yuniarto, & David Setiadi. (2025). Penerapan Algoritma Machine Learning dalam Prediksi Prestasi Akademik Mahasiswa. *Router : Jurnal Teknik Informatika dan Terapan*, *3*(1), 15–23. https://doi.org/10.62951/router.v3i1.389

Rozi, M. F., Siregar, H., Hambali, Y. A., & Rasim, R. (2025). Rancang Bangun Sistem Manajemen Akademik Mahasiswa Berbasis Mobile Multiplatform Menggunakan Flutter. *Jurnal Komputer Teknologi Informasi Sistem Informasi (JUKTISI)*, *4*(2), 459–468. https://doi.org/10.62712/juktisi.v4i2.436

Salam, A., Yusliana Bakti, R., Lukman, L., & Irhamna Rahman, F. (2024). Penerapan Algoritma Mobilenet Single Shot Detector untuk Deteksi Api dan Asap Berpotensi Kebakaran pada Citra Hutan. *Arus Jurnal Sains dan Teknologi (AJST)*, *2*(2). https://doi.org/https://doi.org/10.57250/ajst.v2i2.663

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 4510–4520. https://doi.org/https://doi.org/10.48550/arXiv.1801.04381

Setyawati, N. A., & Adrimuna, S. A. (2022). Perancangan Sistem Informasi Pemesanan Dan Pembayaran Jasa Website. *Jurnal Nasional Komputasi dan Teknologi Informasi (JNKTI)*, *5*(6), 955–959. https://doi.org/10.32672/jnkti.v5i6.5385

Shatravin, V., Shashev, D., & Shidlovskiy, S. (2023). Implementation of the SoftMax Activation for Reconfigurable Neural Network Hardware Accelerators. *Applied Sciences*, *13*(23), 12784. https://doi.org/10.3390/app132312784

Siska Narulita, Ahmad Nugroho, & M. Zakki Abdillah. (2024). Diagram Unified Modelling Language (UML) untuk Perancangan Sistem Informasi Manajemen Penelitian dan Pengabdian Masyarakat (SIMLITABMAS). *Bridge : Jurnal publikasi Sistem Informasi dan Telekomunikasi*, *2*(3), 244–256. https://doi.org/10.62951/bridge.v2i3.174

Supriono, Y. P. (2024). *Ensiklopedia The Heritage Of Batik, Identitas Pemersatu Kebanggaan Bangsa*. Penerbit Andi. https://books.google.co.id/books?id=Q5T3EAAAQBAJ

Tjandrawibawa, P., & Tanzil, M. Y. (2022). Perancangan Motif Berbasis Wastra Batik Jawa Hokokai Untuk Produk Fesyen Generasi Milenial. *JURNAL RUPA*, *7*(2), 107. https://doi.org/10.25124/rupa.v7i2.4920

Wahyudi, T., Supriyanta, S., & Faqih, H. (2021). Pengembangan Sistem Informasi Presensi Menggunakan Metode Waterfall. *Indonesian Journal on Software Engineering (IJSE)*, *7*(2), 120–129. https://doi.org/10.31294/ijse.v7i2.11091

Wayahdi, M. R., & Ruziq, F. (2023). Pemodelan Sistem Penerimaan Anggota Baru dengan Unified Modeling Language (UML) (Studi Kasus: Programmer Association of Battuta). *Jurnal Minfo Polgan*, *12*(1), 1514–1521. https://doi.org/10.33395/jmp.v12i1.12870

Wibowo, A. R., Astuti, Y. P., Kartikadarma, E., Subhiyakto, E. R., Anisa, N., Winarsih, S., & Rohman, M. S. (2024). Penerapan Metode Naïve Bayes Classifier untuk Klasifikasi Sentimen pada Judul Berita. *LogicLink : Journal of Artificial Intelligence and Multimedia in Informatics*, *1*(1), 1–12. https://doi.org/https://doi.org/10.28918/logiclink.v1i1.7684